Registered Attribute-Based Encryption with Publicly Verifiable Certified Deletion, Everlasting Security, and More

Shayeef Murshid, Ramprasad Sarkar, Mriganka Mandal

Cryptology and Security Research Unit
R. C. Bose Centre for Cryptology and Security
Indian Statistical Institute
203 B.T. Road, Kolkata 700108, India
shayeef.murshid91@gmail.com, {rpsarkar_p,mriganka}@isical.ac.in

March 8, 2026

Abstract. Certified deletion ensures that encrypted data can be irreversibly deleted, preventing future recovery even if decryption keys are later exposed. Although existing works have achieved certified deletion across various cryptographic primitives, they rely on central authorities, leading to inherent escrow vulnerabilities. This raises the question of whether certified deletion can be achieved in decentralized frameworks such as Registered Attribute-Based Encryption (RABE) that combines fine-grained access control with user-controlled key registration. This paper presents the *first* RABE schemes supporting certified deletion and certified everlasting security. Specifically, we obtain the following:

- We first design a privately verifiable RABE with Certified Deletion (RABE-CD) scheme by combining our newly proposed shadow registered ABE (Shad-RABE) with one-time symmetric key encryption with certified deletion.
- We then construct a publicly verifiable RABE-CD scheme using Shad-RABE, witness encryption, and one-shot signatures, allowing any party to validate deletion certificates without accessing secret keys.
- We also extend to privately verifiable RABE with Certified Everlasting Deletion (RABE-CED) scheme, integrating quantum-secure RABE with the certified everlasting lemma. Once a certificate is produced, message privacy becomes information-theoretic even against unbounded adversaries.
- We finally realize a publicly verifiable RABE-CED scheme by employing digital signatures for the BB84 states, allowing universal verification while ensuring that deletion irreversibly destroys information relevant to decryption.

Contents

1 Introduction

Certified deletion ensures that encrypted data, once deleted, becomes permanently and irretrievably unrecoverable. Achieving this property is a fundamental challenge at the intersection of classical cryptography and quantum information. In purely classical settings, this problem is intractable: digital information can be copied arbitrarily and stored indefinitely, making it impossible to ensure the destruction of every copy. An adversary may retain hidden replicas and later exploit leaked decryption keys or future advances in cryptanalysis to recover the underlying plaintext. Thus, within the classical cryptographic paradigm, certified deletion is widely regarded as unattainable.

Quantum Certified Deletion. Quantum mechanics provides a framework for addressing this limitation through the no-cloning theorem [WZ09], which forbids the duplication of arbitrary quantum states. This intrinsic unclonability sharply distinguishes quantum information from its classical counterpart and enables cryptographic guarantees unattainable in classical settings. Exploiting this property, in 2020, Broadbent *et al.* [BI20] first introduced the quantum encryption with certified deletion protocol, wherein classical messages are encoded into quantum ciphertexts that cannot be copied. In this setting, a recipient of a quantum ciphertext can either decrypt it to recover the message or perform a deletion procedure that produces a verifiable certificate for deletion. Once deleted, the message becomes irretrievable, even if the decryption key is later revealed. This paradigm continues a broader line of quantum-enabled cryptographic primitives that leverage physical unclonability, including quantum key distribution [BB14], quantum money [Wie83], and quantum secret sharing [HBB99].

Since its inception, the notion of certified deletion has progressively been extended to a diverse set of cryptographic primitives [HMNY21, BGG+23, BK23, BR24]. In particular, Bartusek *et al.* [BK23] showed that for any encryption $\mathsf{X} \in \{$public-key, attribute-based, fully homomorphic, witness, timed release$\}$, a generic compiler can transform any (post-quantum) X-encryption scheme into one supporting certified deletion. In addition, they presented a compiler that upgrades statistically binding commitments into statistically binding commitments with certified everlasting hiding. Collectively, these results underscore the increasing generality of certified deletion and highlight its emerging role as a foundational primitive in hybrid classical-quantum cryptographic constructions.

Challenges in Centralization. Despite these advances, most existing general-purpose frameworks for certified deletion, including those of [HMNY21, BGG+23, BK23, BR24], rely crucially on *centralized trust*. In nearly all known constructions, a designated authority is entrusted with generating and distributing cryptographic keys. Such a trust model inherently introduces key-escrow vulnerabilities: the security of the entire system collapses if the authority is compromised, reintroducing the same single point of failure that traditional cryptography has long struggled to eliminate. Eliminating this reliance on centralized trust is there-

fore not just an optimization in terms of security but a prerequisite for deploying certified deletion in realistic large-scale systems.

A notable exception to this centralized approach is the foundational work of Broadbent *et al.* [BI20], whose protocol operates without a central authority. However, their construction is inherently restricted to a one-to-one communication model and cannot be readily extended to multi-user or public-key settings without a central authority. This limitation exposes a fundamental gap in the current landscape and motivates the following central question:

Can certified deletion be achieved in advanced encryption schemes without relying on a trusted authority?

Registered Attribute-Based Encryption. Attribute-Based Encryption (ABE) [SW05] extends the traditional public-key encryption paradigm that supports fine-grained access control over encrypted data. In a ciphertext-policy ABE (CP-ABE) scheme, users are issued secret keys associated with attribute sets, while ciphertexts are generated with embedded access policies. A user can decrypt a ciphertext if and only if the attributes bound to their secret key satisfy the ciphertext policy. The dual construction, referred to as key-policy ABE (KP-ABE), interchanges the roles of attributes and access policies. However, both the conventional ABE schemes [SW05, GPSW06, GVW15] are inherently centralized: a single authority issues all user secret keys, thereby introducing the same key-escrow vulnerability. Registration-Based Encryption (RBE) [GHMR18] overcomes this limitation by decentralizing key generation, users generate their own key pairs and register only their public keys, while a curator aggregates them into a master public key without holding any private information. This model eliminates the problem of central key escrow while preserving global encryptability. Building on this foundation, Registered Attribute-Based Encryption (RABE) [HLWW23] extends decentralization to attribute-controlled settings: users independently register both their keys and attributes, and encryption policies are evaluated relative to the registered attributes. Recent constructions achieve practicality under bilinear pairings [HLWW23, ZZGQ23] and post-quantum security under lattice assumptions [CHW25, ZZC+25, PST25, WW25].

The convergence of certified deletion with decentralized cryptography naturally raises the question of whether these two paradigms can be integrated without compromising security. Extending certified deletion to RABE would provide an unprecedented combination of properties: decentralized trust (without the key-escrow problem), fine-grained access control, and provable guarantees of irreversible data deletion. This raises the following research question:

Can certified deletion be incorporated into the registered framework?

However, this integration is technically challenging. In RABE, users control their own secret keys, while the curator manages and updates the helper secret key

for users. In contrast, certified deletion must ensure that all the decryption capabilities are irreversibly destroyed. Coordinating these independent components to guarantee complete information destruction is difficult.

Certified Everlasting Security. An extension of certified deletion security is *certified everlasting security*, which guarantees that once deletion certificates are produced, security becomes information-theoretic: even adversaries with unlimited computational power cannot recover the information after deletion. Recent works have demonstrated certified everlasting security across diverse primitives, including commitments [HMNY22], zero-knowledge proofs [HMNY22], garbling schemes [HKM+24], non-committing encryption [HKM+24], predicate encryption [HKM+24], fully-homomorphic encryption [BK23], time-release encryption [BK23], witness encryption [BK23], and functional encryption [HKM+24]. These results suggest that certified everlasting security forms a unifying paradigm for quantum cryptography.

Taken together, these developments lead to the following fundamental research question:

Can we construct a registered attribute-based encryption protocol with certified everlasting security?

1.1 Our Results

We provide affirmative answers to all the above questions. To the best of our knowledge, we are the first to present the lattice-based key-policy registered attribute-based encryption schemes that support both certified deletion and certified everlasting deletion. We summarize our main findings as follows:

We first introduce a new primitive (key-policy) shadow-registered attribute-based encryption (Shad-RABE) with receiver non-committing security, constructed from an indistinguishability obfuscation ($i\mathcal{O}$), zero-knowledge argument (ZKA), and registered attribute-based encryption (RABE) . The Shad-RABE primitive serves as the foundational building block for subsequent constructions. In addition, we show that lattice-based instantiations of Shad-RABE can be obtained by combining the RABE scheme of Champion *et al.* [CHW25], the ZKA of Bitansky *et al.* [BS20], and the $i\mathcal{O}$ framework of Cini *et al.* [CLW25]. A detailed description appears in Section 5.

We also design the Registered Attribute-Based Encryption with Certified Deletion (RABE-CD) under two different verification models. The first construction, referred to as RABE-PriVCD, achieves privately verifiable deletion by combining our proposed Shad-RABE with One-Time Symmetric Key Encryption with Certified Deletion (OTSKE-CD) [BI20]. This combination requires both the underlying RABE functionality and the simulation features required for certified deletion. We also demonstrate a lattice-based instantiation of RABE-PriVCD by

integrating our generic construction with a lattice-based Shad-RABE secure under ℓ-succinct LWE, plain LWE and equivocal LWE assumptions (*cf.* Section 5.3) along with an unconditionally secure OTSKE-CD scheme [BI20]. The resulting protocol achieves certified deletion security, which is discussed in detail in Section 6. The second construction, referred to as RABE-PubVCD, which integrates our proposed Shad-RABE, witness encryption, and one-shot signatures, allowing deletion certificates to be publicly validated by any party without access to secret keys. We instantiate RABE-PubVCD over lattices by combining three central building blocks: the lattice-based Shad-RABE protocol secure under ℓ-succinct LWE, plain LWE and equivocal LWE (*cf.* Section 5.3); extractable witness encryption instantiated from the LWE assumption and its variants [VWW22, Tsa22]; and one-shot signatures [AGKZ20, SZ25]. The resulting scheme presents the lattice-based publicly verifiable RABE-CD design, as shown in Section 7.

We then further extend our framework to RABE with Certified Everlasting Deletion (RABE-CED), which guarantees message privacy even against computationally unbounded adversaries once a valid deletion certificate is issued. In the privately verifiable setting, our construction combines quantum-secure RABE with the certified everlasting lemma [BK23], employing quantum states prepared in random bases together with classical encryption that conceals the basis information. Since generating a deletion certificate requires measurement of these quantum states, the necessary decryption information is irreversibly destroyed, ensuring the indistinguishability of ciphertexts even against unbounded adversaries. The detailed description of this construction is presented in Section 8. In the publicly verifiable setting, we utilize one-time unforgeable signatures for BB84 states, which allow universal verification of deletion certificates. The certified everlasting lemma guarantees that BB84 states $|x\rangle_\theta$ cannot be reconstructed without the bases θ, and any incorrect measurement permanently erases the information about x. This construction is also analyzed in depth in Section 9.

1.2 Related Work

Threshold and Multi-Authority Approaches. The key escrow problem in identity-based encryption [BF01, PS08] and attribute-based encryption [SW05, GPSW06, GVW15] motivated new approaches to reduce centralized trust. An early line of work [BF01, CHSS02, PS08, KG10] applied threshold cryptography, where the master secret key was divided among several authorities, and the generation of keys required their collective participation. Although effective in reducing single point of failure, these schemes remained vulnerable to collusion between authorities. Another direction was multi-authority ABE [Cha07, LCLS08, MKE08, CC09, LW11, RW15, DKW21b, DKW21a, WWW22], where separate attribute domains were controlled by different entities. This distribution limited the power of individual authorities, but did not fundamentally eliminate the systemic risk posed by the compromise of trusted authorities.

Registration-Based Encryption. In 2018, a paradigm shift occurred with the introduction of Registration-Based Encryption (RBE) [GHMR18], which funda-

mentally restructured the trust model rather than redistributing existing vulnerabilities. This line of work gave rise to Registered Attribute-Based Encryption (RABE) [HLWW23], where users generate their own secret/public key pair and register the corresponding public keys along with attribute information through a curator. The curator aggregates these public keys into a succinct master public key, while users employ helper secret keys to enable decryption. Early RABE schemes [HLWW23, ZZGQ23] relied on bilinear pairings over composite order groups and supported policies that can be represented through linear secret sharing, laying the groundwork for decentralized attribute-based access control.

Revocation in Registered ABE. Practical deployment prompted the study of revocation mechanisms within RABE. The first formal analysis introduced Deletable RABE (DRABE) and Directly Revocable RABE (RRABE) [AAH+25], addressing temporal access control in dynamic systems. Later work [LCL+25] expanded this to fine-grained revocation, supporting both file-specific access removal and permanent user deregistration. These advances enhanced the flexibility of the administrator while preserving the decentralized trust foundation of registered frameworks.

Post-Quantum Secure RABE. Significant theoretical progress has extended RABE into the post-quantum setting. A scheme [ZZC+25] supporting unbounded users with a transparent setup was built from the private-coin evasive LWE assumption. It avoids random oracles but relies on complex assumptions that are now under scrutiny [VWW22, BDJ+24, BUW24]. In 2025, Champion *et al.* [CHW25] proposed a key-policy RABE for bounded-depth circuits under the ℓ-succinct LWE assumption in the random oracle model, trading stronger expressiveness for reliance on heuristic models. Recently, Pal *et al.* [PST25] achieved post-quantum secure ciphertext-policy and key-policy RABE schemes and registered predicate encryption, supporting unbounded circuit depth. In parallel, multi-authority extensions [LWW25] introduced independent curators for separate attribute domains, enabling policies across multiple authorities while maintaining the registration-based structure.

Quantum Encryption with Certified Deletion. In parallel, quantum cryptography introduced mechanisms for verifiable data deletion. Quantum encryption with certified deletion [BI20] exploits the no-cloning property of quantum states, ensuring that deleted information cannot be recovered, even by unbounded adversaries. This principle has been extended to public-key encryption, ABE, fully homomorphic encryption, and witness encryption [HMNY21, BGG+23, BK23, BR24], highlighting its wide applicability.

Certified Everlasting Security. The concept of certified deletion has further evolved into certified everlasting security, which ensures that once deletion occurs, privacy persists indefinitely. Examples include certified everlasting commitments and zero-knowledge proofs for QMA [HMNY22], as well as frameworks for all-or-nothing encryption based on BB84 states [BK23]. These results extend naturally to functional encryption, garbled circuits, and non-committing encryption [HKM+24], demonstrating the breadth of everlasting security guarantees.

While many certified deletion systems are limited to private verifiability, where verification of deletion is restricted to the designated parties, practical applications often require publicly verifiable guarantees. Addressing this need, recent works have introduced publicly verifiable certified deletion for public-key encryption, ABE, and fully homomorphic encryption [Por22, KNY23], thereby enabling universal auditability and supporting regulatory compliance in decentralized settings.

2 Technical Overview

This section presents a high-level technical overview of our constructions for registered attribute-based encryption with certified deletion and certified everlasting deletion, considering both privately and publicly verifiable frameworks, depicted in Fig. 1.

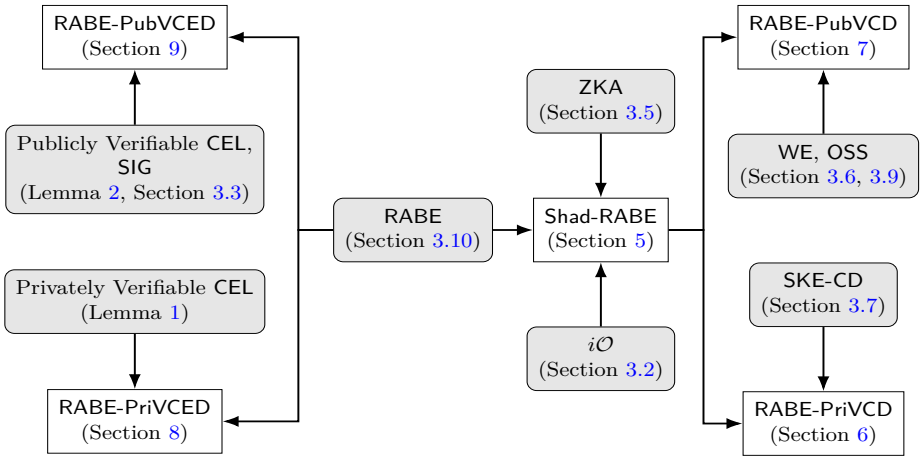

Fig. 1: High-level Overview of Our Constructions

Notations. We begin by introducing some basic notation that will be used throughout this paper. The notation $x \leftarrow X$ denotes that x is selected uniformly at random from a finite set X. If A is an algorithm (which may be classical, probabilistic, or quantum), then $y \leftarrow A(x)$ represents the output of A on input x, and we write $y \leftarrow A(x; r)$ when A uses explicit randomness r. λ and τ denote a security parameter and a policy-family parameter respectively. For a distribution D, we write $x \leftarrow D$ to denote sampling according to D. The assignment $y := z$ indicates that the variable y is set or defined to be equal to z. For a positive integer n, we denote the set $\{1, 2, \ldots, n\}$ by $[n] := \{1, \ldots, n\}$, and $[N]_p$ represents the set of all p-element subsets of $\{1, 2, \ldots, N\}$. The symbol λ denotes the security parameter. For classical strings x and y, their concatenation

is written as $x\|y$. For a bit string $s \in \{0,1\}^n$, both s_i and $s[i]$ refer to its i-th bit. The acronym QPT stands for quantum polynomial-time algorithms, while PPT refers to probabilistic polynomial-time algorithms using classical computation. A function $f : \mathbb{N} \to \mathbb{R}$ is said to be negligible if, for every constant $c > 0$, there exists $\lambda_0 \in \mathbb{N}$ such that $f(\lambda) < \lambda^{-c}$ for all $\lambda > \lambda_0$. We use the notation $f(\lambda) \leq \mathsf{negl}(\lambda)$ to express that f is a negligible function. For a bit string x, we write x_j to mean the j-th bit of x. For bit strings $x, \theta \in \{0,1\}^\ell$, we write $|x\rangle_\theta$ to mean the BB84 state $\bigotimes_{j\in[\ell]} H^{\theta_j} |x_j\rangle$ where H is the Hadamard operator. The trace distance, denoted by $\mathsf{TD}(\cdot,\cdot)$, between two states ρ and σ is defined as $\mathsf{TD}(\rho,\sigma) = \|\rho - \sigma\|_{\mathrm{tr}}$, where $\|A\|_{\mathrm{tr}} \coloneqq \mathrm{Tr}\left(\sqrt{A^\dagger A}\right)$. We use the symbol $\overset{c}{\approx}$ to denote computational indistinguishability between distributions.

2.1 Towards Registered ABE-CD from ABE-CD

In a registered attribute-based encryption scheme, users independently generate key pairs $(\mathsf{pk}, \mathsf{sk})$ using local randomness, ensuring that no trusted party is ever exposed to their secret information. The users then submit only their public keys, together with their policies, to a curator. The curator performs an administrative role: aggregating submitted keys into an evolving master public key mpk and producing helper secret key hsk for registered users. Crucially, the curator never learns the secret keys of users, preserving security even against a malicious or compromised curator.

Verifiable deletion for the ciphertext further augments privacy: a recipient can produce a classical certificate that proves the irreversible destruction of the quantum ciphertext. A valid certificate of deletion guarantees that no adversary can recover the message, even in possession of the secret keys. This verifiable destruction mechanism was introduced in attribute-based encryption with certified deletion ($\mathsf{ABE\text{-}CD}$) of Hiroka *et al.* [HMNY21]. However, integrating such a feature into a registered ABE framework is highly non-trivial, as the curator does not hold any secret information, such as the master secret key msk, in the registered ABE system. Before developing our construction, we recall the definition of $\mathsf{ABE\text{-}CD}$ [HMNY21]:

- **Setup**: Given the security parameter 1^λ, this algorithm outputs a public key pk and a master secret key msk.
- **KeyGen**: This algorithm takes as input msk and a policy P to return a secret key sk_P.
- **Encryption**: Taking as input pk, an attribute X, and a message μ, this algorithm outputs a verification key vk and a ciphertext CT.
- **Decryption**: On input the secret key sk_P and a ciphertext CT, the decryption algorithm outputs a message μ' or $\perp$.
- **Delete**: The delete algorithm takes as input a ciphertext CT and outputs a certification cert.
- **Verify**: Given the verification key vk and the certificate cert, this algorithm outputs $\top$ or $\perp$.

Starting Point. As a warm-up to our actual scheme, in the following, we first describe the attribute-based encryption with certified deletion (ABE-CD) scheme proposed by Hiroka *et al.* [HMNY21]. Their framework is built upon receiver non-committing attribute-based encryption (RNCABE), which serves as a building block for achieving ABE-CD scheme. The main challenge here is to reconcile the fine-grained access control provided by attribute-based encryption with the irreversible deletion guarantees required by certified deletion. To address this, Hiroka *et al.* combine RNCABE with a one-time secret key encryption protocol that supports certified deletion, thus embedding deletion capabilities directly into the attribute-based paradigm.

Understanding their construction requires first grasping the underlying framework of RNCABE, which provides the essential simulation-based properties needed for security in the presence of deletion functionality. Therefore, we briefly review the formal syntax of RNCABE, defined over a message space $\mathcal{M}$, an attribute space $\mathcal{X}$, and a policy space $\mathcal{P}$.

- **Setup**: Given the security parameter 1^λ, the setup algorithm outputs a master public key mpk and a master secret key msk.
- **KeyGen**: On input of the master secret key msk and a policy P, the key generation algorithm generates the corresponding secret key sk_P.
- **Encryption**: Using the master public key mpk, an attribute X, and a message m, the encryption algorithm produces a ciphertext CT.
- **Decryption**: The decryption algorithm recovers the message m' from CT using a secret key sk, or outputs $\perp$ in case of failure.
- **FakeSetup**: The fake setup algorithm produces a master public key mpk together with auxiliary information aux, serving as an alternative to the real setup.
- **FakeCT**: Given the master public key mpk, an auxiliary information aux, and an attribute X, the fake encryption algorithm outputs a fake ciphertext $\widetilde{\mathsf{CT}}$.
- **FakeSK**: The fake key generation algorithm generates a fake secret key $\widetilde{\mathsf{sk}}$ corresponding to policy P.
- **Reveal**: On input the master public key mpk, an auxiliary information aux, a fake ciphertext $\widetilde{\mathsf{CT}}$, and a message m, the reveal algorithm outputs a fake master secret key $\widetilde{\mathsf{msk}}$.

First Modification: Towards Registered ABE with Certified Deletion. As an initial step towards our goal, we focus on constructing a registered attribute-based encryption scheme with certified deletion, without yet addressing its full-fledged security guarantees. In a registered attribute-based setting, users must generate their own public and secret key pairs independently, without relying on any trusted central authority. Consequently, to extend the existing ABE-CD framework into a registered setting, we need to introduce new algorithms and

modify existing ones from the protocol of Hiroka *et al.* [HMNY21]. In particular, the following changes are required:

- **Setup**: Taking as input the length of the security parameter λ and the size of the attribute universe $\mathcal{U}$, the setup algorithm outputs a common reference string crs.
- **KeyGen**: Given the common reference string crs, an auxiliary state aux, and a policy P, each user independently runs the key generation algorithm to derive its own key pair $(\mathsf{pk}, \mathsf{sk})$ before registration.
- **RegPK**: On input of the common reference string crs, an auxiliary state aux, a user's public key pk, and a policy P, the registration algorithm updates the master public key mpk, and the auxiliary state aux to incorporate the registration of the new user.
- **Update**: The update algorithm, given the common reference string crs, auxiliary state aux, and a registered public key pk, outputs a helper secret key hsk that allows the associated user to decrypt ciphertexts for which they are authorized.

In the registered setting, the functionality of the trusted authority is replaced by a transparent public entity, referred to as the curator, which executes the RegPK and Update algorithms. Before registering a public key pk, the curator can invoke a validation procedure to ensure its correctness, as the key may have been maliciously generated.

Second Modification: Architecture of Shad-RABE. Building on the model of Hiroka *et al.* [HMNY21], we next aim to extend the notion of receiver non-committing attribute-based encryption (RNCABE) to a decentralized setting. This extension introduces several conceptual challenges, primarily due to the need for specialized simulation algorithms that preserve the registration framework. Conceptually, this can be viewed as adapting RNCABE to the registered framework.

In the RNCABE framework of Hiroka *et al.* [HMNY21], the trusted authority holds a master secret key msk, which is used to generate and distribute user secret keys. In contrast, within the registered setting, the curator is a transparent and publicly verifiable entity, and hence cannot hold or access any secret information. This fundamental restriction makes a direct adaptation of RNCABE infeasible.

Alternatively, one may view the incorporation of registration mechanisms into RNCABE as yielding a registered ABE scheme with non-committing security. Since both key generation and the distribution of helper secret keys in the Update procedure are already managed by the curator, embedding simulation algorithms within the registered ABE framework provides a natural pathway toward our goal. In particular, the simulation algorithms are used only within the security experiment, executed by the challenger, and have no bearing on the scheme's correctness. We refer to this extended framework as *Shadow Registered Attribute-Based Encryption* ($\mathsf{Shad\text{-}RABE}$), which serves as the core building block for our

subsequent constructions. Specifically, Shad-RABE augments the classical RABE algorithms with five additional simulation procedures:

- **SimKeyGen**: Given the common reference string crs, an auxiliary state aux, an access policy P, and an ancillary dictionary B (possibly empty) indexed by public keys, the simulator runs the key-generation procedure to produce a simulated public key $\widetilde{\mathsf{pk}}$ along with an updated dictionary B.
- **SimRegPK**: The simulated registration procedure takes as input the common reference string crs, an auxiliary state aux, a public key pk, an access policy P, and the ancillary dictionary B, and outputs a simulated master public key $\widetilde{\mathsf{mpk}}$ together with an updated auxiliary state $\widetilde{\mathsf{aux}}$.
- **SimCorrupt**: The simulated corrupt algorithm, on input the common reference string crs, a public key pk, and the ancillary dictionary B, returns a simulated secret key $\widetilde{\mathsf{sk}}$ corresponding to pk.
- **SimCT**: Given the simulated master public key $\widetilde{\mathsf{mpk}}$, the dictionary B, and an attribute X, the ciphertext simulation algorithm produces a simulated ciphertext $\widetilde{\mathsf{ct}}$.
- **Reveal**: Given a public key $\widetilde{\mathsf{pk}}$, the dictionary B, a simulated ciphertext $\widetilde{\mathsf{ct}}$, and a target message μ, the reveal algorithm outputs a simulated secret key $\widetilde{\mathsf{sk}}$ that decrypts $\widetilde{\mathsf{ct}}$ to μ.

Security for Shad-RABE. The security foundation of Shad-RABE builds upon a receiver non-committing security framework that combines indistinguishability obfuscation with zero-knowledge arguments possessing statistical soundness and computational zero-knowledge properties. The security analysis employs a sequence of hybrid transformations that systematically transition from real-world operations to ideal-world simulations, ensuring that no polynomial-time adversary can distinguish between these worlds with non-negligible probability.

The proof methodology begins with the original Shad-RABE security experiment, where all cryptographic operations are performed according to their original algorithms. The analysis then progressively introduces simulation components through computationally indistinguishable transformations. The first transformation replaces left-or-right decryption circuits embedded within obfuscated programs with left-only variants, thereby eliminating the scheme's dependence on the random selector string z, which determines the bit representation to use for each message position. This change relies on the security properties of indistinguishability obfuscation, which ensures that functionally equivalent circuits remain computationally indistinguishable when obfuscated.

Next, to integrate zero-knowledge arguments within the registered framework, we consider constructions that rely on a common reference string crs. In such settings, each new user registration would necessitate an update of the crs. Our initial attempt was to realize Shad-RABE using existing NIZK schemes with updatable common reference strings [GKM+18, ARS20]. However, these schemes

either fail to achieve statistical soundness or depend on pairing-based assumptions that are potentially insecure against quantum polynomial-time (QPT) adversaries.

To overcome these limitations, we employ a zero-knowledge argument system that does not require any common reference string, while still guaranteeing soundness and zero-knowledge properties against QPT adversaries. The analysis transitions to simulated ZKA components, where challenge proofs are generated using simulation algorithms: $\widetilde{\pi} \leftarrow \langle Sim \rangle(d^*, y)$, where d^*, y are the statement and an element of $\{0,1\}^*$, respectively. This transformation leverages the computational zero-knowledge property of the ZKA protocol to ensure that the simulated proofs are indistinguishable from the real ones. This makes the proof components independent of the actual challenge message.

Another hybrid transformation introduces asymmetric encryption patterns that break the symmetry in how different message bits are encrypted. For each bit position i, the construction encrypts different values depending on the selector bit: $\mathsf{CT}^*_{i,z[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,z[i]}, X^*, \mu[i])$ encrypts the actual message bit, while $\mathsf{CT}^*_{i,1-z[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,1-z[i]}, X^*, 1-\mu[i])$ encrypts its complement. This asymmetric approach is validated through a position-by-position hybrid argument that relies on the semantic security of the underlying RABE scheme.

Finally, the analysis achieves complete independence from the target message μ employing a new random string $z^* \leftarrow \{0,1\}^{\ell_m}$ and encrypting fixed values: $\mathsf{CT}^*_{i,z^*[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,z^*[i]}, X^*, 0)$ and $\mathsf{CT}^*_{i,1-z^*[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,1-z^*[i]}, X^*, 1)$. This systematic elimination of dependencies, from the random selector string z, through real ZKA proofs, to the target message μ itself, combined with the computational indistinguishability of consecutive transformations, establishes that no polynomial-time adversary can distinguish between real and ideal executions with non-negligible probability. The resulting Shad-RABE primitive provides the simulation capabilities necessary for constructing secure RABE schemes with certified deletion, enabling security proofs that remain valid even when adversaries can adaptively corrupt users and request deletion certificates.

2.2 Design of RABE-PriVCD under LWE Assumptions

We propose a construction of Registered Attribute-Based Encryption with Privately Verifiable Certified Deletion (RABE-PriVCD), designed to jointly realize decentralized access control and quantum-secure data deletion. The framework follows a hybrid encryption strategy, which distinctly separates the role of registered ABE from the quantum mechanisms required certified deletion.

At the core of our design lies the observation that the two functionalities, registered ABE and certified deletion, can be combined in a modular manner within a hybrid structure. To this end, we integrate two specialized primitives:

- **Shad-RABE**, which manages access control, decentralized key registration, and fine-grained policy enforcement.
- **SKE-CD**, symmetric key encryption with certified deletion, which leverages quantum principles to guarantee that deleted data remains irretrievable, even against unbounded adversaries.

Two-Layer Encryption. The construction operates through a two-layer encryption approach. During encryption, a fresh symmetric key is generated and encrypted using Shad-RABE under the specified attribute set, while the actual message is encrypted using SKE-CD with the symmetric key. This design ensures that attributes are enforced through the registered ABE mechanism, while the certified deletion security is derived from the quantum properties of the symmetric scheme. The core encryption algorithm $\mathsf{Encrypt}(\mathsf{mpk}, X, \mu)$ demonstrates the hybrid approach:

- Generate symmetric key $\mathsf{ske.sk} \leftarrow \mathsf{SKE\text{-}CD.KeyGen}(1^\lambda)$
- Compute Shad-RABE ciphertext $\mathsf{srabe.ct} \leftarrow \mathsf{Shad\text{-}RABE.Encrypt}(\mathsf{mpk}, X, \mathsf{ske.sk})$
- Compute symmetric ciphertext $\mathsf{ske.ct} \leftarrow \mathsf{SKE\text{-}CD.Enc}(\mathsf{ske.sk}, \mu)$
- Output ciphertext $\mathsf{ct} := (\mathsf{srabe.ct}, \mathsf{ske.ct})$ and verification key $\mathsf{vk} := \mathsf{ske.sk}$

The security guarantees derive from the interaction between both components:

- **Access Control Security:** Shad-RABE ensures that only users with policies P satisfying $P(X) = 1$ can recover the symmetric key k. This holds even for adaptive corruptions, dynamic user registration, and key update operations.
- **Certified Deletion Security:** Once a valid deletion certificate is produced, the SKE-CD scheme guarantees that the message μ can no longer be recovered by any quantum polynomial-time (QPT) adversary.
- **Composition Security:** The hybrid approach preserves both security properties. Unauthorized users cannot access the information because they cannot recover k. After deletion, even authorized users cannot recover the data due to the information-theoretic deletion guarantees.

Post-Quantum Realization. Our construction achieves post-quantum security through a modular framework. Specifically, the Shad-RABE component is realized from lattice-based building blocks, namely: a registered ABE scheme under the ℓ-succinct LWE assumption [CHW25], a zero-knowledge argument based on plain LWE [BS20], and an indistinguishability obfuscation scheme relying on equivocal LWE [CLW25]. These primitives jointly ensure resistance against quantum adversaries and enable simulation techniques required for certified deletion proofs. Complementing this, the SKE-CD component employs an unconditionally secure one-time symmetric encryption with certified deletion [BI20], thus providing information-theoretic guarantees beyond standard computational assumptions. This separation allows independent analysis of ABE functionality and

deletion mechanisms, while their integration delivers decentralized key management, fine-grained access control, and verifiable data deletion with private verification.

2.3 Design of RABE-PubVCD under LWE Assumptions

In the RABE-PriVCD framework, the verification key coincides with the symmetric key ske.sk generated by the SKE-CD scheme. Since ske.sk is used directly in the two-layer encryption of message μ, verification remains inherently private; public verification cannot be achieved in this setting. Consequently, to construct RABE-PubVCD, we cannot rely on SKE-CD protocol.

Core Idea. To overcome this limitation, we employ the notion of one-shot signatures [AGKZ20]. A one-shot signature scheme allows the generation of a classical public key oss.pk along with a quantum secret key oss.sk. The secret key can be used to sign message 0 (producing σ_0) or message 1 (producing σ_1), but never both. Importantly, signatures can be verified publicly, making them well-suited for publicly verifiable deletion.

Our construction integrates one-shot signatures with extractable witness encryption. The encryption of a message μ proceeds by encoding μ within a witness encryption instance, where the statement corresponds to the existence of a valid signature on 0. The deletion certificate, on the other hand, is realized as a one-shot signature on 1. Once such a certificate is issued, the one-shot property ensures that no valid signature on 0 can ever be generated, thereby guaranteeing certified deletion. Public verifiability follows immediately, as any third party can check signatures. In order to prevent an adversary from decrypting the ciphertext before issuing the deletion certificate, we add an additional layer of encryption, for which we implement Shad-RABE.

Protocol Execution. The protocol unfolds as follows: the sender first generates one-shot signature parameters oss.crs and shares them with the receiver. The receiver uses these to create a key pair $(\mathsf{oss.pk}, \mathsf{oss.sk})$, returns the public key to the sender, and keeps the signing key secret. With oss.pk, the sender defines a witness statement that asserts the existence of a valid signature on 0, encrypts μ under this statement by witness encryption, and then encapsulates the resulting ciphertext using Shad-RABE under the chosen attribute set X. The final ciphertext is transmitted to the receiver. At the end of this process, the sender publishes the public verification key $\mathsf{vk} = (\mathsf{oss.crs}, \mathsf{oss.pk})$, while the receiver obtains the ciphertext $\mathsf{ct} = (\mathsf{srabe.ct}, \mathsf{oss.sk})$ containing both the encrypted data and the deletion capability.

Security Guarantees. The one-shot property guarantees that no adversary can produce both σ_0 and σ_1. Thus, once a deletion certificate σ_1 is issued, the witness encryption becomes undecryptable, certifying irreversible deletion. Since one-shot signatures are publicly verifiable, any party can confirm deletion by checking

$$\mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 1) = \top.$$

Only the signing key oss.sk involves quantum information and is generated locally by the receiver; all remaining components, including the ciphertext srabe.ct, are entirely classical. Consequently, the interaction between the sender and receiver can proceed over a classical channel. In contrast to prior frameworks that rely on quantum communication or trusted verification authorities, our construction operates entirely over classical communication and supports fully public verifiability.

2.4 Achieving Certified Everlasting Deletion Security

We present two constructions that achieve certified everlasting security, ensuring that once deletion is executed, the underlying message can never be recovered, even by adversaries with unlimited computational power in the future. Our methodology leverages fundamental quantum mechanical properties, particularly the uncertainty principle, to design schemes that support both private and public verification.

Quantum One-Time Pad. Our constructions are based on a quantum one-time pad implemented with BB84 states. To encrypt a single message bit b, the encryptor samples two uniformly random strings, $x, \theta \leftarrow \{0,1\}^\lambda$. It then prepares the quantum state $|x\rangle_\theta = \bigotimes_{j\in[\lambda]} H^{\theta_j} |x_j\rangle$, where each qubit is encoded in either the computational basis (if $\theta_j = 0$) or the Hadamard basis (if $\theta_j = 1$).

The message bit b is masked using the bits of x that were encoded in the Hadamard basis. The classical part of the ciphertext will contain an encryption of the basis string θ and the masked message, $b \oplus \bigoplus_{j:\theta_j=1} x_j$. The quantum state $|x\rangle_\theta$ itself serves as the quantum component of the ciphertext. An authorized decryptor, upon recovering θ, can measure the quantum state in the correct bases to learn x and subsequently unmask the message.

Privately Verifiable Certified Everlasting Deletion. In our first construction, RABE with Privately Verifiable Certified Everlasting Deletion (RABE-PriVCED), the encryption procedure begins by generating (x, θ) and preparing the quantum state $|x\rangle_\theta$. A bitwise concealment value is then computed as $m^* := b \oplus \bigoplus_{i:\theta_i=1} x_i$, after which (θ, m^*) is encrypted using RABE under the attribute X. The resulting ciphertext therefore consists of a classical component and a quantum register. During decryption, the algorithm first recovers (θ, m^*) from the classical ciphertext, measures $|x\rangle_\theta$ in the specified bases to reconstruct x, and subsequently retrieves the original message. Deletion is achieved by measuring the quantum register in the Hadamard basis, producing outcomes that serve as a deletion certificate. Since verifying this certificate requires knowledge of θ, which remains secret within the decryption process, the deletion in this scheme is only privately verifiable.

Publicly Verifiable Certified Everlasting Deletion. To achieve public verifiability, we upgrade our construction using a technique inspired by Kitagawa *et al.* [KNY23] to authenticate the quantum state. Our idea is to generate a signature for the classical string x by coherently running a signing algorithm on the

quantum state itself. This creates a "coherently signed" BB84 state, which embeds a quantum signature that is verifiable using a classical public key. Here, the encryptor generates a signature key pair $(\mathsf{sigk}, \mathsf{vk})$ and includes sigk along with (θ, m^*) in the classical component of the ciphertext. The quantum component is a coherently signed BB84 state of the form $U_{\mathsf{sign}} |x\rangle_\theta |0 \dots 0\rangle = |x\rangle_\theta |\mathsf{Sign}(\mathsf{sigk}, x)\rangle$, with vk made public. During decryption, the signature register can be "uncomputed" using sigk to recover the pure BB84 state, after which the procedure mirrors that of the privately verifiable scheme. Deletion is performed by measuring the entire quantum state in the computational basis, yielding both a string x' and a classical signature σ. The pair (x', σ) forms the deletion certificate, which any third party can validate using vk. This makes deletion publicly verifiable without requiring access to hidden basis information.

Security Foundation. The security relies on fundamental quantum mechanical principles formalized through the Certified Everlasting Lemma (CEL) [BK23]. A key distinction from the notion of Certified Deletion lies in the adversary's post-deletion capabilities. In certified deletion security, the challenger reveals all honest secret keys after successful deletion verification, and security relies on the computational assumption that the adversary cannot distinguish the encrypted messages despite having these keys. In contrast, Certified Everlasting security does not grant access to honest secret keys but instead guarantees information-theoretic indistinguishability of the residual state. When BB84 states prepared on random bases are measured incorrectly during deletion, the resulting probability distributions become computationally indistinguishable for any encrypted messages. The proof employs sophisticated quantum information techniques, including EPR pairs, deferred measurements, and projective measurement analysis. Once deletion occurs through incorrect basis measurement, the quantum superposition irreversibly collapses, making message recovery impossible even with unlimited computational resources. Unlike classical cryptography, which relies on computational assumptions potentially vulnerable to quantum computers or algorithmic advances, our everlasting deletion guarantees remain secure indefinitely. After valid deletion, the privacy protection persists regardless of future technological developments.

3 Preliminaries

This section provides an overview of the cryptographic tools used in this work.

3.1 Public Key Encryption

We present the formal definition of the public key encryption (PKE) scheme, along with its correctness and security framework.

Syntax. Let λ be a security parameter, and let p, q, and r be polynomial functions in λ. A PKE scheme consists of a tuple of algorithms, $\prod_{\mathsf{PKE}} = (\mathsf{KeyGen}, \mathsf{Encrypt}, \mathsf{Decrypt})$ with message space $\mathcal{M} := \{0,1\}^n$, ciphertext space $\mathcal{C} :=$

$\{0,1\}^{p(\lambda)}$, public key space $\mathcal{PK} := \{0,1\}^{q(\lambda)}$, and secret key space $\mathcal{SK} := \{0,1\}^{r(\lambda)}$, where n is the length of the message. The algorithms are specified as follows:

- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$. The key generation algorithm takes as input the length of the security parameter 1^λ to output a public key $\mathsf{pk} \in \mathcal{PK}$ and a secret key $\mathsf{sk} \in \mathcal{SK}$.
- $(\mathsf{CT}) \leftarrow \mathsf{Encrypt}(\mathsf{pk}, m)$. The encryption algorithm takes as input pk and a message $m \in \mathcal{M}$ to output a ciphertext $\mathsf{CT} \in \mathcal{C}$.
- $(m' \vee \perp) \leftarrow \mathsf{Decrypt}(\mathsf{sk}, \mathsf{CT})$. The decryption algorithm takes as input sk and CT to output either the message m' or $\perp$, indicating decryption failure.

We require that a PKE scheme $\prod_{\mathsf{PKE}} = (\mathsf{KeyGen}, \mathsf{Encrypt}, \mathsf{Decrypt})$ satisfies the following properties.

- **Correctness.** There exists a negligible function negl such that for any $\lambda \in \mathbb{N}$, $m \in \mathcal{M}$, it holds that

$$\Pr\left[\mathsf{Decrypt}(\mathsf{sk}, \mathsf{CT}) \neq m \;\middle|\; \begin{array}{l}(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda) \\ \mathsf{CT} \leftarrow \mathsf{Encrypt}(\mathsf{pk}, m)\end{array}\right] \leq \mathsf{negl}(\lambda).$$

- **IND-CPA Security.** We define indistinguishability under chosen-plaintext attack (IND-CPA) security through the following experiment, denoted $\mathsf{Exp}^{\mathsf{IND-CPA}}_{\mathsf{PKE},\mathcal{A}}(\lambda, b)$, for a PKE scheme and a quantum polynomial-time adversary (QPT) $\mathcal{A}$.
  - The challenger samples a public/secret key pair $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and sends the public key pk to $\mathcal{A}$.
  - The adversary $\mathcal{A}$ selects two messages $(m_0, m_1) \in \mathcal{M}^2$ and submits them to the challenger.
  - The challenger selects a bit $b \in \{0,1\}$ uniformly at random, computes $\mathsf{CT}_b \leftarrow \mathsf{Encrypt}(\mathsf{pk}, m_b)$, and sends CT_b to $\mathcal{A}$.
  - The adversary $\mathcal{A}$ outputs a guess $b' \in \{0,1\}$ for bit b.

The advantage of $\mathcal{A}$ in the above experiment is defined by

$$\mathsf{Adv}^{\mathsf{IND-CPA}}_{\mathsf{PKE},\mathcal{A}}(\lambda) := \left|\Pr[\mathsf{Exp}^{\mathsf{IND-CPA}}_{\mathsf{PKE},\mathcal{A}}(\lambda, 0) = 1] - \Pr[\mathsf{Exp}^{\mathsf{IND-CPA}}_{\mathsf{PKE},\mathcal{A}}(\lambda, 1) = 1]\right|.$$

We say that the $\prod_{\mathsf{PKE}}$ protocol is IND-CPA secure if for all QPT adversaries $\mathcal{A}$, the advantage is negligible in λ, *i.e.,* the following holds:

$$\mathsf{Adv}^{\mathsf{IND-CPA}}_{\mathsf{PKE},\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda).$$

A notable example is the Regev PKE scheme [Reg09] that achieves IND-CPA security under the Learning with Errors (LWE) assumption against QPT adversaries. We refer the reader to [Reg09, GPV08] for a more detailed treatments of the LWE assumption and additional constructions of the post-quantum secure PKE schemes.

3.2 Indistinguishability Obfuscation

Let $\{\mathcal{C}_\lambda\}_{\lambda\in\mathbb{N}}$ be a family of circuits. A PPT algorithm $i\mathcal{O}$ is an indistinguishability obfuscator for the circuit class if it satisfies the following:

- **Correctness:** For every security parameter $\lambda \in \mathbb{N}$, every circuit $C \in \mathcal{C}_\lambda$, and every input x, the obfuscated circuit preserves functionality:

$$\Pr[C'(x) = C(x) \mid C' \leftarrow i\mathcal{O}(C)] = 1.$$

- **Indistinguishability:** For any QPT distinguisher $\mathcal{D}$ and any pair of functionally equivalent circuits $C_0, C_1 \in \mathcal{C}_\lambda$ such that $C_0(x) = C_1(x)$ for all inputs x and $|C_0| = |C_1|$, the obfuscations are computationally indistinguishable:

$$\left|\Pr[\mathcal{D}(i\mathcal{O}(C_0)) = 1] - \Pr[\mathcal{D}(i\mathcal{O}(C_1)) = 1]\right| \leq \mathsf{negl}(\lambda).$$

In recent literature, candidates for indistinguishability obfuscation secure against QPT adversaries have been proposed [GP21, WW21, BDGM20]. Moreover, the work of [CLW25] proposes a post-quantum $i\mathcal{O}$ candidate based on the NTRU assumption and a novel Equivocal LWE problem, employing a trapdoor-based Gaussian hint-release mechanism that ensures correctness while preventing structural leakage, with security proven against QPT adversaries.

3.3 Digital Signatures

We now formally define a digital signature scheme, including its syntax, correctness, and a specialized one-time unforgeability notion against quantum adversaries.

Syntax. A digital signature (SIG) scheme consists of a tuple of algorithms $\prod_{\mathsf{SIG}} = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Verify})$ defined over the message space $\mathcal{M}$, signature space $\mathcal{S}$, verification key space $\mathcal{VK}$, and signature key space $\mathcal{SK}$. The algorithms are described in the following:

- $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$. The key generation algorithm takes as input the length of the security parameter 1^λ to output a key pair consisting of a verification key $\mathsf{vk} \in \mathcal{VK}$, and a signing key $\mathsf{sk} \in \mathcal{SK}$.
- $(\sigma) \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$. Taking as input the signing key sk and a message $m \in \mathcal{M}$, the signing algorithm outputs a signature $\sigma \in \mathcal{S}$.
- $\{0, 1\} \leftarrow \mathsf{Verify}(\mathsf{vk}, m, \sigma)$. On the input of the verification key vk, a message m and a signature σ, the verification algorithm returns 1 (accept) if the signature is valid or 0 (reject) otherwise.

A digital signature scheme $\prod_{\mathsf{SIG}}$ must satisfy the following properties.

- **Correctness.** There exists a negligible function $\mathsf{negl}(\cdot)$ such that for all security parameters $\lambda \in \mathbb{N}$ and for all messages $m \in \mathcal{M}$, we have

$$\Pr\left[\mathsf{Verify}(\mathsf{vk}, m, \sigma) = 1 \;\middle|\; \begin{array}{l}(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda) \\ \sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)\end{array}\right] \geq 1 - \mathsf{negl}(\lambda).$$

- **One-Time Unforgeability (OT-UF) for BB84 States.** We define one-time unforgeability (OT-UF) security for digital signatures in the presence of quantum adversaries with access to BB84 quantum states. We say that a digital signature scheme with the tuples of algorithm $\prod_{\mathsf{SIG}} = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Verify})$ satisfies one-time unforgeability against BB84 attacks if no QPT adversary $\mathcal{A}$ can win the following security game with non-negligible probability.
  - The challenger generates a key pair $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda)$ and sends vk to $\mathcal{A}$.
  - $\mathcal{A}$ selects a message $m^* \in \mathcal{M}$ and receives a valid signature $\sigma^* \leftarrow \mathsf{Sign}(\mathsf{sk}, m^*)$.
  - Finally, $\mathcal{A}$ outputs a pair (m', σ').
  - $\mathcal{A}$ wins if $m' \neq m^*$ and $\mathsf{Verify}(\mathsf{vk}, m', \sigma') = 1$.

  The adversary may additionally have quantum access to BB84 states encoding the signed message. We say that a digital signature scheme is one-time unforgeable for BB84 states if this success probability is negligible in λ.

In the current literature, Kitagawa et al. [KNY23] constructed a digital signature scheme satisfying one-time unforgeability for BB84 states under the existence of one-way functions.

3.4 NP Languages

In computational complexity theory, NP represents a non-deterministic polynomial time machine [Tur36, Coo00] (*i.e.,* a machine that can move more than one state from a given configuration). A language $\mathcal{L}$ over an alphabet Σ is called an NP languages iff there exists a $k \in \mathbb{N}$ and a polynomial checking relation $R_{\mathcal{L}} \subseteq \Sigma^* \times \Sigma^*$ such that for all strings $x \in \Sigma^*$,

$$x \in \mathcal{L} \iff \exists y \; (|y| \leq |x|^k \text{ and } R_{\mathcal{L}}(x, y)),$$

where $|x|$ and $|y|$ are the lengths of the strings x and y, respectively.

3.5 Zero-Knowledge Arguments

We define a zero-knowledge argument system (ZKA) as an interactive protocol between two parties. Let $\langle A(w), B(z)\rangle(x)$ denote the interaction between interactive machines A and B on a common input x, where A receives auxiliary input w and B receives auxiliary input z. We refer to the protocol as $\prod_{\mathsf{ZKA}} = \langle A, B\rangle$.

Definition 1 (Classical Proof and Argument Systems for NP). *Let (P, V) be a protocol with an honest PPT prover P and an honest PPT verifier V for a language $\mathcal{L} \in \mathsf{NP}$, satisfying:*

1. ***Perfect Completeness:*** *For any $x \in \mathcal{L}$, $R_{\mathcal{L}}(x, w)$ and $z \in \{0,1\}^*$*
$$\Pr[\langle P(w), V(z)\rangle(x) = 1] = 1.$$
2. ***Soundness:*** *For any quantum polynomial-size prover $P^* = \{P^*_\lambda, \rho_\lambda\}_{\lambda\in\mathbb{N}}$, there exists a negligible function $\mu(\cdot)$ such that for any security parameter $\lambda \in \mathbb{N}$ and any $x \in \{0,1\}^\lambda \setminus \mathcal{L}$ and $z \in \{0,1\}^*$*
$$\Pr[\langle P^*_\lambda(\rho_\lambda), V(z)\rangle(x) = 1] \leq \mu(\lambda).$$
A protocol with computational soundness is called an argument.
3. ***Zero Knowledge:*** *There exists a quantum polynomial-time simulator Sim such that for any quantum polynomial-size verifier $V^* = \{V^*_\lambda, \rho_\lambda\}_{\lambda\in\mathbb{N}}$,*
$$\left\{\langle P(w), V^*_\lambda(\rho_\lambda)\rangle(x)\right\}_{\lambda,x,w} \overset{c}{\approx} \left\{\mathsf{Sim}(x, V^*_\lambda, \rho_\lambda)\right\}_{\lambda,x},$$
where $\lambda \in \mathbb{N}$, $x \in \mathcal{L} \cap \{0,1\}^\lambda$, and $R_{\mathcal{L}}(x, w)$.
If V^* is a classical circuit, then the simulator is computable by a classical polynomial-time algorithm.
$$\{\langle P(w), V^*(z)\rangle(x)\}_{x,w,z} \overset{c}{\approx} \{\langle \mathsf{Sim}\rangle(x, z)\}_{x,z}$$
when the distinguishing gap is considered as a function of $|x|$.

If the LWE assumption holds against quantum polynomial-size adversaries, then there exists a post-quantum zero-knowledge interactive argument system (P, V) for all languages in NP [BS20].

3.6 Witness Encryption

Witness Encryption (WE), introduced by Garg *et al.* [GGSW13a], enables the encryption of a message m with a statement x from an NP language such that decryption is possible only with a corresponding valid witness w. We proceed to formally define the syntax of the WE scheme and outline its correctness and security guarantees, particularly in the context of quantum adversaries.

Syntax. A witness encryption scheme for an NP language $\mathcal{L}$ consists of a pair of algorithms $\prod_{\mathsf{WE}} = (\mathsf{Encrypt}, \mathsf{Decrypt})$ described below.

- $(\mathsf{CT}) \leftarrow \mathsf{Encrypt}(1^\lambda, x, m)$. On input a security parameter 1^λ, an instance $x \in \mathcal{L}$, and a message $m \in \mathcal{M}$, the encryption algorithm returns a ciphertext CT.
- $(m \vee \perp) \leftarrow \mathsf{Decrypt}(\mathsf{CT}, w)$. Taking as input a ciphertext CT and a witness w such that $(x, w) \in R_{\mathcal{L}}$, the decryption algorithm outputs either the message m or a distinguished failure symbol $\perp$.

A WE scheme $\prod_{\mathsf{WE}}$ should satisfy the following correctness and security properties:

- **Correctness.** We require that for all security parameters $\lambda \in \mathbb{N}$, all messages $m \in \mathcal{M}$, and all $(x, w) \in R_{\mathcal{L}}$, decryption correctly recovers the original message except with negligible probability:

$$\Pr\left[\mathsf{Decrypt}(\mathsf{Encrypt}(1^\lambda, x, m), w) = m\right] \geq 1 - \mathsf{negl}(\lambda).$$

- **Security.** We consider two security attributes for witness encryption: soundness and extractability.
  - **Soundness.** A WE scheme satisfies soundness if for any $x \notin \mathcal{L}$, ciphertexts corresponding to different messages are computationally indistinguishable. More precisely, for any QPT adversary $\mathcal{A}$, and for all $m_0, m_1 \in \mathcal{M}$ and $x \notin \mathcal{L}$, we define the advantage:

$$\mathsf{Adv}^{\mathsf{WE}}_{\mathcal{A}}(\lambda) = \Big|\Pr[\mathcal{A}(\mathsf{Encrypt}(1^\lambda, x, m_0)) = 1] - \Pr[\mathcal{A}(\mathsf{Encrypt}(1^\lambda, x, m_1)) = 1]\Big|.$$

    We require that $\mathsf{Adv}^{\mathsf{WE}}_{\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda)$.

    We emphasize that the correctness property of WE scheme guarantees the successful decryption when $x \in \mathcal{L}$ and a valid witness w for the relation R is provided. In contrast, the soundness property ensures that if $x \notin \mathcal{L}$, no efficient adversary can distinguish between encryptions of different messages, and thus cannot recover meaningful information. We remark that the definition intentionally does not address the case where $x \in \mathcal{L}$ but no valid witness for x is known. In such cases, the behaviour of the decryption algorithm is unspecified by the correctness and soundness definitions, and no guarantees are provided.
  - **Extractability**. Extractability property [GKP+13,HMNY21] ensures that if an adversary can distinguish between encryptions of two messages with non-negligible advantage, then one can efficiently extract a valid witness for the instance. Formally, for any QPT adversary $\mathcal{A}$, any polynomial $p(\lambda)$, and any $m_0, m_1 \in \mathcal{M}$, there exists a QPT extractor $\mathcal{E}$ and polynomial $q(\lambda)$ such that for any auxiliary quantum state aux potentially entangled with external registers, if:

$$\left|\Pr[\mathcal{A}(\mathsf{aux}, \mathsf{Encrypt}(1^\lambda, x, m_0)) = 1] - \Pr[\mathcal{A}(\mathsf{aux}, \mathsf{Encrypt}(1^\lambda, x, m_1)) = 1]\right| \geq \frac{1}{p(\lambda)},$$

    then, the extractor succeeds in producing a valid witness with non-negligible probability:

$$\Pr\left[(x, \mathcal{E}(1^\lambda, x, \mathsf{aux})) \in R_{\mathcal{L}}\right] \geq \frac{1}{q(\lambda)}.$$

Extractable witness encryption was introduced in [GKP+13] as a syntactic modification of the witness encryption framework of [GGSW13b].

3.7 Secret Key Encryption with Certified Deletion

Broadbent and Islam introduced the concept of encryption with certified deletion in the context of secret key encryption (SKE) [BI20]. Their framework considers a one-time usage model, where the secret key is employed for a single encryption instance. We now present the formal definition of a secret key encryption scheme with certified deletion (SKE-CD) and outline its associated security requirements.

Syntax of SKE-CD. A secret key encryption scheme with certified deletion (SKE-CD) is a tuple of QPT algorithms $\prod_{\mathsf{SKE\text{-}CD}} = (\mathsf{KeyGen}, \mathsf{Encrypt}, \mathsf{Decrypt}, \mathsf{Deletion}, \mathsf{Verify})$ with plaintext space $\mathcal{M}$ and key space $\mathcal{K}$, with the following algorithms.

- $(\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$. On input the length of the security parameter 1^λ, the key generation algorithm outputs a secret key $\mathsf{sk} \in \mathcal{K}$.
- $(\mathsf{CT}) \leftarrow \mathsf{Encrypt}(\mathsf{sk}, m)$. The encryption algorithm takes as input sk and a plaintext $m \in \mathcal{M}$ to output a ciphertext CT.
- $(m' \vee \perp) \leftarrow \mathsf{Decrypt}(\mathsf{sk}, \mathsf{CT})$. Taking as input sk and CT, the decryption algorithm outputs a plaintext $m' \in \mathcal{M}$ or $\perp$.
- $(\mathsf{cert}) \leftarrow \mathsf{Deletion}(\mathsf{CT})$. The deletion algorithm, on input the ciphertext CT, produces a deletion certificate cert.
- $(\top \vee \perp) \leftarrow \mathsf{Verify}(\mathsf{sk}, \mathsf{cert})$. The verification algorithm takes as input the secret key sk and the certificate cert to return $\top$ or $\perp$.

- **Correctness.** A SKE-CD scheme $\prod_{\mathsf{SKE\text{-}CD}} = (\mathsf{KeyGen}, \mathsf{Encrypt}, \mathsf{Decrypt}, \mathsf{Deletion}, \mathsf{Verify})$ is said to be correct if it satisfies the following two properties.
  - **Decryption correctness:** For any $\lambda \in \mathbb{N}$, $m \in \mathcal{M}$,

$$\Pr\left[\mathsf{Decrypt}(\mathsf{sk}, \mathsf{CT}) \neq m \;\middle|\; \begin{array}{l} \mathsf{sk} \leftarrow \mathsf{KeyGen}(1^\lambda), \\ \mathsf{CT} \leftarrow \mathsf{Encrypt}(\mathsf{sk}, m) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

  - **Verification correctness:** For any $\lambda \in \mathbb{N}$, $m \in \mathcal{M}$,

$$\Pr\left[\mathsf{Verify}(\mathsf{sk}, \mathsf{cert}) = \perp \;\middle|\; \begin{array}{l} \mathsf{sk} \leftarrow \mathsf{KeyGen}(1^\lambda), \\ \mathsf{CT} \leftarrow \mathsf{Encrypt}(\mathsf{sk}, m), \\ \mathsf{cert} \leftarrow \mathsf{Deletion}(\mathsf{CT}) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

- **Certified Deletion Security.** The notion of one-time certified deletion (OT-CD) security for a SKE-CD scheme is defined via the security experiment $\mathsf{Exp}^{\mathsf{OT-CD}}_{\mathsf{SKE\text{-}CD},\,\mathcal{A}}(\lambda, b)$, which models the interaction between the challenger and a QPT adversary $\mathcal{A}$.
  - **KeyGen Phase:** The challenger generates a secret key $\mathsf{sk} \leftarrow \mathsf{KeyGen}(1^\lambda)$.
  - **Challenge:** $\mathcal{A}$ submits two messages $(m_0, m_1) \in \mathcal{M}^2$ to the challenger. It samples a bit $b \in \{0, 1\}$ and computes the ciphertext $\mathsf{CT}_b \leftarrow \mathsf{Encrypt}(\mathsf{sk}, m_b)$, which is then sent to $\mathcal{A}$.

- **Certificate Verification:** $\mathcal{A}$ sends cert to the challenger. The challenger computes $\mathsf{Verify}(\mathsf{sk}, \mathsf{cert})$. If the output is $\bot$, the challenger sends $\bot$ to $\mathcal{A}$. If the output is $\top$, the challenger sends sk to $\mathcal{A}$.
- **Guess:** $\mathcal{A}$ outputs a guess $b' \in \{0,1\}$.

The advantage of $\mathcal{A}$ in breaking certified deletion is defined as

$$\mathsf{Adv}^{\mathsf{OT-CD}}_{\mathsf{SKE\text{-}CD},\,\mathcal{A}}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{OT-CD}}_{\mathsf{SKE\text{-}CD},\,\mathcal{A}}(\lambda,0)=1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{OT-CD}}_{\mathsf{SKE\text{-}CD},\,\mathcal{A}}(\lambda,1)=1\right]\right|.$$

We say that a $\mathsf{SKE\text{-}CD}$ scheme $\prod_{\mathsf{SKE\text{-}CD}}$ achieves $\mathsf{OT\text{-}CD}$ security if for all QPT adversaries $\mathcal{A}$, the advantage is negligible in λ, *i.e.,* the following holds:

$$\mathsf{Adv}^{\mathsf{OT-CD}}_{\mathsf{SKE\text{-}CD},\,\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda).$$

A secret key encryption scheme satisfying the $\mathsf{OT\text{-}CD}$ security property is referred to as an $\mathsf{OTSKE\text{-}CD}$ scheme.

Claim 1 (Impossibility of Reverse Certified Deletion). *The notion of one-time certified deletion security ensures that once a valid deletion certificate has been produced, the corresponding ciphertext can no longer be decrypted. A natural question arises as to whether the converse can also be enforced, that once a ciphertext is decrypted, it should no longer be possible to generate a valid deletion certificate. However, this property is impossible to achieve due to the decryption correctness. If the quantum decryption algorithm* $\mathsf{Decrypt}$ *correctly recovers the plaintext from a quantum ciphertext* CT *with overwhelming probability, then by the gentle measurement lemma, the ciphertext remains only negligibly perturbed by the decryption process. As a result, it remains feasible to apply the deletion algorithm and obtain a valid certificate subsequently.*

It is noted that in existing constructions of secret key encryption with certified deletion, the secret key is a classical bitstring, while ciphertexts are quantum states. The following result, due to Broadbent and Islam [BI20], establishes the unconditional existence of such schemes.

Theorem 2 (Unconditional Existence of OT-CD Secure SKE). *There exists a one-time secret key encryption scheme with certified deletion that satisfies OT-CD security unconditionally, where the message space is* $\mathcal{M} = \{0,1\}^{\ell_m}$ *and the key space is* $\mathcal{K} = \{0,1\}^{\ell_k}$ *for some polynomials* ℓ_m, ℓ_k.

3.8 Certified Everlasting Lemmas

In this section, we revisit the certified everlasting lemma of Bartusek and Khurana [BK23], which provides cryptography with privately verifiable deletion. Later, Kitagawa *et al.* [KNY23] generalized this notion to achieve publicly verifiable deletion. Building on these foundations, we design a new registered attribute-based encryption scheme that supports publicly verifiable deletion.

Lemma 1 (Certified Everlasting Lemma [BK23]). *Let $\{\mathcal{Z}_\lambda(\cdot,\cdot,\cdot)\}_{\lambda\in\mathbb{N}}$ be an efficient quantum operation taking three inputs: (i) a λ-bit string θ, (ii) a single bit β, (iii) a quantum register A. For any QPT $\mathcal{B}$ algorithm, define the experiment $\widetilde{\mathcal{Z}}^{\mathcal{B}}_\lambda(b)$ over quantum states by executing $\mathcal{B}$ according to the following procedure:*

- *Sample $x,\theta \leftarrow \{0,1\}^\lambda$ uniformly at random.*
- *Initialize $\mathcal{B}$ with the state $\mathcal{Z}_\lambda\left(\theta,\, b\oplus\bigoplus_{i:\theta_i=1} x_i,\, |x\rangle_\theta\right)$.*
- *Let the output of $\mathcal{B}$ be a classical string $x'\in\{0,1\}^\lambda$ along with a residual quantum state on its register B.*
- *If $x_i = x'_i$ for all indices i where $\theta_i = 0$, output B. Otherwise, output a special failure symbol $\perp$.*

Assume that for any QPT $\mathcal{A}$, any $\theta\in\{0,1\}^\lambda$, $\beta\in\{0,1\}$, and any efficiently samplable state $|\psi\rangle_{A,C}$ over registers A and C, the following indistinguishability condition holds:

$$\left|\Pr[\mathcal{A}(\mathcal{Z}_\lambda(\theta,\beta,A),C)=1]-\Pr[\mathcal{A}(\mathcal{Z}_\lambda(0^\lambda,\beta,A),C)=1]\right|\le \mathsf{negl}(\lambda).$$

Then, for any QPT $\mathcal{B}$, we have

$$\mathsf{TD}(\widetilde{\mathcal{Z}}^{\mathcal{B}}_\lambda(0),\,\widetilde{\mathcal{Z}}^{\mathcal{B}}_\lambda(1))\le\mathsf{negl}(\lambda).$$

Kitagawa *et al.* [KNY23] upgraded the Lemma 1 to a publicly verifiable version by employing a signature scheme with one-time unforgeability for BB84 states.

Lemma 2 (Publicly Verifiable Certified Everlasting Lemma [KNY23]). *Let $\prod_{\mathsf{SIG}} = (\mathsf{Gen},\mathsf{Sign},\mathsf{Verify})$ denote a signature scheme that is one-time unforgeable for BB84 states. Consider a family of efficient quantum operations $\{\mathcal{Z}_\lambda(\cdot,\cdot,\cdot,\cdot,\cdot)\}_{\lambda\in\mathbb{N}}$, each taking five inputs: (i) a verification key vk, (ii) a signing key sigk, (iii) a λ-bit string θ, (iv) a single bit β, (v) a quantum register A. For any QPT algorithm $\mathcal{B}$, define the experiment $\widetilde{\mathcal{Z}}^{\mathcal{B}}_\lambda(b)$ over quantum states as the outcome of executing $\mathcal{B}$ according to the following procedure:*

- *Sample random strings $x,\theta\in\{0,1\}^\lambda$ and generate a key pair $(\mathsf{vk},\mathsf{sigk})\leftarrow\mathsf{Gen}(1^\lambda)$.*
- *Construct the quantum state $|\psi\rangle$ by applying the mapping*

$$|m\rangle\,|0\ldots0\rangle \;\longmapsto\; |m\rangle\,|\mathsf{Sign}(\mathsf{sigk},m)\rangle$$

  to the input $|x\rangle_\theta\otimes|0\ldots0\rangle$.
- *Initialize $\mathcal{B}$ with parameters*

$$\mathcal{Z}_\lambda\left(\mathsf{vk},\mathsf{sigk},\theta,\, b\oplus\bigoplus_{i:\theta_i=1} x_i,\,|\psi\rangle\right).$$

– *Parse $\mathcal{B}$'s output as a pair (x', σ') and a residual quantum state on register B.*

– *If $\mathsf{Verify}(\mathsf{vk}, x', \sigma') = \top$, output the final state of B; otherwise output the special symbol $\bot$.*

Suppose that, for every QPT adversary $\mathcal{A}$, any key pair $(\mathsf{vk}, \mathsf{sigk})$ of $\prod_{\mathsf{SIG}}$, any $\theta \in \{0,1\}^\lambda$, any $\beta \in \{0,1\}$, and any efficiently samplable quantum state $|\psi\rangle_{A,C}$ over registers A and C, the following properties hold:

$$\left|\Pr\left[\mathcal{A}(\mathcal{Z}_\lambda(\mathsf{vk}, \mathsf{sigk}, \theta, \beta, A), C) = 1\right] - \Pr\left[\mathcal{A}(\mathcal{Z}_\lambda(\mathsf{vk}, 0^{\ell_{\mathsf{sigk}}}, \theta, \beta, A), C) = 1\right]\right| \leq \mathsf{negl}(\lambda),$$
$$\left|\Pr\left[\mathcal{A}(\mathcal{Z}_\lambda(\mathsf{vk}, \mathsf{sigk}, \theta, \beta, A), C) = 1\right] - \Pr\left[\mathcal{A}(\mathcal{Z}_\lambda(\mathsf{vk}, \mathsf{sigk}, 0^\lambda, \beta, A), C) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

It then follows that, for every QPT adversary $\mathcal{B}$,

$$\mathsf{TD}(\widetilde{\mathcal{Z}}_\lambda^{\mathcal{B}}(0), \widetilde{\mathcal{Z}}_\lambda^{\mathcal{B}}(1)) \leq \mathsf{negl}(\lambda).$$

3.9 One-Shot Signature

In this section, we present the formal definition of a one-shot signature (OSS) scheme as introduced by Amos *et al.* [AGKZ20]. Such schemes are designed for settings where each key pair is used to sign at most one message and rely on quantum capabilities for key generation and signing.

Syntax. A one-shot signature scheme consists of a tuple of QPT algorithms $\prod_{\mathsf{OSS}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ over a message space $\mathcal{M}$ and signature space $\mathcal{S}$, which are described below.

– $(\mathsf{crs}) \leftarrow \mathsf{Setup}(1^\lambda)$. On input the security parameter 1^λ, the setup algorithm outputs a classical common reference string crs.

– $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(\mathsf{crs})$. Given the common reference string crs, the key generation algorithm outputs a key pair consisting of a classical public key pk and a quantum secret key sk.

– $(\sigma) \leftarrow \mathsf{Sign}(m, \mathsf{sk})$. The signing algorithm intakes the quantum secret-key sk and a message m from the message space $\mathcal{M}$, which have to sign. It outputs a signature $\sigma \in \mathcal{S}$ and makes publicly available.

– $(\top \vee \bot) \leftarrow \mathsf{Verify}(\mathsf{crs}, \mathsf{pk}, m, \sigma)$. Taking as input the common reference string crs, classical public-key pk, message m and the signature σ, the algorithm can verify whether the signature is valid or not.

$$\mathsf{Verify}(crs, pk, m, \sigma) = \begin{cases} \top & \text{if } \sigma \text{ is valid.} \\ \bot & \text{if } \sigma \text{ is invalid.} \end{cases}$$

A one-shot signature scheme must satisfy correctness and security properties as defined below.

- **Correctness.** We say that an OSS scheme is said to be correct if for all security parameters $\lambda \in \mathbb{N}$ and all messages $m \in \mathcal{M}$, the probability that a valid signature fails to verify is negligible:

$$\Pr\left[\mathsf{Verify}(crs, \mathsf{pk}, m, \sigma) = \bot \;\middle|\; \begin{array}{l} crs \leftarrow \mathsf{Setup}(1^\lambda), \\ (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(crs), \\ \sigma \leftarrow \mathsf{Sign}(m, \mathsf{sk}) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

- **Security.** We say that the one-shot signature scheme is secure if no QPT adversary $\mathcal{A}$ can produce two valid message-signature pairs under the same public key. Specifically, for all such adversaries $\mathcal{A}$ and for all $i \in \{0, 1\}$, we require:

$$\Pr\left[\mathsf{Verify}(crs, \mathsf{pk}, m_i, \sigma_i) = \top \;\wedge\; m_0 \neq m_1 \;\middle|\; \begin{array}{l} crs \leftarrow \mathsf{Setup}(1^\lambda), \\ (\mathsf{pk}, m_i, \sigma_i) \leftarrow \mathcal{A}(crs) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

  The one-shot nature of the signature illustrates that the signing key can be used to produce a signature on at most one message, and any attempt to produce a second distinct signed message should fail verification with overwhelming probability.

3.10 Registered Attribute-Based Encryption

In this section, we present the formal definition of a (key-policy) registered attribute-based encryption (RABE) scheme that supports an attribute universe of polynomial size, adapted from the framework introduced in [HLWW23]. The notion of RABE extends the concept of registration-based encryption (RBE) [GHMR18] by incorporating attribute-based access control into the registration-based paradigm.

Syntax. Let λ be the security parameter. We define the attribute universe as $\mathcal{U} = \{\mathcal{U}_\tau\}_{\tau \in \mathbb{N}}$, the policy space over $\mathcal{U}$ as $\mathcal{P} = \{\mathcal{P}_\tau\}_{\tau \in \mathbb{N}}$, where each $P \in \mathcal{P}_\tau$ is a mapping $P : \mathcal{P}_\tau : \mathcal{U}_\tau \to \{0, 1\}$ and the message space as $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$. A RABE scheme is defined as a tuple of PPT algorithms $\prod_{\mathsf{RABE}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt})$ with the following functionalities:

- $(\mathsf{crs}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$. The setup algorithm generates a common reference string crs based on the security parameter λ and the policy-family parameter τ.
- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$. Taking as input the common reference string crs, an auxiliary state aux, and a policy $P \in \mathcal{P}_\tau$, the key generation algorithm outputs a public/secret key pair $(\mathsf{pk}, \mathsf{sk})$.
- $(\mathsf{mpk}, \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$. The registration algorithm takes as input the common reference string crs, the current auxiliary state aux, a public key pk, and a policy $P \in \mathcal{P}_\tau$. It updates the auxiliary state to aux' and outputs the corresponding master public key mpk.

– $(\mathsf{ct}) \leftarrow \mathsf{Encrypt}(\mathsf{mpk}, X, \mu)$. Given the master public key mpk, an attribute $X \in \mathcal{U}_\tau$, and a message $\mu \in \mathcal{M}$, the encryption algorithm outputs a ciphertext ct.

– $(\mathsf{hsk}) \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$. The update algorithm, given the common reference string crs, the auxiliary state aux, and a public key pk, produces a helper key hsk to facilitate the decryption process.

– $(\mu') \leftarrow \mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, X, \mathsf{ct})$. Given a secret key sk, a helper key hsk,an attribute $X \in \mathcal{U}_\tau$ and a ciphertext ct, the decryption algorithm outputs either a message $\mu' \in \mathcal{M}$, the failure symbol $\perp$, or the flag $\mathsf{GetUpdate}$ to indicate that re-synchronization is required.

Correctness and Efficiency. A RABE scheme $\prod_{\mathsf{RABE}}$ is said to satisfy correctness if any honestly registered user can successfully decrypt ciphertexts whose associated policies are satisfied by their attribute set, regardless of the presence of adversarially registered keys. Specifically, suppose that a user registers her public key with an access policy. In that case, she must be able to decrypt all ciphertexts encrypted under the corresponding (or any subsequent) master public key, provided the attribute is satisfied by the access policy. This guarantee must hold even when malicious users register malformed keys, as long as the key curator behaves semi-honestly. Given a security parameter λ, policy parameter τ the correctness and efficiency of a RABE scheme is formally defined through the following interaction between an adversary $\mathcal{A}$ and a challenger.

- **Setup phase:** The challenger samples the common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$, initializes the auxiliary input $\mathsf{aux} \leftarrow \perp$ and initial master public key $\mathsf{mpk}_0 \leftarrow \perp$. It also sets the counters $\mathsf{ctr}_{\mathsf{reg}} \leftarrow 0$ and $\mathsf{ctr}_{\mathsf{enc}} \leftarrow 0$, and initializes $\mathsf{ctr}^*_{\mathsf{reg}} \leftarrow \infty$ to store the index of the target key. Finally, it sends crs to $\mathcal{A}$.
- **Query phase:** The adversary $\mathcal{A}$ may adaptively issue the following queries.
  - **Register non-target key query:** On input of a public key pk, and a access policy $P \in \mathcal{P}_\tau$, the challenger increments $\mathsf{ctr}_{\mathsf{reg}} \leftarrow \mathsf{ctr}_{\mathsf{reg}} + 1$ and computes $(\mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$. It updates $\mathsf{aux} \leftarrow \mathsf{aux}'$ and returns $(\mathsf{ctr}_{\mathsf{reg}}, \mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux})$ to $\mathcal{A}$.
  - **Register target key query:** On input a policy $P^* \in \mathcal{P}_\tau$, if a target key has already been registered, the challenger returns $\perp$. Otherwise, it increments the counter $\mathsf{ctr}_{\mathsf{reg}} \leftarrow \mathsf{ctr}_{\mathsf{reg}} + 1$ and generates $(\mathsf{pk}^*, \mathsf{sk}^*) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P^*)$. It then registers the key to obtain $(\mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*, P^*)$, and computes the helper decryption key $\mathsf{hsk}^* \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*)$. The challenger sets $\mathsf{aux} \leftarrow \mathsf{aux}'$, updates $\mathsf{ctr}^*_{\mathsf{reg}} \leftarrow \mathsf{ctr}_{\mathsf{reg}}$, and returns$(\mathsf{ctr}_{\mathsf{reg}}, \mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux}, \mathsf{pk}^*, \mathsf{sk}^*, \mathsf{hsk}^*)$ to $\mathcal{A}$.
  - **Encryption query:** On input of an public key index $\mathsf{ctr}^*_{\mathsf{reg}} \leq i \leq \mathsf{ctr}_{\mathsf{reg}}$, a message $\mu_{\mathsf{ctr}_{\mathsf{enc}}}$, and an attribute $X_{\mathsf{ctr}_{\mathsf{enc}}} \in \mathcal{U}_\tau$, if no target key has been registered or $P^*(X_{\mathsf{ctr}_{\mathsf{enc}}}) = 0$, the challenger returns $\perp$. Otherwise,

it increments the counter $\mathsf{ctr}_{\mathsf{enc}} \leftarrow \mathsf{ctr}_{\mathsf{enc}} + 1$ and generates $\mathsf{ct}_{\mathsf{ctr}_{\mathsf{enc}}} \leftarrow \mathsf{Encrypt}(\mathsf{mpk}_i, X_{\mathsf{ctr}_{\mathsf{enc}}}, \mu_{\mathsf{ctr}_{\mathsf{enc}}})$ and returns the pair $(\mathsf{ctr}_{\mathsf{enc}}, \mathsf{ct}_{\mathsf{ctr}_{\mathsf{enc}}})$.

– **Decryption query:** On input of a ciphertext index $1 \leq j \leq \mathsf{ctr}_{\mathsf{enc}}$, the challenger computes $m'_j \leftarrow \mathsf{Decrypt}(\mathsf{sk}^*, \mathsf{hsk}^*, X_j, \mathsf{ct}_j)$. If $m'_j = \mathsf{GetUpdate}$, it updates the helper key $\mathsf{hsk}^* \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*)$ and recomputes m'_j. If $m'_j \neq m_j$, the challenger sets $b \leftarrow 1$ and halts.

If the adversary has finished making queries and the experiment has not halted (as a result of a decryption query), then the experiment outputs $b = 0$.

The scheme $\prod_{\mathsf{RABE}}$ is correct and efficient if for all (possibly unbounded) adversaries $\mathcal{A}$ making at most a polynomial number of queries, the following properties hold:

- **Correctness:** There exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,
$$\Pr[b = 1] = \mathsf{negl}(\lambda).$$
It achieves perfect correctness if $\Pr[b = 1] = 0$.
- **Compactness:** Let N denote the number of registration queries. There exists a universal polynomial $\mathsf{poly}(\cdot, \cdot, \cdot)$ such that for all $i \in [N]$,
$$|\mathsf{mpk}_i| = \mathsf{poly}(\lambda, |\mathcal{U}|, \log i), \quad |\mathsf{hsk}^*| = \mathsf{poly}(\lambda, |\mathcal{U}|, \log N).$$
- **Update efficiency:** The Update algorithm is invoked at most $O(\log N)$ times, each execution running in $\mathsf{poly}(\log N)$ time in the RAM model, which allows sublinear input access.

Security. The security notion for a (key-policy) registered ABE scheme is analogous to the standard ABE. In the security game, the adversary may register users whose policies are satisfied by the challenge attribute set, as long as their secret keys are honestly generated by the challenger and thus unknown to the adversary. Furthermore, the adversary may register arbitrary public keys for policies of its choice, provided that none satisfy the challenge attributes.

Let $\prod_{\mathsf{RABE}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt})$ be a registered ABE scheme with attribute universe $\mathcal{U}$, policy space $\mathcal{P}$, and message space $\mathcal{M}$. For a security parameter λ, an adversary $\mathcal{A}$, and a bit $b \in \{0, 1\}$, we define the following security game $\mathsf{Exp}^{\mathsf{RABE}}_{\mathcal{A}}(\lambda, b)$ between $\mathcal{A}$ and the challenger:

- **Setup:** The challenger runs $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$ and initializes: $\mathsf{aux} \leftarrow \perp$, $\mathsf{mpk} \leftarrow \perp$, a counter $\mathsf{ctr} \leftarrow 0$ for honest-key registrations, $\mathcal{C} \leftarrow \emptyset$ for corrupted public keys, and a dictionary $D \leftarrow \emptyset$ mapping public keys to attribute sets. If $\mathsf{pk} \notin D$, then $D[\mathsf{pk}] \leftarrow \emptyset$. The challenger sends crs to $\mathcal{A}$.
- **Query phase:** In this phase, $\mathcal{A}$ may adaptively issue the following queries.

- **Register corrupted key query:** $\mathcal{A}$ submits a public key pk, and a policy $P \in \mathcal{P}_\tau$. The challenger computes $(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$, updates $\mathsf{mpk} \leftarrow \mathsf{mpk}'$, $\mathsf{aux} \leftarrow \mathsf{aux}'$, adds pk to $\mathcal{C}$, and sets $D[\mathsf{pk}] \leftarrow D[\mathsf{pk}] \cup \{P\}$. It returns $(\mathsf{mpk}', \mathsf{aux}')$ to $\mathcal{A}$.
- **Register honest key query:** $\mathcal{A}$ specifies a policy $P \in \mathcal{P}_\tau$. The challenger updates the counter $\mathsf{ctr} \leftarrow \mathsf{ctr}+1$ and generates a key pair $(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$. Subsequently, it executes $(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_{\mathsf{ctr}}, P)$, and updates $\mathsf{mpk} \leftarrow \mathsf{mpk}'$, $\mathsf{aux} \leftarrow \mathsf{aux}'$, and $D[\mathsf{pk}_{\mathsf{ctr}}] \leftarrow D[\mathsf{pk}_{\mathsf{ctr}}] \cup \{P\}$. Finally, it returns $(\mathsf{ctr}, \mathsf{mpk}', \mathsf{aux}', \mathsf{pk}_{\mathsf{ctr}})$ to $\mathcal{A}$.
- **Corrupt honest key query:** $\mathcal{A}$ submits $i \in [\mathsf{ctr}]$. The challenger retrieves $(\mathsf{pk}_i, \mathsf{sk}_i)$, adds pk_i to $\mathcal{C}$, and returns sk_i to $\mathcal{A}$.

- **Challenge:** $\mathcal{A}$ outputs two messages $\mu_0^*, \mu_1^* \in \mathcal{M}$ and an attribute $X \in \mathcal{U}_\tau$. The challenger samples a bit b from $\{0,1\}$, computes $\mathsf{ct}^* \leftarrow \mathsf{Encrypt}(\mathsf{mpk}, X^*, \mu_b^*)$, and returns ct^* to $\mathcal{A}$.
- **Output:** Finally, $\mathcal{A}$ outputs a bit $b' \in \{0,1\}$.

Let $S = \{P' \in D[\mathsf{pk}] \mid \mathsf{pk} \in \mathcal{C}\}$ be the collection of policies of all corrupted keys. An adversary $\mathcal{A}$ is admissible if for all $P \in S$, it holds that $P(X^*) = 0$. We say that $\prod_{\mathsf{RABE}}$ scheme is secure if for all efficient admissible adversaries, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE}}(\lambda, 0) = 1] - \Pr[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE}}(\lambda, 1) = 1] \right| = \mathsf{negl}(\lambda).$$

4 RABE with Certified Deletion and Everlasting Security

In this section, we introduce two variants of ciphertext deletion for registered attribute-based encryption. We first formalize the Registered Attribute-Based Encryption with Certified Deletion (RABE-CD), specifying its syntax, correctness, and security guarantees. We then upgrade this framework to capture the notion of Registered Attribute-Based Encryption with Certified Everlasting Deletion (RABE-CED), together with its corresponding correctness and security definitions. For both RABE-CD and RABE-CED systems, we further distinguish between privately and publicly verifiable settings and establish their respective security guarantees.

4.1 System Model of RABE-CD

Let λ be the security parameter, and $\mathcal{U} = \{\mathcal{U}_\tau\}_{\tau \in \mathbb{N}}$ denote the attribute universe. Let $\mathcal{P} = \{\mathcal{P}_\tau\}_{\tau \in \mathbb{N}}$ represent the policy space and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ represent the message space. A RABE-CD scheme consists of the tuple of efficient algorithms $\prod_{\mathsf{CD}}^{\mathsf{RABE}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt}, \mathsf{Delete}, \mathsf{Verify})$ with the following syntax:

- $(\mathsf{crs}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$. On the input of the length of the security parameter λ and the size of the attribute universe $\mathcal{U}$, the setup algorithm outputs a common reference string crs.
- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$. Given the common reference string crs, an auxiliary state aux (possibly empty) and a policy $P \in \mathcal{P}_\tau$, the key generation algorithm outputs a public key pk and secret key sk.
- $(\mathsf{mpk}, \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$. The registration algorithm takes as input a common reference string crs, an auxiliary state aux, a public key pk, and a policy $P \in \mathcal{P}_\tau$ to return an updated master public key mpk and auxiliary state aux'.
- $(\mathsf{vk}, \mathsf{ct}) \leftarrow \mathsf{Encrypt}(\mathsf{mpk}, X, \mu)$. The encryption algorithm takes as input the master public key mpk, an attribute $X \in \mathcal{U}_\tau$, and a message μ to produce a verification key vk together with a ciphertext ct.
- $(\mathsf{hsk}) \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$. Taking as input the common reference string crs, an auxiliary state aux, and a public key pk, the update algorithm deterministically outputs a helper secret key hsk.
- $(\mu \cup \{\perp, \mathsf{GetUpdate}\}) \leftarrow \mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, X, \mathsf{ct})$. On input of the secret key sk, a helper secret key hsk, an attribute $X \in \mathcal{U}_\tau$ and a ciphertext ct, the decryption algorithm outputs the message μ, a failure symbol $\perp$, or the flag $\mathsf{GetUpdate}$ indicating that an updated helper secret key is needed.
- $(\mathsf{cert}) \leftarrow \mathsf{Delete}(\mathsf{ct})$. The deletion algorithm generates a deletion certificate cert for the given ciphertext ct.
- $(b \in \{0,1\}) \leftarrow \mathsf{Verify}(\mathsf{vk}, \mathsf{cert})$. Given a verification key vk and a deletion certificate cert, the verification algorithm outputs 0 or 1.

4.2 Correctness and Efficiency of RABE-CD

Let $\prod_{\mathsf{CD}}^{\mathsf{RABE}}$ denote a registered attribute-based encryption scheme with certified deletion protocol defined over an attribute universe $\mathcal{U}$, a policy space $\mathcal{P}$, and a message space $\mathcal{M}$. To define the correctness of $\prod_{\mathsf{CD}}^{\mathsf{RABE}}$, we specify the following two experiments, Decryption Game and Verification Game, played between an adversary $\mathcal{A}$ and a challenger.

- **Setup phase:** The challenger starts by sampling the common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$. Then it initializes the auxiliary input as $\mathsf{aux} \leftarrow \perp$ and sets the initial master public key to $\mathsf{mpk}_0 \leftarrow \perp$. Next, it initializes a counter $\mathsf{ctr}_{\mathsf{reg}} \leftarrow 0$ to record the number of registration queries and another counter $\mathsf{ctr}_{\mathsf{enc}} \leftarrow 0$ to track the number of encryption queries. In addition, it sets $\mathsf{ctr}^*_{\mathsf{reg}} \leftarrow \infty$ as the index of the target key, which may be updated during the game. At the end of this phase, the challenger sends crs to $\mathcal{A}$.
- **Query phase:** The adversary $\mathcal{A}$ may adaptively issue the following queries.

- **Register non-target key query:** In this query, $\mathcal{A}$ provides a public key pk and a policy $P \in \mathcal{P}$. The challenger first increments the counter $\mathsf{ctr}_{\mathsf{reg}} \leftarrow \mathsf{ctr}_{\mathsf{reg}}+1$, then registers the key by computing $(\mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$. It updates the auxiliary data by setting $\mathsf{aux} \leftarrow \mathsf{aux}'$, and replies to $\mathcal{A}$ with $(\mathsf{ctr}_{\mathsf{reg}}, \mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux})$.
- **Register target key query:** In the register target key query, $\mathcal{A}$ specifies a policy $P^* \in \mathcal{P}$. If a target key has already been registered, the challenger replies with $\perp$. Otherwise, it increments the counter $\mathsf{ctr}_{\mathsf{reg}} \leftarrow \mathsf{ctr}_{\mathsf{reg}}+1$, samples $(\mathsf{pk}^*, \mathsf{sk}^*) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P^*)$, and registers the key by computing $(\mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*, P^*)$. Then it computes the helper decryption key $\mathsf{hsk}^* \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*)$, updates $\mathsf{aux} \leftarrow \mathsf{aux}'$, and records the index of the target key as $\mathsf{ctr}^*_{\mathsf{reg}} \leftarrow \mathsf{ctr}_{\mathsf{reg}}$. At the end, it responds to $\mathcal{A}$ with $(\mathsf{ctr}_{\mathsf{reg}}, \mathsf{mpk}_{\mathsf{ctr}_{\mathsf{reg}}}, \mathsf{aux}, \mathsf{pk}^*, \mathsf{hsk}^*, \mathsf{sk}^*)$.
- **Encryption query:** In an encryption query, $\mathcal{A}$ provides an index $\mathsf{ctr}^*_{\mathsf{reg}} \leq i \leq \mathsf{ctr}_{\mathsf{reg}}$, a message $\mu_{\mathsf{ctr}_{\mathsf{enc}}} \in \mathcal{M}$, and an attribute $X_{\mathsf{ctr}_{\mathsf{enc}}} \in \mathcal{U}_\tau$. If no target key has been registered or if $P(X_{\mathsf{ctr}_{\mathsf{enc}}}) = 0$, the challenger replies with $\perp$. Otherwise, it increments $\mathsf{ctr}_{\mathsf{enc}} \leftarrow \mathsf{ctr}_{\mathsf{enc}} + 1$ and computes $\mathsf{ct}_{\mathsf{ctr}_{\mathsf{enc}}} \leftarrow \mathsf{Encrypt}(\mathsf{mpk}_i, X_{\mathsf{ctr}_{\mathsf{enc}}}, \mu_{\mathsf{ctr}_{\mathsf{enc}}})$. The challenger then replies with $(\mathsf{ctr}_{\mathsf{enc}}, \mathsf{ct}_{\mathsf{ctr}_{\mathsf{enc}}})$.
- **Decryption query:** In a decryption query, the adversary submits a ciphertext index $1 \leq j \leq \mathsf{ctr}_{\mathsf{enc}}$. The challenger computes $m'_j \leftarrow \mathsf{Decrypt}(\mathsf{sk}^*, \mathsf{hsk}^*, X_j, \mathsf{ct}_j)$. If $m'_j = \mathsf{GetUpdate}$, it updates $\mathsf{hsk}^* \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*)$ and recomputes $m'_j \leftarrow \mathsf{Decrypt}(\mathsf{sk}^*, \mathsf{hsk}^*, X_j, \mathsf{ct}_j)$. If $m'_j \neq \mu_j$, the experiment halts with output $b = 0$ (Decryption Game).
- **Verification query:** In a verification query, the adversary submits a ciphertext index j. The challenger computes $\mathsf{cert}_j \leftarrow \mathsf{Delete}(\mathsf{ct}_j)$ and $v_j \leftarrow \mathsf{Verify}(\mathsf{vk}_j, \mathsf{cert}_j)$. If $v_j \neq 1$, the experiment halts with output $b = 0$ (Verification Game).

If the adversary finishes making queries without halting the Decryption Game, the experiment outputs $b = 1$. Similarly, if the Verification Game does not halt, the output is $b = 1$.

We say that the scheme $\prod^{\mathsf{RABE}}_{\mathsf{CD}}$ is correct and efficient if for all (possibly unbounded) $\mathcal{A}$ making at most polynomially many queries, the following hold:

- **Decryption correctness:** There exist a negligible function negl in λ such that $\Pr[b = 0] = \mathsf{negl}(\lambda)$ in the Decryption Game.
- **Verification correctness:** There exists a negligible function negl in λ such that $\Pr[b = 0] = \mathsf{negl}(\lambda)$ in the Verification Game.
- **Compactness:** Let N be the number of registration queries. There exists a polynomial poly such that for all $i \in [N]$,

$$|\mathsf{mpk}_i| = \mathsf{poly}(\lambda, |\mathcal{U}|, \log i), \quad |\mathsf{hsk}^*| = \mathsf{poly}(\lambda, |\mathcal{U}|, \log N).$$

- **Update efficiency:** Let N denote the number of registration queries issued by $\mathcal{A}$ in the above game. During the game, the challenger invokes Update at most $\mathcal{O}(\log N)$ times, and each invocation runs in $\mathsf{poly}(\log N)$ time in the RAM model. In particular, we model Update as a RAM program with random access to its input, then its running time may be smaller than the input length.

4.3 Certified Deletion Security of RABE-CD

Let $\prod_{\mathsf{CD}}^{\mathsf{RABE}}$ denote a registered attribute-based encryption with certified deletion protocol defined over an attribute universe $\mathcal{U}$, a policy space $\mathcal{P}$, and a message space $\mathcal{M}$. For a security parameter λ, a bit $b \in \{0,1\}$, and an adversary $\mathcal{A}$, we define the following security game $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CD}}(\lambda, b)$ between $\mathcal{A}$ and a challenger:

- **Setup phase:** The challenger samples the common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$. It then initializes the internal state as follows: auxiliary input $\mathsf{aux} \leftarrow \bot$, master public key $\mathsf{mpk} \leftarrow \bot$, and counter $\mathsf{ctr} \leftarrow 0$ to track the number of honest key registration queries. It also initializes empty sets $C \leftarrow \emptyset, H \leftarrow \emptyset$ to record corrupted and honest public keys, respectively, and an empty dictionary $D \leftarrow \emptyset$ to map public keys to their associated attribute sets. For notational convenience, if $\mathsf{pk} \notin D$, we define $D[\mathsf{pk}] := \emptyset$. Finally, the challenger sends crs to the adversary $\mathcal{A}$.
- **Query phase:** The adversary $\mathcal{A}$ can issue the following queries:
  - **Register corrupted key query:** $\mathcal{A}$ submits a public key pk along with a policy $P \in \mathcal{P}$. The challenger then executes

    $$(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$$

    and updates the following

    $$\mathsf{mpk} \leftarrow \mathsf{mpk}',\ C \leftarrow C \cup \{\mathsf{pk}\}$$
    $$\mathsf{aux} \leftarrow \mathsf{aux}',\ D[\mathsf{pk}] \leftarrow D[\mathsf{pk}] \cup \{P\}.$$

    Finally, the challenger replies to $\mathcal{A}$ with $(\mathsf{mpk}', \mathsf{aux}')$.
  - **Register honest key query:** In an honest-key-registration query, the adversary specifies a policy $P \in \mathcal{P}$. The challenger increments the counter $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$, samples a key pair $(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$, and registers it by computing $(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_{\mathsf{ctr}}, P)$. The challenger then updates its state as follows:

    $$\mathsf{mpk} \leftarrow \mathsf{mpk}',\ H \leftarrow H \cup \{\mathsf{pk}_{\mathsf{ctr}}\},$$
    $$\mathsf{aux} \leftarrow \mathsf{aux}',\ D[\mathsf{pk}_{\mathsf{ctr}}] \leftarrow D[\mathsf{pk}_{\mathsf{ctr}}] \cup \{P\}.$$

    Finally, it replies to the adversary with $(\mathsf{ctr}, \mathsf{mpk}', \mathsf{aux}', \mathsf{pk}_{\mathsf{ctr}})$.

– **Corrupt honest key query:** $\mathcal{A}$ specifies an index $1 \leq i \leq \mathsf{ctr}$. Let $(\mathsf{pk}_i, \mathsf{sk}_i)$ be the key pair generated during the i-th honest key query. The challenger updates its state by removing pk_i from H and adding it to C:

$$H \leftarrow H \setminus \{\mathsf{pk}_i\},\ C \leftarrow C \cup \{\mathsf{pk}_i\}.$$

It then returns sk_i to $\mathcal{A}$.

- **Challenge phase:** In the challenge phase, $\mathcal{A}$ chooses two messages $\mu_0^*, \mu_1^* \in \mathcal{M}$ together with an attribute $X^* \in \mathcal{U}_\tau$. Then it sends the tuple (μ_0^*, μ_1^*, X^*) to the challenger. The challenger first verifies that

$$P(X^*) = 0 \quad \text{for all } P \in \{D[\mathsf{pk}] : \mathsf{pk} \in C\}.$$

Subsequently, the challenger selects a random bit $b \in \{0,1\}$ and executes the encryption algorithm

$$(\mathsf{vk}^*, \mathsf{ct}_b^*) \leftarrow \mathsf{Encrypt}(\mathsf{mpk}, X^*, \mu_b^*),$$

and returns ct_b^* to $\mathcal{A}$.

- **Deletion phase:** At some point, the adversary $\mathcal{A}$ produces a deletion certificate cert and submits it to the challenger. The challenger verifies its validity by computing

$$\mathsf{valid} \leftarrow \mathsf{Verify}(\mathsf{vk}^*, \mathsf{cert}).$$

If $\mathsf{valid} = 0$, the challenger aborts the experiment and outputs $\perp$.
If $\mathsf{valid} = 1$, the challenger discloses the following information:
– All honest secret keys $\{\mathsf{sk}_i : \mathsf{pk}_i \in H\}$,
– All helper keys generated via the update algorithm

$$\{\mathsf{hsk}_i : \mathsf{hsk}_i \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_i),\ \mathsf{pk}_i \in H\}.$$

- **Output phase:** The adversary outputs a bit $b' \in \{0,1\}$.

We say that $\prod_{\mathsf{CD}}^{\mathsf{RABE}}$ is secure if for all efficient and admissible QPT adversaries $\mathcal{A}$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$

$$\left|\Pr\left[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE-CD}}(\lambda, 0) = 1\right] - \Pr\left[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE-CD}}(\lambda, 1) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

Depending on the nature of the verification key, the RABE-CD framework can be classified into two distinct variants: registered attribute-based encryption with privately verifiable certified deletion (RABE-PriVCD) and registered attribute-based encryption with publicly verifiable certified deletion (RABE-PubVCD).

Privately Verifiable Certified Deletion. The syntax, correctness, and security requirements of RABE-PriVCD follow directly from the definitions of RABE-CD provided in Section 4.1. It is important to note that the verification of

a deletion certificate is restricted to the sender, who is the sole possessor of the verification key. The security experiment for RABE-PriVCD, denoted by $\mathsf{Exp}^{\mathsf{RABE\text{-}PriVCD}}\mathcal{A}(\lambda, b)$, is identical to the experiment $\mathsf{Exp}^{\mathsf{RABE\text{-}CD}}\mathcal{A}(\lambda, b)$.

Publicly Verifiable Certified Deletion. The syntax and correctness of RABE-PubVCD follow the general framework of RABE-CD, described in Section 4.1, with the only modification in the encryption algorithm Encrypt. In this setting, encryption is realized through an interactive protocol between a sender Sndr and a receiver Rcvr. On input (mpk, X, μ) to Sndr, where mpk denotes the master public key, $X \subseteq \mathcal{U}$ stands an attribute set and μ the message, and no input to Rcvr, the protocol outputs a ciphertext and an associated verification key as follows:

$$(\mathsf{vk}, \mathsf{ct}) \leftarrow \mathsf{Encrypt}\big(\mathsf{Sndr}(\mathsf{mpk}, X, \mu), \mathsf{Rcvr}\big).$$

It is important to note that in the publicly verifiable model, anyone can check whether a deletion certificate in the verification algorithm is valid or not.

The security of RABE-PubVCD is evaluated using the same security game as $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CD}}(\lambda, b)$. The defining characteristic of the RABE-PubVCD security model is its resilience against an adversary who possesses the verification key vk. In other words, the confidentiality of the system remains intact even if vk is made publicly available. In this context, the security game for RABE-PubVCD is referred to as $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PubVCD}}(\lambda, b)$.

In the subsequent subsection, we consider the realization of ciphertext deletion functionalities within the RABE framework under certified everlasting security. This notion extends certified deletion by providing information-theoretic security guarantees after deletion, however, without revealing honest secret keys in the post-deletion phase..

4.4 System Model for RABE-CED

The syntax and correctness conditions of RABE with Certified Everlasting Deletion (RABE-CED) follow identically from those of RABE-CD, as defined in Sections 4.1 and 4.2. The only substantive difference lies in the notion of security. In RABE-CED, the adversary is assumed to be polynomial-time during protocol execution but may become computationally unbounded afterwards. Even with such post-execution power, any message whose deletion has been certified remains secure. In the following section, we now formalize the certified everlasting deletion security in the RABE framework.

4.5 Certified Everlasting Security for RABE-CED

Let $\prod_{\mathsf{CED}}^{\mathsf{RABE}}$ denote a registered attribute-based encryption scheme with certified everlasting deletion framework, defined over an attribute universe $\mathcal{U}$, a policy space $\mathcal{P}$, and a message space $\mathcal{M}$. Note that the syntax is same as $\prod_{\mathsf{CD}}^{\mathsf{RABE}}$. For a security parameter λ, a challenge bit $b \in \{0, 1\}$, and an adversary $\mathcal{A}$, we formalize

the certified everlasting security experiment $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CED}}(\lambda, b)$, played between $\mathcal{A}$ and a challenger, as follows.

- **Setup:** The challenger samples $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$, and initializes the internal state as follows:

$$\begin{aligned}&\mathsf{aux} \leftarrow \bot,\ C \leftarrow \emptyset,\ D \leftarrow \emptyset,\\ &\mathsf{mpk} \leftarrow \bot,\ \mathsf{ctr} \leftarrow 0,\ H \leftarrow \emptyset,\end{aligned}$$

  where C maintains the set of corrupted public keys, H maintains the set of honest public keys, and D records the attribute sets registered to each public key. Finally, the challenger provides crs to the adversary $\mathcal{A}$.

- **Query phase:** The adversary $\mathcal{A}$ can issue the following queries:
  - **Register Corrupted Key:** Upon receiving a submission $(\mathsf{pk}, P \in \mathcal{P})$ from $\mathcal{A}$, the challenger computes

$$(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P),$$

    updates its internal state as follows:

$$\begin{aligned}&\mathsf{mpk} \leftarrow \mathsf{mpk}',\ D[\mathsf{pk}] \leftarrow D[\mathsf{pk}] \cup \{P\}\\ &C \leftarrow C \cup \{\mathsf{pk}\},\ \mathsf{aux} \leftarrow \mathsf{aux}',\end{aligned}$$

    and returns $(\mathsf{mpk}', \mathsf{aux}')$ to $\mathcal{A}$.
  - **Register Honest Key:** When $\mathcal{A}$ specifies a policy $P \in \mathcal{P}$, the challenger increments the counter $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$, generates a fresh key pair

$$(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$$

    and executes the registration procedure

$$(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_{\mathsf{ctr}}, P).$$

    The challenger then updates its states:

$$\begin{aligned}&\mathsf{mpk} \leftarrow \mathsf{mpk}',\ D[\mathsf{pk}_{\mathsf{ctr}}] \leftarrow D[\mathsf{pk}_{\mathsf{ctr}}] \cup \{P\},\\ &\mathsf{aux} \leftarrow \mathsf{aux}',\ H \leftarrow H \cup \{\mathsf{pk}_{\mathsf{ctr}}\},\end{aligned}$$

    and responds with $(\mathsf{ctr}, \mathsf{mpk}', \mathsf{aux}', \mathsf{pk}_{\mathsf{ctr}})$.
  - **Corrupt Honest Key:** On input $i \in [1, \mathsf{ctr}]$, the challenger marks pk_i as corrupted:

$$C \leftarrow C \cup \{\mathsf{pk}_i\},\ H \leftarrow H \setminus \{\mathsf{pk}_i\},$$

    and outputs the corresponding secret key sk_i.

- **Challenge Phase:** The adversary $\mathcal{A}$ submits a challenge an attribute $X^* \in \mathcal{U}_\tau$. The challenger verifies that $P(X^*) = 0$ for all $P \in S$, where $S = \{P \in D[\mathsf{pk}] : \mathsf{pk} \in C\}$ and computes

$$(\mathsf{vk}^*, \mathsf{ct_b}^*) \leftarrow \mathsf{Encrypt}(\mathsf{mpk}, X^*, b),$$

  and returns $\mathsf{ct_b}^*$ to the adversary.

- **Output Phase:** The adversary $\mathcal{A}$ produces a deletion certificate cert together with a residual state ρ. The challenger verifies the certificate by computing

$$\mathsf{valid} \leftarrow \mathsf{Verify}(\mathsf{vk}^*, \mathsf{cert})$$

  If $\mathsf{valid} = 0$, the experiment terminates with output $\perp$. Otherwise, if $\mathsf{valid} = 1$, the experiment concludes with output ρ.

We say that $\prod_{\mathsf{CED}}^{\mathsf{RABE}}$ is secure if for all QPT adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that

$$\mathsf{TD}\left(\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CED}}(\lambda, 0), \mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CED}}(\lambda, 1)\right) \leq \mathsf{negl}(\lambda),$$

where $\mathsf{TD}(\cdot, \cdot)$ denotes the trace distance.

The experiment considers the challenge messages restricted to a single bit. The definition naturally extends to multi-bit messages by encrypting each bit independently and concatenating the resulting ciphertexts, where indistinguishability is preserved through a standard hybrid argument.

In line with the RABE-CD framework, the RABE-CED model also encompasses two variants, distinguished by the nature of the verification key: registered attribute-based encryption with privately verifiable certified everlasting deletion (RABE-PriVCED) and registered attribute-based encryption with publicly verifiable certified everlasting deletion (RABE-PubVCED).

Privately Verifiable Certified Everlasting Deletion. The syntax and correctness of RABE-PriVCED can be directly derived from the general framework of RABE-CD, as specified in Section 4.1. In addition, the security model for RABE-PriVCED is also identical to the experiment $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CED}}(\lambda, b)$.

Publicly Verifiable Certified Everlasting Deletion. The syntax and correctness of RABE-PubVCED also follow directly from the general framework of RABE-CD, as introduced in Section 4.1. Its security model coincides with the experiment $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CED}}(\lambda, b)$, except that during the challenge phase, the adversary additionally receives $(\mathsf{vk}^*, \mathsf{ct}_b^*)$ to enable public verifiability.

To realize the RABE-CD scheme under private and public verification settings, we need to introduce a fundamental cryptographic primitive termed Shadow Registered Attribute-Based Encryption (Shad-RABE). This primitive serves as the core building block that underpins our certified deletion constructions. In the subsequent section, we formalize the syntax and security of Shad-RABE and present its construction.

5 Shadow Registered Attribute-Based Encryption

In this section, we present the construction of our generic (key-policy) Shad-RABE protocol by composing registered attribute-based encryption, zero-knowledge argument and indistinguishability obfuscation in a modular manner. We formally analyze its security to establish that the resulting construction satisfies the desired privacy and correctness properties. Furthermore, we present a lattice-based instantiation of Shad-RABE, ensuring post-quantum security of the framework.

5.1 System Model of Shad-RABE

A (key-policy) shadow registered attribute-based encryption (Shad-RABE) scheme with attribute universe $\mathcal{U}$, policy space $\mathcal{P}$, and message space $\mathcal{M}$ is a tuple of efficient algorithms $\prod_{\mathsf{Shad\text{-}RABE}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt}, \mathsf{SimKeyGen}, \mathsf{SimRegPK}, \mathsf{SimCT}, \mathsf{SimCorrupt}, \mathsf{Reveal})$ with the following syntax:

- $(\mathsf{crs}) \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$. On input of the length of the security parameter λ and a policy-family parameter τ , the setup algorithm outputs a common reference string crs.
- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$. Given the common reference string crs, auxiliary state aux, and a policy $P \in \mathcal{P}$, this algorithm produces a public/secret key pair $(\mathsf{pk}, \mathsf{sk})$.
- $(\mathsf{mpk}, \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$. The registration algorithm takes as input a common reference string crs, an auxiliary state aux, a public key pk, and a policy $P \in \mathcal{P}$ to return a master public key mpk and an updated auxiliary state aux'.
- $(\mathsf{ct}) \leftarrow \mathsf{Encrypt}(\mathsf{mpk}, X, \mu)$. The sender encrypts a message $\mu \in \mathcal{M}$ under the master public key mpk and attribute $X \in \mathcal{U}_\tau$, producing a ciphertext ct.
- $(\mathsf{hsk}) \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$. This algorithm outputs a helper secret key hsk given the common reference string crs, an auxiliary state aux, and a registered public key pk as inputs.
- $(\mu \cup \{\perp, \mathsf{GetUpdate}\}) \leftarrow \mathsf{Decrypt}(\mathsf{ct}, \mathsf{sk}, \mathsf{hsk}, X)$. The decryptor executes this algorithm using the ciphertext ct, the secret key sk, $X \in \mathcal{U}_\tau$ and the helper secret key hsk as input. The algorithm outputs the message μ, the failure symbol $\perp$, or the instruction $\mathsf{GetUpdate}$ indicating that an updated helper secret key is required.
- $(\mathsf{pk}, \mathsf{B}') \leftarrow \mathsf{SimKeyGen}(\mathsf{crs}, \mathsf{aux}, P, \mathsf{B})$. Given the common reference string crs, an auxiliary state aux, a policy $P \in \mathcal{P}$, and an ancillary dictionary B (possibly empty) indexed by public keys, the simulated key generation algorithm outputs a public key pk together with an updated ancillary dictionary B'.
- $(\widetilde{\mathsf{mpk}}, \widetilde{\mathsf{aux}}) \leftarrow \mathsf{SimRegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P, \mathsf{B})$. The simulated key registration algorithm, given the common reference string crs, an auxiliary state aux, a public key pk, and a policy P, produces a simulated master public key $\widetilde{\mathsf{mpk}}$ and an updated auxiliary state $\widetilde{\mathsf{aux}}$.

– $(\widetilde{\mathsf{sk}}) \leftarrow \mathsf{SimCorrupt}(\mathsf{crs}, \mathsf{pk}, \mathsf{B})$. The simulated corruption algorithm takes as input the common reference string crs, the public key pk and an ancillary dictionary B to return a simulated secret key $\widetilde{\mathsf{sk}}$ corresponding to pk.

– $(\widetilde{\mathsf{ct}}) \leftarrow \mathsf{SimCT}(\widetilde{\mathsf{mpk}}, \mathsf{B}, X)$. The simulated ciphertext generation algorithm simulates the entire encryption algorithm under the attribute $X \in \mathcal{U}_\tau$ using the simulated master public key $\widetilde{\mathsf{mpk}}$, and an ancillary dictionary B, and outputs a simulated ciphertext $\widetilde{\mathsf{ct}}$.

– $(\widetilde{\mathsf{sk}}) \leftarrow \mathsf{Reveal}(\mathsf{pk}, \mathsf{B}, \widetilde{\mathsf{ct}}, \mu)$. Given a public key pk, a dictionary B, a simulated ciphertext $\widetilde{\mathsf{ct}}$, and a target message μ, this algorithm generates a simulated secret key $\widetilde{\mathsf{sk}}$.

Correctness and Efficiency of Shad-RABE. The Shad-RABE protocol preserves the same correctness and efficiency guarantees as the underlying registered ABE scheme.

Security of Shad-RABE. Let a (key-policy) shadow registered attribute-based encryption scheme $\prod_{\mathsf{Shad\text{-}RABE}}$ be defined over an attribute universe $\mathcal{U}$, policy space $\mathcal{P}$ and message space $\mathcal{M}$. For a security parameter λ, a bit $b \in \{0,1\}$, and an adversary $\mathcal{A}$, we define the following security game $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{Shad\text{-}RABE}}(\lambda, b)$ between $\mathcal{A}$ and a challenger:

- **Setup.** The challenger begins by generating a common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^{|\mathcal{U}|})$. It initializes its internal variables as follows: the auxiliary state $\mathsf{aux} \leftarrow \perp$, the master public key $\mathsf{mpk} \leftarrow \perp$, and a counter $\mathsf{ctr} \leftarrow 0$ to track the number of honestly registered keys. Additionally, two empty sets $C \leftarrow \emptyset$ and $H \leftarrow \emptyset$ are initialized to store corrupted and honest public keys, respectively. The challenger also initializes empty dictionaries, $D \leftarrow \emptyset$ and $\mathsf{B} \leftarrow \emptyset$, indexed by the public keys. For ease of notation, it is assumed that if $\mathsf{pk} \notin D$, then $D[\mathsf{pk}] := \emptyset$ and same for B. Finally, the challenger sends the common reference string crs to the adversary $\mathcal{A}$.
- **Query Phase:** During this phase, the adversary $\mathcal{A}$ is allowed to make the following types of queries:
  - **Register corrupted key query.** Upon receiving a public key pk and a policy $P \in \mathcal{P}$ from $\mathcal{A}$, the challenger proceeds as follows:

$$(\mathsf{mpk}', \mathsf{aux}') \leftarrow \begin{cases} \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P), & \text{if } b = 0 \\ \mathsf{SimRegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P, \mathsf{B}), & \text{if } b = 1 \end{cases}$$

    It then updates its internal state as:

$$\mathsf{mpk} \leftarrow \mathsf{mpk}', \mathsf{aux} \leftarrow \mathsf{aux}', C \leftarrow C \cup \{\mathsf{pk}\}, D[\mathsf{pk}] \leftarrow D[\mathsf{pk}] \cup \{P\}.$$

    The challenger returns $(\mathsf{mpk}', \mathsf{aux}')$ to $\mathcal{A}$.

– **Register honest key query:** When $\mathcal{A}$ specifying a policy $P \in \mathcal{P}$, the challenger increments its counter $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$, and performs the following based on the bit b:

$$\left.\begin{aligned}(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) &\leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)\\ (\mathsf{mpk}', \mathsf{aux}') &\leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_{\mathsf{ctr}}, P)\end{aligned}\right\} \quad \text{if } \ b = 0$$

$$\left.\begin{aligned}(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{B}') &\leftarrow \mathsf{SimKeyGen}(\mathsf{crs}, \mathsf{aux}, \mathsf{B})\\ (\mathsf{mpk}', \mathsf{aux}') &\leftarrow \mathsf{SimRegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_{\mathsf{ctr}}, P, \mathsf{B})\end{aligned}\right\} \quad \text{if } \ b = 1.$$

It then updates:

$$\mathsf{mpk} \leftarrow \mathsf{mpk}', \mathsf{aux} \leftarrow \mathsf{aux}', H \leftarrow H \cup \{\mathsf{pk}_{\mathsf{ctr}}\}, D[\mathsf{pk}_{\mathsf{ctr}}] \leftarrow D[\mathsf{pk}_{\mathsf{ctr}}] \cup \{P\}, \mathsf{B} \leftarrow \mathsf{B}'.$$

It then replies with $(\mathsf{ctr}, \mathsf{mpk}', \mathsf{aux}', \mathsf{pk}_{\mathsf{ctr}})$.

– **Corrupt honest key query:** The adversary may request the corruption of an honest key by specifying an index i such that $1 \leq i \leq \mathsf{ctr}$. The challenger responds based on the bit b:

(a) If $b = 0$, it returns the actual secret key sk_i.

(b) If $b = 1$, it computes a simulated secret key $\widetilde{\mathsf{sk}}_i \leftarrow \mathsf{SimCorrupt}(\mathsf{crs}, \mathsf{pk}_i, \mathsf{B})$.

In either case, pk_i is removed from the honest set H and added to the corrupted set C.

- **Challenge phase:** The adversary $\mathcal{A}$ selects a message $\mu \in \mathcal{M}$ and an attribute $X^* \in \mathcal{U}_\tau$ such that $P(X^*) = 0$ for all $P \in S$, where $S = \{P \in D[\mathsf{pk}] \mid \mathsf{pk} \in C\}$. The challenger then prepares the challenge ciphertext as follows:

$$\mathsf{ct}^* \leftarrow \begin{cases}\mathsf{Encrypt}(\mathsf{mpk}, X^*, \mu) & \text{if } b = 0,\\ \mathsf{SimCT}(\mathsf{mpk}, \mathsf{aux}, \mathsf{B}, X^*) & \text{if } b = 1.\end{cases}$$

For every honest public key $\mathsf{pk}_i \in H$, the challenger generates the corresponding secret and helper keys as follows:

$$\mathsf{sk}_i^* \leftarrow \begin{cases}\mathsf{sk}_i & \text{if } b = 0,\\ \mathsf{Reveal}(\mathsf{pk}_i, \mathsf{B}, \mathsf{ct}^*, \mu) & \text{if } b = 1,\end{cases}$$
$$\mathsf{hsk}_i \leftarrow \mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_i).$$

The challenger then returns to the adversary the tuple: $(\mathsf{ct}^*, \{\mathsf{sk}_i^*\}_{\mathsf{pk}_i \in H}, \{\mathsf{hsk}_i\}_{\mathsf{pk}_i \in H})$.

- **Output phase:** The adversary finally outputs a bit $b' \in \{0, 1\}$.

A Shad-RABE scheme is said to be secure if for every efficient quantum polynomial-time (QPT) adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{Shad\text{-}RABE}}(\lambda, 0) = 1\right] - \Pr\left[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{Shad\text{-}RABE}}(\lambda, 1) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

5.2 Generic Construction of Shad-RABE Protocol

In this section, we construct a shadow registered attribute-based encryption (Shad-RABE) scheme with plaintext space $\{0,1\}^{\ell_m}$, attribute universe $\mathcal{U}$, and policy space $\mathcal{P}$. Our construction builds upon the following cryptographic components:

- A registered attribute-based encryption scheme with the tuple of algorithms $\prod_{\mathsf{RABE}} = \mathsf{RABE}.(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt})$ defined over the same $\mathcal{U}$ and $\mathcal{P}$.
- An interactive proof system given by the interactive machines $\langle \mathcal{P}, \mathcal{V} \rangle$ for an NP language $\mathcal{L}$ characterized by the relation R defined as:

$$R := \Big\{ \big((\{\mathsf{mpk}_{i,0}, \mathsf{mpk}_{i,1}\}_{i\in[\ell_m]}, \{\mathsf{CT}_{i,0}, \mathsf{CT}_{i,1}\}_{i\in[\ell_m]}, X), \{(\mu[i], r_{i,0}, r_{i,1})\}_{i\in[\ell_m]} \big) \\ \mid \forall i \in [\ell_m],\ \forall b \in \{0,1\},\ \mathsf{CT}_{i,b} = \mathsf{RABE.Encrypt}(\mathsf{mpk}_{i,b}, X, \mu[i]; r_{i,b}) \Big\}.$$

  Each $r_{i,b}$ is sampled uniformly from the randomness space of the RABE.Encrypt algorithm.
- An indistinguishability obfuscator $i\mathcal{O}$ for polynomial-size circuits.

The algorithms of Shad-RABE are detailed below.

$\mathsf{Setup}(1^\lambda, 1^\tau)$:
- For every $i \in [\ell_m]$ and $b \in \{0,1\}$, generate $\mathsf{crs}_{i,b} \leftarrow \mathsf{RABE.Setup}(1^\lambda, 1^\tau)$.
- Sample $y \leftarrow \{0,1\}^*$ uniformly at random.
- Output $\mathsf{crs} := (\{\mathsf{crs}_{i,b}\}_{i\in[\ell_m],\ b\in\{0,1\}}, y)$.

$\mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$:
- Parse $\mathsf{crs} = (\{\mathsf{crs}_{i,b}\}_{i,b}, y)$.
- Sample $z \leftarrow \{0,1\}^{\ell_m}$ uniformly at random.
- For all $i \in [\ell_m]$, $b \in \{0,1\}$, generate $(\mathsf{pk}_{i,b}, \mathsf{sk}_{i,b}) \leftarrow \mathsf{RABE.KeyGen}(\mathsf{crs}_{i,b}, \mathsf{aux}_{i,b}, P)$.
- Set $\mathsf{pk} := (\{\mathsf{pk}_{i,b}\}_{i,b})$.
- Construct the secret key $\mathsf{sk} := i\mathcal{O}(\mathcal{D}[\{\mathsf{sk}_{i,z[i]}\}_i, z])$ where the circuit is defined in Fig. 2.
- Output $(\mathsf{pk}, \mathsf{sk})$.

$\mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$:
- Parse $\mathsf{crs} = (\{\mathsf{crs}_{i,b}\}_{i,b}, y)$ and $\mathsf{pk} = (\{\mathsf{pk}_{i,b}\}_{i,b})$.
- If $\mathsf{aux} = \perp$, initialize $\mathsf{aux}_{i,b} := \perp$ for all i, b.
- For each i, b, compute $(\mathsf{mpk}_{i,b}, \mathsf{aux}'_{i,b}) \leftarrow \mathsf{RABE.RegPK}(\mathsf{crs}_{i,b}, \mathsf{aux}_{i,b}, \mathsf{pk}_{i,b}, P)$.
- Set $\mathsf{mpk} := (\{\mathsf{mpk}_{i,b}\}_{i,b}, y)$ and $\mathsf{aux}' := \{\mathsf{aux}'_{i,b}\}_{i,b}$.
- Output $(\mathsf{mpk}, \mathsf{aux}')$.

$\mathsf{Encrypt}(\mathsf{mpk}, \mathrm{X}, \mu)$:

- Parse $\mathsf{mpk} = (\{\mathsf{mpk}_{i,b}\}_{i,b}, y)$
- For each i, b, compute $\mathsf{CT}_{i,b} \leftarrow \mathsf{RABE.Encrypt}(\mathsf{mpk}_{i,b}, X, \mu[i]; r_{i,b})$.
- Let $d := (\{\mathsf{mpk}_{i,b}, \mathsf{CT}_{i,b}\}_{i,b}, X)$ and $w := (\mu, \{r_{i,b}\}_{i,b})$.
- Compute $\pi \leftarrow \langle \mathcal{P}(w), \mathcal{V}(y) \rangle(d)$.
- Output $\mathsf{ct} := (\{\mathsf{CT}_{i,b}\}_{i,b}, \pi)$.

$\mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$:

- Parse $\mathsf{crs} = (\{\mathsf{crs}_{i,b}\}_{i,b}, y)$, $\mathsf{aux} = \{\mathsf{aux}_{i,b}\}_{i,b}$, and $\mathsf{pk} = \{\mathsf{pk}_{i,b}\}_{i,b}$.
- For each $i \in [\ell_m]$, $b \in \{0,1\}$, compute $\mathsf{hsk}_{i,b} \leftarrow \mathsf{RABE.Update}(\mathsf{crs}_{i,b}, \mathsf{aux}_{i,b}, \mathsf{pk}_{i,b})$.
- Output $\mathsf{hsk} := \{\mathsf{hsk}_{i,b}\}_{i,b}$.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, \mathrm{X}, \mathsf{ct})$:

- Parse $\mathsf{sk} = \mathcal{D}_e$ and $\mathsf{hsk} = \{\mathsf{hsk}_{i,b}\}_{i,b}$.
- Compute and return $\mu := \mathcal{D}_e(\mathsf{hsk}, \mathsf{ct})$.

Left or Right Decryption Circuit $\mathcal{D}$

Input: hsk, CT
Hardcoded: $\{\mathsf{sk}_{i,z[i]}\}_{i\in[\ell_m]}$, $z \in \{0,1\}^{\ell_m}$

1. Parse $\mathsf{CT} = \left(\{\mathsf{CT}_{i,0}, \mathsf{CT}_{i,1}\}_{i\in[\ell_m]}, \pi\right)$
2. Parse $\mathsf{hsk} = \left(\{\mathsf{hsk}_{i,0}, \mathsf{hsk}_{i,1}\}_{i\in[\ell_m]}\right)$
3. If $\pi \neq 1$, then output $\perp$
4. For each $i \in [\ell_m]$, compute:

$$m[i] \leftarrow \mathsf{RABE.Decrypt}(\mathsf{sk}_{i,z[i]}, \mathsf{hsk}_{i,z[i]}, X, \mathsf{CT}_{i,z[i]})$$

5. Output: $m := m[1] \parallel m[2] \parallel \cdots \parallel m[\ell_m]$

Fig. 2: Left or Right Decryption Circuit

$\mathsf{SimKeyGen}(\mathsf{crs}, \mathsf{aux}, P, \mathsf{B})$:

- Parse $\mathsf{crs} = (\{\mathsf{crs}_{i,b}\}_{i,b}, y)$.
- For all $i \in [\ell_m], b \in \{0,1\}$, compute $(\mathsf{pk}_{i,b}, \mathsf{sk}_{i,b}) \leftarrow \mathsf{RABE.KeyGen}(\mathsf{crs}_{i,b}, \mathsf{aux}_{i,b}, P)$.
- Sample $z^*_{\mathsf{pk}} \leftarrow \{0,1\}^{\ell_m}$ uniformly at random.
- Set $\mathsf{pk} := (\{\mathsf{pk}_{i,b}\}_{i,b})$ and update $\mathsf{B}[\mathsf{pk}] = (\{\mathsf{sk}_{i,b}\}_{i,b}, z^*_{\mathsf{pk}})$.
- Output $(\widetilde{\mathsf{pk}}, \mathsf{B})$.

$\mathsf{SimRegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$:

- Parse $\mathsf{crs} = (\{\mathsf{crs}_{i,b}\}_{i,b}, y)$ and $\mathsf{pk} = (\{\mathsf{pk}_{i,b}\}_{i,b})$.

- For each i, b, compute $(\widetilde{\mathsf{mpk}}_{i,b}, \widetilde{\mathsf{aux}}_{i,b}) \leftarrow \mathsf{RABE.RegPK}(\mathsf{crs}_{i,b}, \mathsf{aux}_{i,b}, \mathsf{pk}_{i,b}, P)$.
- Set $\widetilde{\mathsf{mpk}} := (\{\widetilde{\mathsf{mpk}}_{i,b}\}_{i,b}, y)$ and $\widetilde{\mathsf{aux}} := \{\widetilde{\mathsf{aux}}_{i,b}\}_{i,b}$.
- Output $(\widetilde{\mathsf{mpk}}, \widetilde{\mathsf{aux}})$.

$\mathsf{SimCorrupt}(\mathsf{crs}, \mathsf{pk}, \mathsf{B})$:

- Parse $\mathsf{pk} = (\{\mathsf{pk}_{i,b}\}_{i,b})$ and $\mathsf{B}[\mathsf{pk}] = (\{\mathsf{sk}_{i,b}\}_{i,b}, z^*_{\mathsf{pk}})$.
- Compute $\widetilde{\mathsf{sk}} := iO(\mathcal{D}_0[\{\mathsf{sk}_{i,0}\}_i])$, where the circuit is defined in Fig. 3.
- Output $\widetilde{\mathsf{sk}}$.

Left Decryption Circuit $\mathcal{D}_0$

Input: hsk, CT
Hardcoded: $\{\mathsf{sk}_{i,0}\}_{i\in[\ell_m]}$

1. Parse $\mathsf{CT} = \big(\{(\mathsf{CT}_{i,0}, \mathsf{CT}_{i,1})\}_{i\in[\ell_m]}, \pi\big)$
2. Parse $\mathsf{hsk} = \big(\{\mathsf{hsk}_{i,0}, \mathsf{hsk}_{i,1}\}_{i\in[\ell_m]}\big)$
3. If $\pi \neq 1$, output $\perp$
4. For each $i \in [\ell_m]$, compute:
$$m[i] \leftarrow \mathsf{RABE.Decrypt}(\mathsf{sk}_{i,0}, \mathsf{hsk}_{i,0}, X, \mathsf{CT}_{i,0})$$
5. Output: $m = m[1] \,\|\, m[2] \,\|\, \cdots \,\|\, m[\ell_m]$

Fig. 3: Left Decryption Circuit

$\mathsf{SimCT}(\widetilde{\mathsf{mpk}}, \mathsf{B}, X)$:

- Parse $\widetilde{\mathsf{mpk}} = (\{\widetilde{\mathsf{mpk}}_{i,b}\}_{i,b}, y)$.
- Compute $z^* := \bigoplus_{\mathsf{pk}\in\mathsf{B}} z^*_{\mathsf{pk}}$ where $\mathsf{B}[\mathsf{pk}] = (\{\mathsf{sk}_{i,b}\}_{i,b}, z^*_{\mathsf{pk}})$.
- For each $i \in [\ell_m]$, compute:
$$\mathsf{CT}^*_{i,z^*[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,z^*[i]}, X, 0),$$
$$\mathsf{CT}^*_{i,1-z^*[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,1-z^*[i]}, X, 1).$$
- Define $d^* := (\{\widetilde{\mathsf{mpk}}_{i,0}, \widetilde{\mathsf{mpk}}_{i,1}, \mathsf{CT}^*_{i,0}, \mathsf{CT}^*_{i,1}\}_i, X)$.
- Compute $\widetilde{\pi} \leftarrow \langle \mathsf{Sim}\rangle(d^*, y)$.
- Output $\widetilde{\mathsf{ct}} := (\{\mathsf{CT}^*_{i,b}\}_{i,b}, \widetilde{\pi})$.

$\mathsf{Reveal}(\mathsf{pk}, \mathsf{B}, \widetilde{\mathsf{ct}}, \mu)$:

- Parse $\mathsf{B}[\mathsf{pk}] = (\{\mathsf{sk}_{i,b}\}_{i,b}, z^*)$.
- Parse $\mu = \mu[1] \,\|\, \mu[2] \,\|\, \cdots \,\|\, \mu[\ell_m]$.

– Compute $\widetilde{\mathsf{sk}} := iO(\mathcal{D}[\{\mathsf{sk}_{i,z^*[i]\oplus\mu[i]}\}_i, z^* \oplus \mu])$.

– Output $\widetilde{\mathsf{sk}}$.

Theorem 3. *If the registered attribute-based encryption scheme $\prod_{RABE}$ is secure and assuming the existence of an indistinguishability obfuscator $i\mathcal{O}$ for P/poly, and an interactive zero-knowledge argument system $\prod_{\mathsf{ZKA}}$ for NP language, then the Shad-RABE construction is secure.*

Proof. We define a sequence of hybrid experiments to transition between real and ideal security games.

- Hyb_0: This corresponds precisely to the security experiment $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{Shad\text{-}RABE}}(\lambda, 0)$. Throughout the analysis, x^* denotes the challenge attribute set and μ refers to the target message chosen by the adversary.

- Hyb_1: This hybrid modifies Hyb_0 by altering the generation of secret keys during the query phase. Instead of using the left-or-right decryption circuit

$$\mathcal{D}[\{\mathsf{sk}_{i,z[i]}\}_{i\in[\ell_m]}, z],$$

the challenger now employs the left-only decryption circuit

$$\mathcal{D}_0[\{\mathsf{sk}_{i,0}\}_{i\in[\ell_m]}].$$

Consequently, for each key-generation query, it provides

$$\widetilde{\mathsf{sk}} = iO\left(\mathcal{D}_0[\{\mathsf{sk}_{i,0}\}_{i\in[\ell_m]}]\right),$$

instead of

$$\mathsf{sk} = iO\left(\mathcal{D}[\{\mathsf{sk}_{i,z[i]}\}_{i\in[\ell_m]}, z]\right).$$

As a result, the secret keys in this hybrid no longer depend on the random selection bit string z.

- Hyb_2: This game is the same as Hyb_1 apart from the fact that it transitions from real to simulated machine $\langle Sim\rangle$ of an interactive proof system given by the interactive machines $\langle \mathcal{P}, \mathcal{V}\rangle$ for an NP language $\mathcal{L}$ characterized by the relation R in the ciphertext.

- Hyb_3: This game is the same as Hyb_2 except that it introduces asymmetric encryption patterns in the challenge ciphertext. Rather than encrypting the same message bit under both master public keys, the challenger now generates position-dependent encryptions. For the combined randomness $z := \bigoplus z_{\text{user}}$, where z_{user} is sampled in KeyGen for every user and each bit position $i \in [\ell_m]$, it computes:

$$\mathsf{CT}^*_{i,z[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,z[i]}, X^*, \mu[i]),$$
$$\mathsf{CT}^*_{i,1-z[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,1-z[i]}, X^*, 1-\mu[i]).$$

- Hyb_4: This game is the same as Hyb_3, except that it is completely independent of the target message μ. The challenger uses the same combined randomness $z^* := \bigoplus z^*_{\text{user}}$, where z^*_{user} is sampled in KeyGen for every user and generates challenge ciphertexts according to:

$$\mathsf{CT}^*_{i,z^*[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,z^*[i]}, X^*, 0),$$
$$\mathsf{CT}^*_{i,1-z^*[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,1-z^*[i]}, X^*, 1).$$

When revealing secret keys in the final phase, the challenger computes for each honest user with randomness z^*_{user}:

$$\mathsf{sk} := iO(\mathcal{D}[\{\mathsf{sk}_{i,z^*_{\text{user}}[i]\oplus\mu[i]}\}_{i\in[\ell_m]}, z^*_{\text{user}} \oplus \mu]).$$

Note that the game Hyb_4 is identical to the simulated experiment $\mathsf{Exp}^{\mathsf{Shad\text{-}RABE}}_{\mathcal{A}}(\lambda, 1)$.

Hybrid	Secret Keys	uNIZK Proofs	Challenge Ciphertext	Dependence on μ
Hyb_0	$iO(\mathcal{D}[\{\mathsf{sk}_{i,z[i]}\}_{i\in[\ell_m]}, z])$	Real proofs	$\mathsf{CT}^*_{i,0}, \mathsf{CT}^*_{i,1}$ encrypt $\mu[i]$	Yes (depends on μ)
Hyb_1	$iO(\mathcal{D}_0[\{\mathsf{sk}_{i,0}\}_{i\in[\ell_m]}])$	↓	↓	↓
Hyb_2	↓	Simulated proofs using $\langle Sim\rangle$	↓	↓
Hyb_3	↓	↓	Asymmetric encryptions: $\mathsf{CT}^*_{i,z[i]}, \mathsf{CT}^*_{i,1-z[i]}$	↓
Hyb_4	$iO(\mathcal{D}[\{\mathsf{sk}_{i,z^*_{\text{user}}[i]\oplus\mu[i]}\}, z^*_{\text{user}} \oplus \mu])$	↓	Encrypt fixed bits 0/1 under z^* randomness	No (independent of μ)

Table 1: Summary of the hybrid sequence used in the security proof of $\mathsf{Shad\text{-}RABE}$. The decryption circuit column shows the obfuscated program used to answer key queries. Simulated proof $\langle Sim\rangle$ appear in Hyb_2, and asymmetric challenge ciphertexts are introduced in Hyb_3. In Hyb_4, the ciphertext becomes entirely independent of the challenge message μ. Shaded entries mark the first point of deviation from the previous hybrid. Arrows (↓) indicate no change from the previous hybrid.

Let the distribution of the output of an execution of Hyb_j, with adversary $\mathcal{A}$, be denoted as $\mathsf{Hyb}_j(\mathcal{A})$. We establish computational indistinguishability between consecutive hybrid games through the following sequence of reductions, overview shown in Fig. 4.

Lemma 3. *Suppose that $\prod_{RABE}$ satisfies perfect correctness, $\prod_{\mathsf{ZKA}}$ achieves soundness property, and the indistinguishability obfuscator $i\mathcal{O}$ ensures security under indistinguishability. Then, the following advantage in distinguishing between hybrids is negligible,*

$$|\Pr[\mathsf{Hyb}_0(\mathcal{A}) = 1] - \Pr[\mathsf{Hyb}_1(\mathcal{A}) = 1]| \leq \mathsf{negl}(\lambda).$$

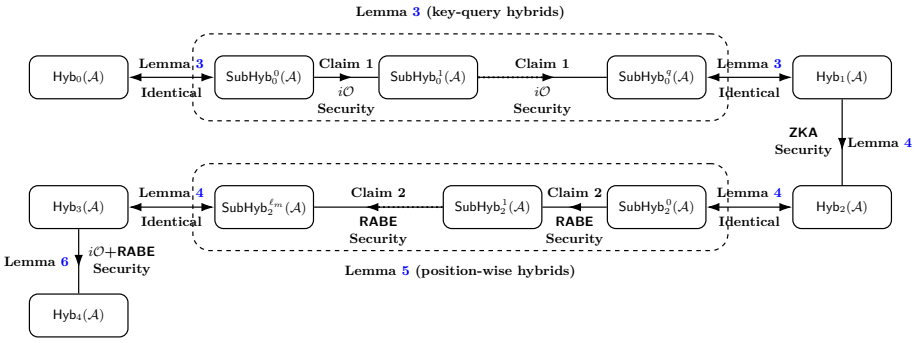

Fig. 4: A single arrow '⟶' represents statistical/computational indistinguishability. The double arrow '⟷' indicates that for both $b \in \{0,1\}$, the adversary is in the same experiment. Above each arrow, we write the lemma/claim that proves indistinguishability, while below, we state the underlying security assumption.

Proof. We employ a hybrid argument over the adversary's key-generation queries. Let q denote the total number of such queries made by the adversary $\mathcal{A}$.

Now, we define a sequence of hybrid sub-experiments SubHyb_0^j for each $j \in [0,q]$, where the challenger responds to the adversary's key queries as follows:

- For queries $k \in [j+1,q]$, the challenger returns $\mathsf{sk} = i\mathcal{O}\left(\mathcal{D}\left[\{\mathsf{sk}_{i,z[i]}\}_{i\in[\ell_m]}, z\right]\right)$.
- For queries $k \in [1,j]$, the challenger returns $\widetilde{\mathsf{sk}} = i\mathcal{O}\left(\mathcal{D}_0\left[\{\mathsf{sk}_{i,0}\}_{i\in[\ell_m]}\right]\right)$.

We observe that $\mathsf{SubHyb}_0^0(\mathcal{A}) \equiv \mathsf{Hyb}_0(\mathcal{A})$, and $\mathsf{SubHyb}_0^q(\mathcal{A}) \equiv \mathsf{Hyb}_1(\mathcal{A})$.

Let $\mathsf{Invalid}$ denote the event that there exists a statement $d^\dagger \notin \mathcal{L}$ such that for $\langle \mathcal{P}(w), \mathcal{V}(y)\rangle(d) = 1$. By the soundness of the interactive argument system $\prod_{\mathsf{ZKA}}$, it holds that

$$\Pr[\mathsf{Invalid}] \leq \mathsf{negl}(\lambda).$$

Conditioned on the event $\neg\mathsf{Invalid}$, the circuits $\mathcal{D}$ and $\mathcal{D}_0$ are functionally equivalent on all valid inputs. This equivalence follows from their construction and the perfect correctness of the underlying RABE scheme. We now formalize the indistinguishability of adjacent subgames.

Claim 1. For every $j \in [1,q]$, the distinguishing advantage between hybrid subexperiments $\mathsf{SubHyb}_0^{j-1}(\mathcal{A})$ and $\mathsf{SubHyb}_0^j(\mathcal{A})$ is negligible, *i.e.*,

$$\left|\Pr[\mathsf{SubHyb}_0^{j-1}(\mathcal{A}) = 1] - \Pr[\mathsf{SubHyb}_0^j(\mathcal{A}) = 1]\right|.$$

Proof. Consider the transition between hybrid sub-experiments SubHyb_0^{j-1} and SubHyb_0^j, which differ only in the j-th key query. In SubHyb_0^{j-1}, the response is

$$\mathsf{sk} = i\mathcal{O}\left(\mathcal{D}\left[\{\mathsf{sk}_{i,z[i]}\}_{i\in[\ell_m]}, z\right]\right),$$

while in SubHyb_0^j, the response is

$$\widetilde{\mathsf{sk}} = i\mathcal{O}\left(\mathcal{D}_0\left[\{\mathsf{sk}_{i,0}\}_{i\in[\ell_m]}\right]\right).$$

Under the assumption $\neg\mathsf{Invalid}$, the two circuits $\mathcal{D}$ and $\mathcal{D}_0$ are functionally equivalent. Hence, by the security of indistinguishability obfuscator $i\mathcal{O}$, any QPT distinguisher can distinguish between these responses with at most negligible probability.

Applying the hybrid sub-experiments argument across the q key queries and invoking the above claim repeatedly, we conclude that

$$|\Pr[\mathsf{Hyb}_0(\mathcal{A})=1] - \Pr[\mathsf{Hyb}_1(\mathcal{A})=1]| \le q \cdot \mathsf{negl}(\lambda) = \mathsf{negl}(\lambda),$$

which completes the proof of the lemma.

Lemma 4. *If the interactive argument system $\prod_{ZKA}$ satisfies the zero-knowledge property, then*

$$|\Pr[\mathsf{Hyb}_1(\mathcal{A})=1] - \Pr[\mathsf{Hyb}_2(\mathcal{A})=1]| \le \mathsf{negl}(\lambda).$$

Proof. The hybrid experiments Hyb_1 and Hyb_2 differ solely in the generation of the ciphertext. Suppose that $|\Pr[\mathsf{Hyb}_1(\mathcal{A})=1] - \Pr[\mathsf{Hyb}_2(\mathcal{A})=1]|$ is non-negligible. Then there exists a distinguisher that separates the ciphertext distributions used in Hyb_1 and Hyb_2 with non-negligible advantage.

This would further imply that the distributions of the real and simulated proof, namely

$$\langle\mathcal{P}(w),\mathcal{V}(y)\rangle(d) \quad \text{and} \quad \langle\mathsf{Sim}\rangle(d^*,y),$$

are computationally distinguishable. Such a distinguisher contradicts the zero-knowledge property of $\prod_{\mathsf{ZKA}}$, since it would allow one to efficiently distinguish the real view of the verifier from a simulated one for instances of size $|y|$ with non-negligible advantage.

Hence, by the zero-knowledge property, the advantage $\left|\Pr[\mathsf{Hyb}_1(\mathcal{A})=1] - \Pr[\mathsf{Hyb}_2(\mathcal{A})=1]\right|$ must be negligible.

Lemma 5. *If the underlying RABE scheme $\prod_{RABE}$ is secure, as described in Section 3.10, then*

$$|\Pr[\mathsf{Hyb}_2(\mathcal{A})=1] - \Pr[\mathsf{Hyb}_3(\mathcal{A})=1]| \le \mathsf{negl}(\lambda).$$

Proof. We employ a position-by-position hybrid argument across the message bits. For each position $i \in [\ell_m]$, we demonstrate that modifying the encryption pattern at position i introduces at most a negligible distinguishing advantage.

We now define the hybrid sub-experiments games SubHyb_2^j for $j \in [0,\ell_m]$, where the challenger responds as follows:

– For positions $i \in [j+1, \ell_m]$, both ciphertexts encrypt the same message bit.

$$\mathsf{CT}^*_{i,b} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,b}, X^*, \mu[i]) \text{ for } b \in \{0,1\}$$

– For positions $i \in [1, j]$, the ciphertexts are generated using the following computation.

$$\mathsf{CT}^*_{i,1-z[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,1-z[i]}, X^*, 1-\mu[i]),$$
$$\mathsf{CT}^*_{i,z[i]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{i,z[i]}, X^*, \mu[i])$$

By construction, $\mathsf{SubHyb}_2^0(\mathcal{A}) \equiv \mathsf{Hyb}_2(\mathcal{A})$ and $\mathsf{SubHyb}_2^{\ell_m}(\mathcal{A}) \equiv \mathsf{Hyb}_3(\mathcal{A})$. To complete the proof, we introduce the following claim.

Claim 2. For every $j \in [1, \ell_m]$, the hybrids $\mathsf{SubHyb}_2^{j-1}(\mathcal{A})$ and $\mathsf{SubHyb}_2^{j}(\mathcal{A})$ are computationally indistinguishable under the security of the RABE scheme, *i.e.,*

$$\left|\Pr[\mathsf{SubHyb}_2^{j-1}(\mathcal{A}) = 1] - \Pr[\mathsf{SubHyb}_2^{j}(\mathcal{A}) = 1]\right|.$$

Proof. To show that adjacent games SubHyb_2^{j-1} and SubHyb_2^{j} for $i \in [1, \ell_m]$, are computationally indistinguishable, we define a reduction $\mathcal{B}_{\mathsf{rabe}}$ against the security of the underlying RABE scheme.

$\mathcal{B}_{\mathsf{rabe}}$ receives master public key $\widetilde{\mathsf{mpk}}$ from the RABE challenger of $\mathsf{Exp}^{\mathsf{RABE}}_{\mathcal{A}}(\lambda, b')$ for $b' \in \{0,1\}$ and sets $\widetilde{\mathsf{mpk}}_{j,1-z[j]} := \widetilde{\mathsf{mpk}}$. For all other public keys, $(i,b) \neq (j, 1-z[j])$, it generates $\widetilde{\mathsf{mpk}}_{i,b}$ itself. It then compiles the overall master public key as

$$\widetilde{\mathsf{mpk}} := \left(\{\widetilde{\mathsf{mpk}}_{i,b}\}_{i\in[\ell_m], b\in\{0,1\}}, y\right),$$

which is given to the adversary $\mathcal{A}$. Similar to before, $\mathcal{B}_{\mathsf{rabe}}$ receives $\mathsf{sk}_{j,1-z[j]}$ from the RABE challenger, and locally generates all other secret keys. Since these keys are never used in any decryption or response, the simulation remains indistinguishable. $\mathcal{B}_{\mathsf{rabe}}$ then constructs

$$\widetilde{\mathsf{sk}} := iO\left(\mathcal{D}_0[\{\mathsf{sk}_{i,0}\}_{i\in[\ell_m]}]\right).$$

At the challenge phase, $\mathcal{A}$ submits (X^*, μ) to the $\mathcal{B}_{\mathsf{rabe}}$ and it sends the message pair $(\mu[j], 1-\mu[j])$ to the RABE challenger. It then computes $\mathsf{CT}^*_{j,z[j]} \leftarrow \mathsf{RABE.Encrypt}\ (\widetilde{\mathsf{mpk}}_{j,z[j]},\ X^*, \mu[j])$, and all remaining ciphertexts $\mathsf{CT}^*_{i,b}$ for $i \neq j$, as in the hybrids SubHyb_2^{j-1} and SubHyb_2^{j}. Finally, it outputs whatever $\mathcal{A}$ outputs.

The reduction proceeds as follows:

– If $b' = 0$, then $\mathsf{CT}^*_{j,1-z[j]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{j,1-z[j]}, X^*, \mu[j])$, which implies that $\mathcal{B}_{\mathsf{rabe}}$ perfect simulation of the hybrid sub-experiment SubHyb_2^{j-1}.

– If $b' = 1$, then $\mathsf{CT}^*_{j,1-z[j]} \leftarrow \mathsf{RABE.Encrypt}(\widetilde{\mathsf{mpk}}_{j,1-z[j]}, X^*, 1-\mu[j])$, which implies that $\mathcal{B}_{\mathsf{rabe}}$ perfect simulation of the hybrid sub-experiment SubHyb_2^{j-1}.

Hence, any non-negligible advantage by $\mathcal{A}$ in distinguishing between SubHyb_2^{j-1} and SubHyb_2^{j} can be used to break the security of the RABE scheme. Therefore, the hybrid sub-experiments are computationally indistinguishable.

Applying the hybrid sub-experiment argument to all ℓ_m positions and using the above claim iteratively, we conclude that

$$|\Pr[\mathsf{Hyb}_2(\mathcal{A}) = 1] - \Pr[\mathsf{Hyb}_3(\mathcal{A}) = 1]| \leq \ell_m \cdot \mathsf{negl}(\lambda) = \mathsf{negl}(\lambda).$$

This concludes the proof of Lemma 5.

Lemma 6. *The hybrid games satisfy perfect indistinguishability:*

$$\Pr\left[\mathsf{Hyb}_3(\mathcal{A}) = 1\right] = \Pr\left[\mathsf{Hyb}_4(\mathcal{A}) = 1\right].$$

Proof. This hybrid transition is a purely formal change that preserves the adversary's view. Specifically, the two games become identical under the variable substitution $z := z^* \oplus \mu$.

In game Hyb_3, the challenge ciphertexts are given by

$$\mathsf{CT}^*_{i,1-z[i]} \text{ encrypts } 1-\mu[i],$$
$$\mathsf{CT}^*_{i,z[i]} \text{ encrypts } \mu[i].$$

After applying the substitution $z := z^* \oplus \mu$, the ciphertexts in Hyb_4 take the form

$$\mathsf{CT}^*_{i,z^*[i]} \text{ encrypts } 0,$$
$$\mathsf{CT}^*_{i,1-z^*[i]} \text{ encrypts } 1.$$

Since the secret keys are generated independently of the selection string z in both hybrids, this change of variables does not affect the underlying distributions. Consequently, the adversary's view is identical across the two hybrids, and we conclude that

$$\Pr[\mathsf{Hyb}_3(\mathcal{A}) = 1] = \Pr[\mathsf{Hyb}_4(\mathcal{A}) = 1].$$

Consequently, by combining Lemmas 3, 4, 5, and 6, we conclude that the Shad-RABE construction satisfies security against all quantum polynomial-time adversaries.

5.3 Instatiation of Shad-RABE from Lattices

Our generic construction of Shad-RABE, presented in Section 5.2 is fully modular and can be instantiated from any secure (key-policy) registered ABE scheme, zero-knowledge argument system, and indistinguishability obfuscation. In this

section, we give a concrete lattice-based instantiation of each component, resulting in a quantum-secure Shad-RABE scheme.

We instantiate our generic framework using the following state-of-the-art lattice-based primitives:

- **Registered ABE Component:** We employ the key-policy registered ABE scheme of Champion *et al.* [CHW25], which supports arbitrary bounded-depth circuit policies. This scheme is proven secure under the falsifiable ℓ-succinct LWE assumption in the random oracle model and provides succinct ciphertexts whose size remains independent of the attribute length.

- **ZKA Component:** We utilize the post-quantum secure zero-knowledge argument system of Bitansky *et al.* [BS20] for arbitrary NP languages. Their construction achieves soundness and zero-knowledge under the plain LWE assumption against quantum adversaries.

- **Indistinguishability Obfuscation Component:** We utilize the lattice-based $i\mathcal{O}$ construction of Cini *et al.* [CLW25], which provides post-quantum security under the equivocal LWE assumption in conjunction with standard NTRU lattice assumptions. This represents the first plausibly post-quantum secure $i\mathcal{O}$ construction from well-studied lattice problems.

Adapting the above-mentioned component as a building block, we can construct a post-quantum secure Shad-RABE protocol from lattices.

6 RABE with Privately Verifiable Certified Deletion

This section first presents a generic construction of registered attribute-based encryption with privately verifiable certified deletion (RABE-PriVCD). The design builds upon the Shad-RABE protocol (*cf.* Section 5.2) and incorporates a symmetric-key encryption scheme with certified deletion (SKE-CD) that guarantees one-time certified deletion (OT-CD) security (*cf.* Section 3.7). Following the generic model, we then propose an instantiation of the RABE-PriVCD scheme, which relies on standard hardness assumptions over lattices.

6.1 Generic Construction of RABE-PriVCD.

We construct a registered attribute-based encryption scheme with privately verifiable certified deletion, denoted by $\prod^{\mathsf{RABE}}_{\mathsf{PriVCD}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt}, \mathsf{Delete}, \mathsf{Verify})$ from the following building blocks: a Shad-RABE scheme, specified by the algorithms, $\prod_{\mathsf{Shad\text{-}RABE}} = \mathsf{Shad\text{-}RABE}.(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt}, \mathsf{SimKeyGen}, \mathsf{SimRegPK}, \mathsf{SimCorrupt}, \mathsf{SimCT}, \mathsf{Reveal})$, and a symmetric-key encryption scheme with certified deletion, comprising algorithms, $\prod_{\mathsf{SKE\text{-}CD}} = \mathsf{SKE\text{-}CD}.(\mathsf{KeyGen}, \mathsf{Encrypt}, \mathsf{Decrypt}, \mathsf{Delete}, \mathsf{Verify})$. The detailed algorithms of RABE-PriVCD protocol are specified below.

$\mathsf{Setup}(1^\lambda, 1^\tau)$:
- Generate $\mathsf{crs} \leftarrow \mathsf{Shad\text{-}RABE.Setup}(1^\lambda, 1^\tau)$.
- Output crs.

$\mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$:
- Generate $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Shad\text{-}RABE.KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$.
- Output $(\mathsf{pk}, \mathsf{sk})$.

$\mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$:
- Generate $(\mathsf{mpk}, \mathsf{aux}') \leftarrow \mathsf{Shad\text{-}RABE.RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$.
- Output $(\mathsf{mpk}, \mathsf{aux}')$.

$\mathsf{Encrypt}(\mathsf{mpk}, X, \mu)$:
- Parse mpk.
- Generate $\mathsf{ske.sk} \leftarrow \mathsf{SKE\text{-}CD.KeyGen}(1^\lambda)$.
- Compute $\mathsf{srabe.ct} \leftarrow \mathsf{Shad\text{-}RABE.Encrypt}(\mathsf{mpk}, X, \mathsf{ske.sk})$.
- Compute $\mathsf{ske.ct} \leftarrow \mathsf{SKE\text{-}CD.Encrypt}(\mathsf{ske.sk}, \mu)$.
- Output ciphertext $\mathsf{ct} := (\mathsf{srabe.ct}, \mathsf{ske.ct})$ and verification key $\mathsf{vk} := \mathsf{ske.sk}$.

$\mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$:
- Generate $\mathsf{hsk} \leftarrow \mathsf{Shad\text{-}RABE.Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$.
- Output hsk.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, X, \mathsf{ct})$:
- Parse $\mathsf{ct} = (\mathsf{srabe.ct}, \mathsf{ske.ct})$.
- Compute $\mathsf{sk}' \leftarrow \mathsf{Shad\text{-}RABE.Decrypt}(\mathsf{sk}, \mathsf{hsk}, X, \mathsf{srabe.ct})$.
- If $\mathsf{sk}' = \perp$ or $\mathsf{sk}' = \mathsf{GetUpdate}$, output sk'.
- Otherwise, compute and output $\mu' \leftarrow \mathsf{SKE\text{-}CD.Decrypt}(\mathsf{sk}', \mathsf{ske.ct})$.

$\mathsf{Delete}(\mathsf{ct})$:
- Parse $\mathsf{ct} = (\mathsf{srabe.ct}, \mathsf{ske.ct})$.
- Generate $\mathsf{ske.cert} \leftarrow \mathsf{SKE\text{-}CD.Delete}(\mathsf{ske.ct})$.
- Output certificate $\mathsf{cert} := \mathsf{ske.cert}$.

$\mathsf{Verify}(\mathsf{vk}, \mathsf{cert})$:
- Parse $\mathsf{vk} = \mathsf{ske.sk}$ and $\mathsf{cert} = \mathsf{ske.cert}$.
- Output $b \leftarrow \mathsf{SKE\text{-}CD.Verify}(\mathsf{ske.sk}, \mathsf{ske.cert})$.

Correctness of RABE-PrivCD. The correctness of the RABE-PrivCD protocol follows directly from the correctness of $\prod_{\mathsf{Shad\text{-}RABE}}$ and $\prod_{\mathsf{SKE\text{-}CD}}$ protocols.

6.2 Proof of Security

This section presents a provable theorem model for the certified deletion security of the $\prod^{\mathsf{RABE}}_{\mathsf{PriVCD}}$ protocol.

Theorem 4. *If the scheme $\prod_{\mathit{Shad\text{-}RABE}}$ is secure and $\prod_{\mathsf{SKE\text{-}CD}}$ satisfies the OT-CD security property, then the protocol $\prod^{\mathit{RABE}}_{\mathit{PriVCD}}$ is secure.*

Proof. Let $\mathcal{A}$ be a QPT adversary and b be a bit. We define the following hybrid game $\mathsf{Hyb}(b)$.

- $\mathsf{Hyb}(b)$: This corresponds to a modification of the original experiment $\mathsf{Exp}^{\mathsf{RABE\text{-}PriVCD}}_{\mathcal{A}}(\lambda, b)$, where actual arguments are replaced with simulated ones. The simulation is carried out as follows:
  - **Setup:** The challenger samples $\mathsf{crs} \leftarrow \mathsf{Shad\text{-}RABE.Setup}(1^\lambda, 1^\tau)$ and initializes the internal state as
  $$\mathsf{aux} \leftarrow \bot,\ \mathsf{mpk} \leftarrow \bot,\ D \leftarrow \emptyset,$$
  $$\mathsf{ctr} \leftarrow 0,\ C \leftarrow \emptyset,\ H \leftarrow \emptyset.$$
  - **Query Phase:** The challenger responds to the adversary's queries using the simulation algorithms:
    (a) For a corrupted key query, upon receiving (pk, P) from the adversary $\mathcal{A}$, the challenger computes
    $$(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{Shad\text{-}RABE.SimRegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P, \mathsf{B}),$$
    and updates the states as
    $$\mathsf{aux} \leftarrow \mathsf{aux}',\ D[\mathsf{pk}] \leftarrow D[\mathsf{pk}] \cup \{P\},$$
    $$\mathsf{mpk} \leftarrow \mathsf{mpk}',\ C \leftarrow C \cup \{\mathsf{pk}\}.$$
    (b) For an honest key query with $P \in \mathcal{P}$, the challenger increments $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$, samples
    $$(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{B}) \leftarrow \mathsf{Shad\text{-}RABE.SimKeyGen}(\mathsf{crs}, \mathsf{aux}, P, \mathsf{B}),$$
    and computes
    $$(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{Shad\text{-}RABE.SimRegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P, \mathsf{B}).$$
    The states are then updated as follows:
    $$\mathsf{mpk} \leftarrow \mathsf{mpk}',\ H \leftarrow H \cup \{\mathsf{pk}_{\mathsf{ctr}}\}$$
    $$\mathsf{aux} \leftarrow \mathsf{aux}',\ D[\mathsf{pk}_{\mathsf{ctr}}] \leftarrow D[\mathsf{pk}_{\mathsf{ctr}}] \cup \{P\}.$$

(c) On corruption of an honest index $i \in [1, \mathsf{ctr}]$ with $\mathsf{pk}_i \in H$, the challenger computes

$$\mathsf{sk}_i \leftarrow \mathsf{Shad\text{-}RABE.SimCorrupt}(\mathsf{crs}, \mathsf{pk}_i, \mathsf{B}),$$

updates

$$H \leftarrow H \setminus \{\mathsf{pk}_i\},\ C \leftarrow C \cup \{\mathsf{pk}_i\},$$

and returns sk_i to $\mathcal{A}$.

– **Challenge:** On receiving (μ_0, μ_1, X^*) from $\mathcal{A}$, the challenger generates

$$\mathsf{ske.sk} \leftarrow \mathsf{SKE\text{-}CD.KeyGen}(1^\lambda),$$

and computes

$$\begin{aligned} \mathsf{srabe.ct}^* &\leftarrow \mathsf{Shad\text{-}RABE.SimCT}(\mathsf{mpk}, \mathsf{B}, X^*), \\ \mathsf{ske.ct}^* &\leftarrow \mathsf{SKE\text{-}CD.Encrypt}(\mathsf{ske.sk}, \mu_0). \end{aligned}$$

It then outputs the challenge pair

$$\mathsf{ct}^* := (\mathsf{srabe.ct}^*, \mathsf{ske.ct}^*),\ \mathsf{vk}^* := \mathsf{ske.sk}$$

to the adversary.

– **Deletion:** After ciphertext deletion, for each uncorrupted $\mathsf{pk}_i \in H$, the challenger computes

$$\begin{aligned} \mathsf{sk}_i &\leftarrow \mathsf{Shad\text{-}RABE.Reveal}(\mathsf{pk}_i, \mathsf{B}, \mathsf{srabe.ct}^*, \mathsf{ske.sk}), \\ \mathsf{hsk}_i &\leftarrow \mathsf{Shad\text{-}RABE.Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_i), \end{aligned}$$

and returns both values to $\mathcal{A}$.

We now state two lemmas that complete the proof.

Lemma 7. *If Shad-RABE is secure, then for every QPT adversary $\mathcal{A}$, there is a negligible* negl *such that*

$$\left|\Pr[\mathsf{Exp}_{\mathcal{A}}^{\mathit{RABE\text{-}PriVCD}}(\lambda, b) = 1] - \Pr[\mathsf{Hyb}(b) = 1]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Suppose that $\mathcal{A}$ distinguishes the two experiments with a non-negligible advantage. We construct a reduction $\mathcal{B}$ that breaks the security of Shad-RABE. The challenger of $\mathsf{Exp}_{\mathcal{B}}^{\mathsf{Shad\text{-}RABE}}(\lambda, b')$ for $b' \in \{0, 1\}$ provides crs to $\mathcal{B}$, which forwards it to $\mathcal{A}$ and initializes C, H, D as in the security game.

All queries of $\mathcal{A}$ are answered by relaying them to the Shad-RABE challenger through its registration and corruption interfaces, with $\mathcal{B}$ maintaining C, H, D consistently.

Upon receiving the challenge input (μ_0, μ_1, X^*) from $\mathcal{A}$, $\mathcal{B}$ samples $\mathsf{ske.sk} \leftarrow \mathsf{SKE\text{-}CD.KeyGen}(1^\lambda)$ and submits $(X^*, \mathsf{ske.sk})$ to its own challenger. It obtains $(\mathsf{srabe.ct}^*, \{\mathsf{sk}_i^*\}_{\mathsf{pk}_i \in H}, \{\mathsf{hsk}_i\}_{\mathsf{pk}_i \in H})$ from the challenger, then computes

$$\mathsf{ske.ct}^* \leftarrow \mathsf{SKE\text{-}CD.Encrypt}(\mathsf{ske.sk}, \mu_b),$$

and returns $(\mathsf{srabe.ct}^*, \mathsf{ske.ct}^*)$ together with $\mathsf{vk}^* = \mathsf{ske.sk}$ to $\mathcal{A}$.

If $\mathcal{A}$ later produces a certificate cert, $\mathcal{B}$ verifies it using $\mathsf{SKE\text{-}CD.Verify}(\mathsf{ske.sk}, \mathsf{cert})$. On success, $\mathcal{B}$ forwards $\{\mathsf{sk}_i^*\}_{\mathsf{pk}_i \in H}$ and $\{\mathsf{hsk}_i\}_{\mathsf{pk}_i \in H}$ to $\mathcal{A}$; otherwise it returns $\perp$. Finally, $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

If $b' = 0$, the simulation is perfect and matches $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}CD}}(\lambda, b)$. If $b' = 1$, the challenger runs the simulation algorithms, and the view of $\mathcal{A}$ is identical to $\mathsf{Hyb}(b)$. Thus, any non-negligible distinguishing advantage of $\mathcal{A}$ implies a break of $\mathsf{Shad\text{-}RABE}$, completing the proof.

Lemma 8. *If* $\mathsf{SKE\text{-}CD}$ *satisfies one-time certified deletion, then for every QPT adversary* $\mathcal{A}$*, there is a negligible* negl *such that*

$$\left|\Pr[\mathsf{Hyb}(0) = 1] - \Pr[\mathsf{Hyb}(1) = 1]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Suppose $\mathcal{A}$ distinguishes $\mathsf{Hyb}(0)$ from $\mathsf{Hyb}(1)$. We construct $\mathcal{B}$ against one-time certified deletion in the experiment $\mathsf{Exp}_{\mathsf{SKE\text{-}CD},\mathcal{B}}^{\mathsf{OT-CD}}(\lambda, b')$ for $b' \in \{0, 1\}$. The reduction generates $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^{|\mathcal{U}|})$, forwards it to $\mathcal{A}$, and answers all registration and corruption queries using the $\mathsf{Shad\text{-}RABE}$ simulation algorithms, keeping the same state as in $\mathsf{Hyb}(\cdot)$. On receiving the challenge (μ_0, μ_1, X^*) from $\mathcal{A}$, it submits (μ_0, μ_1) to its own challenger and obtains $\mathsf{ske.ct}^*$ (encrypting $\mu_{b'}$). It then prepares $\mathsf{ct}_{\mathsf{Shad}}^* \leftarrow \mathsf{Shad\text{-}RABE.SimCT}(\mathsf{mpk}, \mathsf{B}, X^*)$ and returns the combined challenge ciphertext $\mathsf{ct}^* = (\mathsf{srabe.ct}^*, \mathsf{ske.ct}^*)$.

If $\mathcal{A}$ outputs a certificate cert, $\mathcal{B}$ forwards it to its challenger. On success, the challenger releases $\mathsf{ske.sk}$, which $\mathcal{B}$ uses to compute $\mathsf{sk}_i \leftarrow \mathsf{Shad\text{-}RABE.Reveal}(\mathsf{pk}_i, \mathsf{B}, \mathsf{srabe.ct}^*, \mathsf{ske.sk})$ for each $\mathsf{pk}_i \in H$, and forwards these to $\mathcal{A}$; otherwise $\perp$ is returned.

When $b' = 0$, the reduction perfectly simulates $\mathsf{Hyb}(0)$; when $b' = 1$, the view coincides with $\mathsf{Hyb}(1)$.

Thus, a distinguisher between $\mathsf{Hyb}(0)$ and $\mathsf{Hyb}(1)$ breaks the one-time certified deletion property of $\mathsf{SKE\text{-}CD}$, which concludes the lemma.

Combining Lemma 7 and Lemma 8, we obtain

$$\begin{aligned}
\left|\Pr[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PriVCD}}(\lambda, 0) = 1] - \Pr[\mathsf{Hyb}(0) = 1]\right| &\leq \mathsf{negl}(\lambda), \\
\left|\Pr[\mathsf{Hyb}(0) = 1] - \Pr[\mathsf{Hyb}(1) = 1]\right| &\leq \mathsf{negl}(\lambda), \\
\left|\Pr[\mathsf{Hyb}(1) = 1] - \Pr[\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PriVCD}}(\lambda, 1) = 1]\right| &\leq \mathsf{negl}(\lambda).
\end{aligned}$$

Together, these bounds establish Theorem 4.

6.3 Instantiation of RABE-PriVCD

We now describe a lattice-based instantiation of our RABE-PriVCD. The construction follows the generic construction developed in the preceding sections and combines state-of-the-art lattice-based primitives in order to achieve post-quantum security. In particular, the instantiation of RABE-PriVCD relies on two fundamental building blocks, both of which can be realized from standard and well-studied lattice assumptions.

- **Shad-RABE Instantiation.** We instantiate the Shad-RABE component using lattice-based primitive as described in Section 5.3. The instantiation of Shad-RABE is secure under the ℓ-succinct LWE, plain LWE, and equivocal LWE assumptions.
- **Symmetric Encryption with Certified Deletion.** From Theorem 2, we invoke the existence of an unconditionally secure one-time symmetric-key encryption scheme with certified deletion (SKE-CD) [BI20]. This primitive guarantees one-time certified deletion security, which is essential to the design of RABE-PriVCD.

Putting the above components together, we obtain a lattice-based instantiation of RABE-PriVCD. The resulting scheme achieves post-quantum security under the ℓ-succinct LWE, plain LWE, and equivocal LWE assumptions, thereby demonstrating the plausibility of secure certified-deletion functionalities in the lattice setting.

7 RABE with Publicly Verifiable Certified Deletion

In this section, we provide a generic construction of registered attribute-based encryption with publicly verifiable certified deletion (RABE-PubVCD) and then describe its lattice-based instantiation. Our construction integrates three core cryptographic primitives: witness encryption (*cf.* Section 3.6), one-shot signatures (*cf.* Section 3.9), and the Shad-RABE protocol (*cf.* Section 5.2).

7.1 Generic Construction of RABE-PubVCD.

We construct a RABE-PubVCD protocol, denoted by $\prod^{\mathsf{RABE}}_{\mathsf{PubVCD}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt}, \mathsf{Delete}, \mathsf{Verify})$, from the following building blocks: a Shad-RABE scheme, specified by the algorithms $\prod_{\mathsf{Shad\text{-}RABE}} = \mathsf{Shad\text{-}RABE}.(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt}, \mathsf{SimKeyGen}, \mathsf{SimRegPK}, \mathsf{SimCorrupt}, \mathsf{SimCT}, \mathsf{Reveal})$, a witness encryption scheme $\prod_{\mathsf{we}} = \mathsf{WE}.(\mathsf{Encrypt}, \mathsf{Decrypt})$, and a one-shot signature scheme $\prod_{\mathsf{oss}} = \mathsf{OSS}.(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$. The detailed algorithms of RABE-PubVCD are specified below.

$\mathsf{Setup}(1^\lambda, 1^\tau)$:

- Generate $\mathsf{crs} \leftarrow \mathsf{Shad\text{-}RABE.Setup}(1^\lambda, 1^\tau)$.
- Output crs.

$\mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$:
- Generate $(\mathsf{srabe.pk}, \mathsf{srabe.sk}) \leftarrow \mathsf{Shad\text{-}RABE.KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$.
- Output $(\mathsf{pk}, \mathsf{sk}) := (\mathsf{srabe.pk}, \mathsf{srabe.sk})$.

$\mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$:
- Parse $\mathsf{pk} = \mathsf{srabe.pk}$.
- Compute $(\mathsf{srabe.mpk}, \mathsf{aux}') \leftarrow \mathsf{Shad\text{-}RABE.RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{srabe.pk}, P)$.
- Output $(\mathsf{mpk}, \mathsf{aux}') := (\mathsf{srabe.mpk}, \mathsf{aux}')$.

$\mathsf{Encrypt}(\mathsf{Sndr}(\mathsf{mpk}, X, \mu), \mathsf{Rcvr})$:
- Sndr parses $\mathsf{mpk} = \mathsf{srabe.mpk}$.
- Sndr generates $\mathsf{oss.crs} \leftarrow \mathsf{OSS.Setup}(1^\lambda)$ and sends it to Rcvr.
- Rcvr generates $(\mathsf{oss.pk}, \mathsf{oss.sk}) \leftarrow \mathsf{OSS.KeyGen}(\mathsf{oss.crs})$, sends $\mathsf{oss.pk}$ to Sndr, and stores $\mathsf{oss.sk}$.
- Sndr computes the witness encryption ciphertext $\mathsf{we.ct} \leftarrow \mathsf{WE.Encrypt}(1^\lambda, x, \mu)$, where

$$x := \{\exists \sigma \text{ s.t. } \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 0) = \top\}.$$

- Sndr computes $\mathsf{srabe.ct} \leftarrow \mathsf{Shad\text{-}RABE.Encrypt}(\mathsf{srabe.mpk}, X, \mathsf{we.ct})$.
- Sndr sends $\mathsf{srabe.ct}$ to Rcvr and outputs $\mathsf{vk} := (\mathsf{oss.crs}, \mathsf{oss.pk})$.
- Rcvr outputs $\mathsf{ct} := (\mathsf{srabe.ct}, \mathsf{oss.sk})$.

$\mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$:
- Parse $\mathsf{pk} = \mathsf{srabe.pk}$.
- Generate $\mathsf{srabe.hsk} \leftarrow \mathsf{Shad\text{-}RABE.Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{srabe.pk})$.
- Output $\mathsf{hsk} := \mathsf{srabe.hsk}$.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, X, \mathsf{ct})$:
- Parse $\mathsf{sk} = \mathsf{srabe.sk}$, $\mathsf{hsk} = \mathsf{srabe.hsk}$, and $\mathsf{ct} = (\mathsf{srabe.ct}, \mathsf{oss.sk})$.
- Compute $\sigma \leftarrow \mathsf{OSS.Sign}(\mathsf{oss.sk}, 0)$.
- Compute $\mathsf{we.ct}' \leftarrow \mathsf{Shad\text{-}RABE.Decrypt}(\mathsf{srabe.sk}, \mathsf{srabe.hsk}, X, \mathsf{srabe.ct})$.
- If $\mathsf{we.ct}' \in \{\bot, \mathsf{GetUpdate}\}$, output $\mathsf{we.ct}'$.
- Otherwise, compute and output $\mu \leftarrow \mathsf{WE.Decrypt}(\mathsf{we.ct}', \sigma)$.

$\mathsf{Delete}(\mathsf{ct})$:
- Parse $\mathsf{ct} = (\mathsf{srabe.ct}, \mathsf{oss.sk})$.
- Compute $\sigma \leftarrow \mathsf{OSS.Sign}(\mathsf{oss.sk}, 1)$.
- Output certificate $\mathsf{cert} := \sigma$.

$\mathsf{Verify}(\mathsf{vk}, \mathsf{cert})$:

- Parse $\mathsf{vk} = (\mathsf{oss.crs}, \mathsf{oss.pk})$ and $\mathsf{cert} = \sigma$.
- Compute and output $b \leftarrow \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 1)$.

Correctness. The correctness of decryption and verification follows directly from the correctness of $\prod_{\mathsf{we}}$ and $\prod_{\mathsf{oss}}$.

7.2 Proof of Security

We prove the security of our construction by showing the following theorem.

Theorem 5. *If the Shad-RABE scheme $\prod_{\mathit{Shad\text{-}RABE}}$ is secure, as discussed in Section 5.1, the witness encryption scheme $\prod_{\mathsf{WE}}$ satisfies extractable security, and the one-shot signature scheme $\prod_{\mathsf{OSS}}$ is secure, then the $\prod^{\mathit{RABE}}_{\mathit{PubVCD}}$ construction achieves certified deletion security with public verification.*

Proof. We establish security via a sequence of hybrid games. As a starting point, we present the real security experiment $\mathsf{Exp}^{\mathsf{RABE\text{-}PubVCD}}_{\mathcal{A}}(\lambda, b)$, which we denote as $\mathsf{Game}^{(b)}_0$.

- $\mathsf{Game}^{(b)}_0$: The experiment proceeds as follows:
  - The challenger runs $\mathsf{Shad\text{-}RABE.Setup}(1^\lambda, 1^{|\mathcal{U}|})$ to obtain the common reference string crs, initializes its internal state, and forwards crs to the adversary $\mathcal{A}$.
  - The adversary $\mathcal{A}$ adaptively issues registration and corruption queries, possibly involving both honest and malicious keys. The challenger responds while updating the master public key mpk and its internal state aux.
  - $\mathcal{A}$ submits a pair of challenge messages $(\mu^*_0, \mu^*_1) \in \mathcal{M}^2$ and a challenge attribute set $X^* \subseteq \mathcal{U}$. Then, the challenger and adversary engage in the interactive encryption protocol:
    (a) The challenger samples $\mathsf{oss.crs} \leftarrow \mathsf{OSS.Setup}(1^\lambda)$ and sends it to $\mathcal{A}$.
    (b) The adversary generates a key pair $(\mathsf{oss.pk}, \mathsf{oss.sk}) \leftarrow \mathsf{OSS.KeyGen}(\mathsf{oss.crs})$ and returns $\mathsf{oss.pk}$ to the challenger.
    (c) The challenger generates a ciphertext through witness encryption $\mathsf{we.ct} \leftarrow \mathsf{WE.Encrypt}(1^\lambda,\ x,\ \mu^*_b)$, where the statement x asserts the existence of a valid one-shot signature, *i.e.,*
    $$x = \{\exists \sigma : \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 0) = \top\}.$$
    (d) The challenger encrypts the witness ciphertext under the $\mathsf{Shad\text{-}RABE}$ scheme:
    $$\mathsf{srabe.ct} \leftarrow \mathsf{Shad\text{-}RABE.Encrypt}(\mathsf{mpk}, X^*, \mathsf{we.ct}).$$

(e) The challenger sends $\mathsf{srabe.ct}$ to $\mathcal{A}$ and stores the verification key $\mathsf{vk}^* := (\mathsf{oss.crs}, \mathsf{oss.pk})$.

– $\mathcal{A}$ returns a deletion certificate $\mathsf{cert} = \sigma$ to the challenger.

– The challenger verifies the certificate by computing $\mathsf{valid} \leftarrow \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 1)$. If $\mathsf{valid} = 0$, it returns $\perp$ to $\mathcal{A}$ and halts. Otherwise, it provides all honest secret keys to $\mathcal{A}$.

– Finally, the adversary outputs a bit $b' \in \{0, 1\}$.

- $\mathsf{Game}_1^{(b)}$: This game is identical to $\mathsf{Game}_0^{(b)}$, except that all operations involving the $\mathsf{Shad\text{-}ABE}$ scheme are performed using its simulation algorithms. Specifically:
  – The challenger replaces the real key generation procedure with its simulator, *i.e.,* honest key pairs are produced via $(\mathsf{pk}, \mathsf{B}) \leftarrow \mathsf{Shad\text{-}RABE.SimKeyGen}(\mathsf{crs}, \mathsf{aux}, P, \mathsf{B})$ instead of $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Shad\text{-}RABE.KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$.
  – The honest public keys are registered using the simulation procedure $(\widetilde{\mathsf{mpk}}, \widetilde{\mathsf{aux}}) \leftarrow \mathsf{Shad\text{-}RABE.SimRegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$ in place of the real registration algorithm $(\mathsf{mpk}, \mathsf{aux}') \leftarrow \mathsf{Shad\text{-}RABE.RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$.
  – The challenge ciphertext is produced via the simulation procedure $\widetilde{\mathsf{ct}} \leftarrow \mathsf{Shad\text{-}RABE.SimCT}(\widetilde{\mathsf{mpk}}, \mathsf{B}, X)$ rather than the original encryption algorithm $\mathsf{ct} \leftarrow \mathsf{Shad\text{-}RABE.Encrypt}(\mathsf{mpk}, X, \mu)$.
  – For corruption queries, the challenger answers with simulated secret keys, computed as $\widetilde{\mathsf{sk}} \leftarrow \mathsf{Shad\text{-}RABE.SimCorrupt}(\mathsf{crs}, \mathsf{pk}, \mathsf{B})$, instead of disclosing the actual secret keys.
  – In the deletion phase, once verification succeeds, the challenger derives the honest secret keys using $\widetilde{\mathsf{sk}} \leftarrow \mathsf{Shad\text{-}RABE.Reveal}(\mathsf{pk}, \mathsf{B}, \widetilde{\mathsf{ct}}, \mathsf{we.ct})$, ensuring they correspond to the correct challenge message.

Lemma 9. *If $\prod_{\mathsf{Shad\text{-}RABE}}$ is secure as defined in Section 5.1, then for every bit $b \in \{0, 1\}$,*

$$\left|\Pr[\mathsf{Game}_0^{(b)} = 1] - \Pr[\mathsf{Game}_1^{(b)} = 1]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Assume, for the sake of contradiction, that

$$\left|\Pr[\mathsf{Game}_0^{(b)} = 1] - \Pr[\mathsf{Game}_1^{(b)} = 1]\right|$$

is non-negligible. We then construct a reduction adversary $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ that breaks the security of the $\mathsf{Shad\text{-}RABE}$ scheme $\prod_{\mathsf{Shad\text{-}RABE}}$. Let $\mathcal{A}$ be the adversary that distinguishes between $\mathsf{Game}_0^{(b)}$ and $\mathsf{Game}_1^{(b)}$. The reduction $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ operates as follows:

- **Setup Phase:** Upon receiving a tuple (pk, P) from $\mathcal{A}$, $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ forwards the query to the $\mathsf{Shad\text{-}RABE}$ challenger and relays the response $(\mathsf{mpk}', \mathsf{aux}')$ to $\mathcal{A}$.

- **Registration Phase:** Upon receiving a tuple (pk, P) from $\mathcal{A}$, $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ forwards the query to the Shad-RABE challenger and relays the response $(\mathsf{mpk}', \mathsf{aux}')$ to $\mathcal{A}$.
  - Corrupted key registration: Whenever $\mathcal{A}$ submits a tuple (pk, P), the reduction $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ forwards this query to the Shad-RABE challenger and returns the challenger's reply $(\mathsf{mpk}', \mathsf{aux}')$ to $\mathcal{A}$.
  - Honest key registration: Upon receiving an access policy P, $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ forwards P to the Shad-RABE challenger and receives $(\mathsf{ctr}, \mathsf{mpk}', \mathsf{aux}', \mathsf{pk}_{\mathsf{ctr}})$, which it returns to $\mathcal{A}$.
  - Corruption query: When $\mathcal{A}$ issues a corruption query, $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ forwards the request to the Shad-RABE challenger and provides $\mathcal{A}$ with the simulated secret key returned.
- **Challenge Phase:** $\mathcal{A}$ submits challenge messages (μ_0^*, μ_1^*) and an attribute set X^*. The reduction proceeds as follows:
  - $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ executes $\mathsf{OSS.Setup}(1^\lambda)$ to obtain a common reference string $\mathsf{oss.crs}$, which is then given to $\mathcal{A}$.
  - $\mathcal{A}$ computes a key pair $(\mathsf{oss.pk}, \mathsf{oss.sk})$ via $\mathsf{OSS.KeyGen}(\mathsf{oss.crs})$ and reveals only the public key $\mathsf{oss.pk}$.
  - To generate the challenge ciphertext, $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ invokes witness encryption as $\mathsf{we.ct} \leftarrow \mathsf{WE.Encrypt}(1^\lambda, x, \mu_b^*)$, with the statement
$$x := \{\exists \sigma \text{ s.t. } \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 0) = \top\}.$$
  - $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ submits the challenge $(\mathsf{we.ct}, X^*)$ to the Shad-RABE challenger and obtains:
$$\mathsf{srabe.ct}^* = \begin{cases} \mathsf{Shad\text{-}RABE.Encrypt}(\mathsf{mpk}, X^*, \mathsf{we.ct}) & \text{if } b = 0, \\ \mathsf{Shad\text{-}RABE.SimCT}(\mathsf{mpk}, \mathsf{B}, X^*) & \text{if } b = 1, \end{cases}$$
$$\mathsf{reveal} = \begin{cases} \{\mathsf{sk}_i : \mathsf{pk}_i \in H\} & \text{if } b = 0, \\ \mathsf{Shad\text{-}RABE.Reveal}(\mathsf{pk}, \mathsf{B}, \mathsf{srabe.ct}^*, \mathsf{we.ct}) & \text{if } b = 1. \end{cases}$$
  - $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ forwards $\mathsf{srabe.ct}^*$ to $\mathcal{A}$ and stores the verification key $\mathsf{vk}^* := (\mathsf{oss.crs}, \mathsf{oss.pk})$.
- **Deletion Phase:** $\mathcal{A}$ submits a deletion certificate $\mathsf{cert} = \sigma$. The reduction verifies it as follows:
$$\mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 1).$$
  If the verification fails, $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ returns $\perp$ to $\mathcal{A}$. Otherwise, it sends the value reveal obtained from the Shad-RABE challenger.
- **Output:** $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ outputs the bit returned by $\mathcal{A}$.

When the internal bit maintained by the $\mathsf{Shad\text{-}RABE}$ challenger is fixed to 0, all computations are performed using the real algorithms of the scheme. In this case, $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ provides a perfect simulation of $\mathsf{Game}_0^{(b)}$ for the adversary $\mathcal{A}$. In contrast, when the internal bit is set to 1, the challenger consistently employs the corresponding simulation procedures, and the resulting transcript is distributed identically to $\mathsf{Game}_1^{(b)}$.

Since, by hypothesis, the adversary $\mathcal{A}$ can distinguish between $\mathsf{Game}_0^{(b)}$ and $\mathsf{Game}_1^{(b)}$ with non-negligible advantage, it follows that $\mathcal{B}_{\mathsf{Shad\text{-}RABE}}$ can also distinguish the real from the simulated execution in the $\mathsf{Shad\text{-}RABE}$ security experiment with the same advantage. This directly contradicts the assumed security of $\prod_{\mathsf{Shad\text{-}RABE}}$.

Lemma 10. *If the one-shot signature scheme $\prod_{\mathsf{OSS}}$ is secure and the witness encryption scheme $\prod_{\mathsf{WE}}$ achieves extractable security, then the distinguishing advantage between the two games is negligible, i.e.,*

$$\left|\Pr[\mathsf{Game}_1^{(0)} = 1] - \Pr[\mathsf{Game}_1^{(1)} = 1]\right| \leq \mathsf{negl}(\lambda).$$

Proof. We prove this lemma by analyzing the adversary's strategy with respect to the validity of the deletion certificate. Let $\mathsf{ValidCert}$ denote the event that adversary $\mathcal{A}$ provides a valid deletion certificate σ such that $\mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 1) = \top$. We can decompose the distinguishing advantage as:

$$\begin{aligned}
&\left|\Pr\left[\mathsf{Game}_1^{(0)} = 1\right] - \Pr\left[\mathsf{Game}_1^{(1)} = 1\right]\right| \\
=&\Big|\Pr\left[\mathsf{Game}_1^{(0)} = 1 \mid \mathsf{ValidCert}\right] \cdot \Pr\left[\mathsf{ValidCert}\right] + \Pr\left[\mathsf{Game}_1^{(0)} = 1 \mid \neg\mathsf{ValidCert}\right] \\
&\cdot \Pr\left[\neg\mathsf{ValidCert}\right] - \Pr\left[\mathsf{Game}_1^{(1)} = 1 \mid \mathsf{ValidCert}\right] \cdot \Pr\left[\mathsf{ValidCert}\right] \\
&- \Pr\left[\mathsf{Game}_1^{(1)} = 1 \mid \neg\mathsf{ValidCert}\right] \cdot \Pr\left[\neg\mathsf{ValidCert}\right]\Big| \\
\leq&\left|\Pr\left[\mathsf{Game}_1^{(0)} = 1 \mid \mathsf{ValidCert}\right] - \Pr\left[\mathsf{Game}_1^{(1)} = 1 \mid \mathsf{ValidCert}\right]\right| \cdot \Pr\left[\mathsf{ValidCert}\right].
\end{aligned}$$

The last inequality follows from the triangle inequality together with the observation that, whenever $\neg\mathsf{ValidCert}$ holds, the two games are indistinguishable since both return $\perp$ to $\mathcal{A}$.

Assume that the above distinguishing advantage is non-negligible. Then there exists an infinite subset $I \subseteq \mathbb{N}$ and a polynomial $p(\cdot)$ such that for every $\lambda \in I$,

$$\begin{aligned}
&\left|\Pr\left[\mathsf{Game}_1^{(0)} = 1 \mid \mathsf{ValidCert}\right] - \Pr\left[\mathsf{Game}_1^{(1)} = 1 \mid \mathsf{ValidCert}\right]\right| \geq \frac{1}{p(\lambda)} \\
&\Pr\left[\mathsf{ValidCert}\right] \geq \frac{1}{p(\lambda)}.
\end{aligned}$$

Let st represent $\mathcal{A}$'s internal state after producing the valid certificate, conditioned on $\mathsf{ValidCert}$.From the extractable security of witness encryption, there exists a QPT extractor $\mathcal{E}$ and a polynomial $q(\cdot)$ such that, for every $\lambda \in I$,

$$\Pr\left[\mathcal{E}(1^\lambda, x, \mathsf{st}) = \sigma' \wedge \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma', 0) = \top\right] \geq \frac{1}{q(\lambda)}.$$

We construct adversary $\mathcal{B}_{\mathsf{oss}}$ that breaks the security of the one-shot signature scheme as follows. $\mathcal{B}_{\mathsf{oss}}$ is given the common reference string $\mathsf{oss.crs}$ from its challenger. It then internally runs $\mathsf{Game}_1^{(b)}$ with adversary $\mathcal{A}$, embedding $\mathsf{oss.crs}$ into the simulated encryption protocol. Once $\mathcal{A}$ outputs a certificate σ, $\mathcal{B}_{\mathsf{oss}}$ invokes the extractor $\mathcal{E}$ to derive another certificate σ'. At this point, $\mathcal{B}_{\mathsf{oss}}$ produces the forgery consisting of $(\mathsf{oss.pk}, 1, \sigma, 0, \sigma')$.

The probability that $\mathcal{B}_{\mathsf{oss}}$ succeeds in this forgery is:

$$\begin{aligned}
&\Pr\left[\mu_0^* \neq \mu_1^* \wedge \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma', 0) = \top\right. \\
&\qquad\qquad \left. \wedge\ \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma, 1) = \top\right] \\
&= \Pr\left[\mathsf{ValidCert}\right] \cdot \Pr\left[\mathcal{E}(1^\lambda, x, \mathsf{st}) = \sigma' \wedge \mathsf{OSS.Verify}(\mathsf{oss.crs}, \mathsf{oss.pk}, \sigma', 0) = \top\right] \\
&\geq \frac{1}{p(\lambda)q(\lambda)}.
\end{aligned}$$

This contradicts the assumed security of $\prod_{\mathsf{oss}}$, thereby completing the proof of the lemma.

We thus conclude the proof of Theorem 5.

7.3 Instantiation of RABE-PubVCD

We now describe a lattice instantiation of our RABE-PubVCD. The construction follows the generic construction developed in the preceding sections. In particular, the instantiation of RABE-PubVCD relies on three fundamental building blocks, Shad-RABE protocol, WE protocol, OSS scheme.

- **Shad-RABE Instantiation:** We realize the Shad-RABE component from lattice-based primitives, as detailed in Section 5.3. The resulting instantiation is secure under the ℓ-succinct LWE, plain LWE, and equivocal LWE assumptions.
- **WE Instantiation:** For extractable witness encryption, candidate constructions have been proposed in the works of [VWW22, Tsa22] from the LWE assumption and its variants, offering significantly better efficiency than earlier $i\mathcal{O}$-based approaches.
- **OSS Instantiation:** One-shot signatures were originally introduced by Amos *et al.* [AGKZ20]. More recently, Shmueli *et al.* [SZ25] provided a standard-model construction secure under (sub-exponential) indistinguishability obfuscation ($i\mathcal{O}$) and LWE. Their work also establishes the first standard-model separation between classical and collapse-binding post-quantum commitments, and further introduces an oracle-based scheme with unconditional security.

Combining these ingredients yields a lattice-based instantiation of RABE-PubVCD. The resulting scheme enjoys post-quantum security under ℓ-succinct LWE, plain LWE, and equivocal LWE, thereby demonstrating the feasibility of certified-deletion functionalities in the lattice setting.

8 RABE with Privately Verifiable Certified Everlasting Deletion

In this section, we propose a construction of registered attribute-based encryption with privately verifiable certified everlasting deletion (RABE-PriVCED). The construction extends the RABE framework by modifying the encryption algorithm to produce a hybrid quantum-classical ciphertext. Concretely, the encryption procedure samples random strings x and θ, and prepares the quantum state $|x\rangle_\theta$, where each qubit is encoded in either the computational or the Hadamard basis according to θ_i. The plaintext is concealed using a one-time pad derived from the positions where $\theta_i = 0$. Subsequently, the masked message together with the basis information θ is encrypted classically, whereas the quantum state can be irreversibly destroyed during the deletion process. In the following subsection, we provide a generic construction of RABE-PriVCED.

8.1 Generic Construction of RABE-PriVCED

We construct a registered attribute-based encryption scheme with privately verifiable certified everlasting deletion (RABE-PriVCED) protocol, denoted by $\prod^{\mathsf{RABE}}_{\mathsf{PriVCED}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt}, \mathsf{Delete}, \mathsf{Ver})$, from the registered attribute-based encryption scheme $\prod_{\mathsf{RABE}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt})$. The detailed algorithms of RABE-PriVCED are specified below.

$\mathsf{Setup}(1^\lambda, 1^\tau)$:
- Generate $\mathsf{crs} \leftarrow \mathsf{RABE.Setup}(1^\lambda, 1^\tau)$.
- Output crs.

$\mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux})$:
- Generate $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{RABE.KeyGen}(\mathsf{crs}, \mathsf{aux})$.
- Output $(\mathsf{pk}, \mathsf{sk})$.

$\mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$:
- Compute $(\mathsf{mpk}, \mathsf{aux}') \leftarrow \mathsf{RABE.RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$.
- Output $(\mathsf{mpk}, \mathsf{aux}')$.

$\mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$:
- Compute $\mathsf{hsk} \leftarrow \mathsf{RABE.Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$.
- Output hsk.

Encrypt(mpk, X, b):

- Sample $x, \theta \leftarrow \{0,1\}^\lambda$ uniformly at random.
- Compute $\mathsf{ct}_{\mathsf{rabe}} \leftarrow \mathsf{RABE.Encrypt}\left(\mathsf{mpk}, X, \left(\theta, b \oplus \bigoplus_{i:\ \theta_i=0} x_i\right)\right)$.
- Output $\mathsf{ct} := (|x\rangle_\theta, \mathsf{ct}_{\mathsf{rabe}})$ and $\mathsf{vk} := (x, \theta)$.

Decrypt(sk, hsk,X, ct):

- Parse $\mathsf{ct} := (|x\rangle_\theta, \mathsf{ct}_{\mathsf{rabe}})$.
- Compute $(\theta, b') \leftarrow \mathsf{RABE.Decrypt}(\mathsf{sk}, \mathsf{hsk}, X, \mathsf{ct}_{\mathsf{rabe}})$.
- If RABE.Decrypt returns $\perp$ or GetUpdate, output the same.
- Measure $|x\rangle_\theta$ in the θ-basis to obtain x.
- Output $b = b' \oplus \bigoplus_{i:\theta_i=0} x_i$.

Delete(ct):

- Parse $\mathsf{ct} := (|x\rangle_\theta, \mathsf{ct}_{\mathsf{rabe}})$.
- Measure $|x\rangle_\theta$ in the Hadamard basis to obtain a string x'.
- Output $\mathsf{cert} := x'$.

Ver(vk, cert):

- Parse vk as (x, θ) and cert as x'.
- Output $\top$ if and only if $x_i = x'_i$ for all i such that $\theta_i = 1$.

Correctness of RABE-PriVCED. The correctness of the RABE-PriVCED protocol is ensured by the scheme's description together with the correctness of RABE.

It is important to note that the entire construction of RABE-PriVCED is based solely on the underlying RABE primitive, which can be instantiated from lattices. As discussed earlier, we employ the key-policy registered ABE scheme of Champion *et al.* [CHW25], supporting arbitrary bounded-depth circuit policies. Consequently, RABE-PriVCED admits a concrete instantiation based on standard lattice assumptions.

8.2 Proof of Security for RABE-PriVCED

In what follows, we provide a detailed proof of security for our RABE-PriVCED construction. The argument relies on the security of the underlying RABE scheme under the LWE assumption, together with the certified everlasting property established earlier.

Theorem 6. *If the underlying RABE scheme is secure under the LWE assumption, then the proposed RABE-PriVCED scheme achieves certified everlasting security.*

Proof. The theorem is proved via a standard argument of indistinguishability. Consider the following two experiments, parameterized by a bit $b \in \{0, 1\}$,

$$\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PriVCED}}(\lambda, b).$$

The adversary $\mathcal{A}$ interacts with the challenger in one of these experiments, and its goal is to distinguish whether it interacts with the $b = 0$ or $b = 1$ world. The security requires that these two experiments be computationally indistinguishable.

Formally, it suffices to show that the statistical distance

$$\mathsf{TD}\left(\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PriVCED}}(\lambda, 0),\ \mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PriVCED}}(\lambda, 1)\right) \leq \mathsf{negl}(\lambda),$$

for every non-uniform QPT adversary $\mathcal{A}$.

By Lemma 1, we already know that once deletion has been verified, the adversary cannot recover information about the challenge message with a non-negligible probability. This ensures that even after the deletion phase, the system retains everlasting confidentiality.

The semantic security of the underlying RABE construction, based on the LWE assumption, ensures that encryptions of two distinct challenge messages cannot be distinguished by any efficient adversary, as long as the challenge policy restrictions are satisfied.

To formalize the adversarial view, we define the distribution $Z_\lambda(\theta, b', X, \mathcal{A})$ as follows:

- The challenger first samples the common reference string

$$\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau),$$

  where τ denotes the policy parameter.
- Using the registration procedure, the challenger generates all registered keys and derives the master public key mpk.
- Finally, the challenger outputs

$$\big(\mathcal{A},\ \mathsf{RABE.Encrypt}(\mathsf{mpk}, X, (\theta, b'))\big),$$

  where (θ, b') encodes the challenge message and the policy P.

This distribution captures the complete view of the adversary during the challenge phase.

The class of adversaries $\mathcal{A}$ considered here consists of non-uniform quantum polynomial-time families

$$\{\mathcal{A}_\lambda,\ |\psi_\lambda\rangle\}_{\lambda \in \mathbb{N}},$$

which may include auxiliary quantum advice states $|\psi_\lambda\rangle$. These adversaries are constrained to respect the policy restrictions defined in the registered ABE security experiment (*i.e.,* they cannot query for decryption keys that would trivially satisfy the challenge policy).

By combining the above steps, we conclude that any distinguishing advantage of $\mathcal{A}$ in the above experiment is bounded by a negligible function in λ. Hence,

$$\mathsf{TD}\left(\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PriVCED}}(\lambda,0),\ \mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PriVCED}}(\lambda,1)\right) \leq \mathsf{negl}(\lambda).$$

This completes the proof.

9 RABE with Publicly Verifiable Certified Everlasting Deletion

In this section, we present a construction of registered attribute-based encryption with publicly verifiable certified everlasting deletion (RABE-PubVCED) scheme. Analogously to the privately verifiable setting described in the previous section, our construction is based on the RABE framework combined with a signature scheme that satisfies the one-time unforgeability for the BB84 states, with a modified encryption algorithm. Specifically, the encryption procedure samples random strings x and θ together with a signature key pair $(\mathsf{vk}, \mathsf{sigk})$, and prepares a quantum state $|\psi\rangle$ by applying a signing operation to the quantum one-time pad state $|x\rangle_\theta$. The plaintext message is masked using the bits at positions where $\theta_i = 1$. The signing key sigk, the basis information θ, and the masked message are then encrypted using the underlying RABE scheme, while the public verification key vk allows anyone to verify deletion certificates via the signature scheme.

9.1 Generic Construction of RABE-PubVCED

We present a registered attribute-based encryption scheme with publicly verifiable certified everlasting deletion (RABE-PubVCED), denoted as $\prod_{\mathsf{PubVCED}}^{\mathsf{RABE}}$ = RABE.(Setup, KeyGen,RegPK, Encrypt, Update, Decrypt, Delete, Verify) defined over an attribute universe $\mathcal{U}$, policy space $\mathcal{P}$, and message space $\{0,1\}$. Our construction utilizes two core cryptographic components: the registered attribute-based encryption scheme $\prod_{\mathsf{RABE}}$ = RABE.(Setup, KeyGen, RegPK, Encrypt, Update, Decrypt) and a deterministic signature scheme $\prod_{\mathsf{SIG}}$ = SIG.(Gen, Sign, Verify). The algorithms comprising RABE-PubVCED are described in detail below.

Setup$(1^\lambda, 1^\tau)$:
- Output $\mathsf{crs} \leftarrow \mathsf{RABE.Setup}(1^\lambda, 1^\tau)$.

KeyGen(crs, aux, P):
- Output $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{RABE.KeyGen}(\mathsf{crs}, \mathsf{aux}, P)$.

RegPK(crs, aux, pk, P):

– Output $(\mathsf{mpk}, \mathsf{aux}') \leftarrow \mathsf{RABE.RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}, P)$.

$\mathsf{Encrypt}(\mathsf{mpk}, \mathrm{X}, \mu)$:
– Generate $x, \theta \leftarrow \{0,1\}^\lambda$ and generate $(\mathsf{vk}, \mathsf{sigk}) \leftarrow \mathsf{SIG.Gen}(1^\lambda)$.
– Generate a quantum state $|\psi\rangle$ by applying the map $|\nu\rangle|0\dots0\rangle \rightarrow |\nu\rangle |\mathsf{SIG.Sign}(\mathsf{sigk}, \nu)\rangle$ to $|x\rangle_\theta \otimes |0\dots0\rangle$.
– Generate $\mathsf{rabe.ct} \leftarrow \mathsf{RABE.Encrypt}(\mathsf{mpk}, X, (\mathsf{sigk}, \theta, \mu \oplus \bigoplus_{i:\theta_i=1} x_i))$.
– Output vk and $\mathsf{ct} := (|\psi\rangle, \mathsf{rabe.ct})$.

$\mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$:
– Output $\mathsf{hsk} \leftarrow \mathsf{RABE.Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, \mathrm{X}, \mathsf{ct})$:
– Parse ct into a quantum state ρ and classical string rabe.ct.
– Compute $(\mathsf{sigk}, \theta, \beta) \leftarrow \mathsf{RABE.Decrypt}(\mathsf{sk}, \mathsf{hsk}, X, \mathsf{rabe.ct})$.
– If RABE.Decrypt outputs $\perp$ or GetUpdate, return the same output.
– Apply the map $|\nu\rangle|0\dots0\rangle \rightarrow |\nu\rangle|\mathsf{SIG.Sign}(\mathsf{sigk}, \nu)\rangle$ to ρ, measure the first λ qubits of the resulting state in Hadamard basis, and obtain $\bar{x}$.
– Output $\mu \leftarrow \beta \oplus \bigoplus_{i:\theta_i=1} \bar{x}_i$.

$\mathsf{Delete}(\mathsf{ct})$:
– Parse ct into a quantum state ρ and a classical string rabe.ct.
– Measure ρ in the computational basis and obtain x' and σ'.
– Output $\mathsf{cert} = (x', \sigma')$.

$\mathsf{Verify}(\mathsf{vk}, \mathsf{cert})$:
– Parse $(z', \sigma') \leftarrow \mathsf{cert}$.
– Output the result of $\mathsf{SIG.Verify}(\mathsf{vk}, z', \sigma')$.

Correctness of RABE-PubCED. The correctness of the RABE-PubCED protocol is guaranteed by the correctness of RABE and SIG.

9.2 Proof of Security

Theorem 7. *If the signature scheme* SIG *ensures one-time unforgeability for BB84 states and the underlying* RABE *scheme is secure under the LWE assumption, then the proposed RABE-PubVCED construction guarantees certified everlasting security.*

Proof. We begin by formalizing the adversary's view in the security experiment. Define $Z_\lambda(\mathsf{vk}, \mathsf{sigk}, \theta, \beta, \mathcal{A})$ to be the following efficient quantum process:

– The challenger first generates the common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\tau)$.
– Using the registration procedure, the challenger produces the master public key mpk and any auxiliary state aux that may be required for the remainder of the experiment.

– The challenger then constructs the challenge ciphertext under the attribute set X^* taken from the everlasting security experiment $\mathsf{Exp}_{\mathcal{A}}^{\mathsf{RABE\text{-}PubVCED}}(\lambda, b)$. Specifically, it computes

$$\mathsf{ct}^* \leftarrow \mathsf{RABE.Encrypt}(\mathsf{mpk}, X^*, (\mathsf{sigk}, \theta, \beta)).$$

– Finally, the process outputs the tuple

$$\big(\mathsf{crs}, \mathsf{mpk}, \mathsf{vk}, \mathsf{ct}^*\big).$$

We now analyze the indistinguishability of the adversary's view under different distributions of the challenge inputs. Let $\mathcal{A}$ be any non-uniform QPT adversary. For any key pair $(\mathsf{vk}, \mathsf{sigk})$ of the signature scheme SIG, for any challenge string $\theta \in \{0,1\}^\lambda$, bit $\beta \in \{0,1\}$, and for any efficiently samplable auxiliary quantum state $|\psi\rangle_{\mathsf{A,C}}$ over registers A and C, the following two indistinguishability conditions hold due to the semantic security of the underlying RABE scheme.

First, replacing the secret signing key sigk with a uniformly random string of the same length ℓ_{sigk} does not alter the adversary's distinguishing advantage by more than a negligible function:

$$\Big| \Pr\left[\mathcal{A}(Z_\lambda(\mathsf{vk}, \mathsf{sigk}, \theta, \beta, \mathsf{A}), \mathsf{C}) = 1\right] - \Pr\left[\mathcal{A}(Z_\lambda(\mathsf{vk}, 0^{\ell_{\mathsf{sigk}}}, \theta, \beta, \mathsf{A}), \mathsf{C}) = 1\right] \Big| \leq \mathsf{negl}(\lambda).$$

Second, replacing the challenge string θ with the all-zero string 0^λ also leaves the adversary's view unchanged up to negligible statistical distance:

$$\Big| \Pr\left[\mathcal{A}(Z_\lambda(\mathsf{vk}, \mathsf{sigk}, \theta, \beta, \mathsf{A}), \mathsf{C}) = 1\right] - \Pr\left[\mathcal{A}(Z_\lambda(\mathsf{vk}, \mathsf{sigk}, 0^\lambda, \beta, \mathsf{A}), \mathsf{C}) = 1\right] \Big| \leq \mathsf{negl}(\lambda).$$

The first bound guarantees that the adversary cannot exploit the actual signing key sigk, since it is computationally indistinguishable from random in the ciphertext view. The second bound ensures that the particular challenge string θ likewise provides no distinguishing advantage. Together, these facts imply that the adversary's view in the challenge experiment is independent of the challenge bit β.

Since the adversary's distinguishing advantage is bounded by a negligible function, it follows that the challenge experiments with $b = 0$ and $b = 1$ are computationally indistinguishable. Hence, by invoking Lemma 2, we conclude that the construction achieves certified everlasting security.

9.3 Instantiation of RABE-PubVCED

We present a concrete lattice-based instantiation of our $\mathsf{RABE\text{-}PubVCED}$ construction that attains post-quantum security. Our instantiation builds on two fundamental lattice-based primitives, both grounded in well-established and extensively studied lattice assumptions:

- **Registered ABE Component:** As mentioned earlier, we adopt the key-policy registered ABE scheme of Champion *et al.* [CHW25], which supports arbitrary bounded-depth circuit policies. This scheme achieves semantic security under the falsifiable ℓ-succinct LWE assumption in the random oracle model, and produces succinct ciphertexts whose size is independent of the attribute length.
- **Signature Component:** For the classical deterministic signature scheme satisfying one-time unforgeability for BB84, we follow the construction framework established by Kitagawa *et al.* [KNY23]. Their approach fundamentally relies on the existence of a secure one-way function as the underlying cryptographic primitive. To achieve post-quantum security, we instantiate this one-way function using lattice-based constructions derived from cyclic lattice structures developed in [Mic07]. This construction preserves the essential one-time unforgeability properties required for our quantum deletion mechanism while grounding the security analysis in the computational hardness of lattice problems, specifically the Learning With Errors and Short Integer Solution (SIS) assumptions that are believed to remain secure against quantum attacks.

Building on these components, we present a lattice-based instantiation of RABE-PubVCED. The scheme achieves post-quantum security under the ℓ-succinct LWE and cyclic lattice assumptions, thereby demonstrating the feasibility of certified-deletion functionalities in the lattice setting.

Acknowledgements

This work is supported in part by the Anusandhan National Research Foundation (ANRF), Department of Science and Technology (DST), Government of India, under File No. EEQ/2023/000164 and the Information Security Education and Awareness (ISEA) Project Phase-III initiatives of the Ministry of Electronics and Information Technology (MeitY) under Grant No. F.No. L-14017/1/2022-HRD.

References

AAH+25. Kyoichi Asano, Nuttapong Attrapadung, Keisuke Hara, Keitaro Hashimoto, and Yohei Watanabe. Key revocation in Registered Attribute-Based Encryption. In *IACR International Conference on Public-Key Cryptography*, pages 102–133. Springer, 2025. https://link.springer.com/chapter/10.1007/978-3-031-91826-1_4.

AGKZ20. Ryan Amos, Marios Georgiou, Aggelos Kiayias, and Mark Zhandry. One-shot signatures and applications to hybrid quantum/classical authentication. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 255–268, 2020. https://dl.acm.org/doi/10.1145/3357713.3384304.

ARS20. Behzad Abdolmaleki, Sebastian Ramacher, and Daniel Slamanig. Lift-and-shift: obtaining simulation extractable subversion and updatable snarks generically. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1987–2005, 2020. https://dl.acm.org/doi/10.1145/3372297.3417228.

BB14. Charles H Bennett and Gilles Brassard. Quantum cryptography: Public key distribution and coin tossing. *Theoretical computer science*, 560:7–11, 2014. https://doi.org/10.1016/j.tcs.2014.05.025.

BDGM20. Zvika Brakerski, Nico Döttling, Sanjam Garg, and Giulio Malavolta. Factoring and pairings are not necessary for iO: Circular-secure LWE suffices. *Cryptology ePrint Archive*, 2020. https://eprint.iacr.org/2020/1024.

BDJ+24. Pedro Branco, Nico Döttling, Abhishek Jain, Giulio Malavolta, Surya Mathialagan, Spencer Peters, and Vinod Vaikuntanathan. Pseudorandom obfuscation and applications. *Cryptology ePrint Archive*, 2024. https://eprint.iacr.org/2024/1742.

BF01. Dan Boneh and Matt Franklin. Identity-based encryption from the weil pairing. In *Annual international cryptology conference*, pages 213–229. Springer, 2001. https://doi.org/10.1007/3-540-44647-8_13.

BGG+23. James Bartusek, Sanjam Garg, Vipul Goyal, Dakshita Khurana, Giulio Malavolta, Justin Raizes, and Bhaskar Roberts. Obfuscation and outsourced computation with certified deletion. *IACR Cryptology ePrint Archive*, 2023:265, 2023. https://ntt-research.com/wp-content/uploads/2023/08/Obfuscation-and-outsourced-computation-with-certified-deletion.pdf.

BI20. Anne Broadbent and Rabib Islam. Quantum encryption with certified deletion. In *Theory of Cryptography Conference*, pages 92–122. Springer, 2020. https://link.springer.com/chapter/10.1007/978-3-030-64381-2_4.

BK23. James Bartusek and Dakshita Khurana. Cryptography with certified deletion. In *Annual International Cryptology Conference*, pages 192–223. Springer, 2023. https://link.springer.com/chapter/10.1007/978-3-031-38554-4_7.

BR24. James Bartusek and Justin Raizes. Secret sharing with certified deletion. In *Annual International Cryptology Conference*, pages 184–214. Springer, 2024. https://link.springer.com/chapter/10.1007/978-3-031-68394-7_7.

BS20. Nir Bitansky and Omri Shmueli. Post-quantum zero knowledge in constant rounds. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 269–279, 2020. https://dl.acm.org/doi/10.1145/3357713.3384324.

BUW24. Chris Brzuska, Akin Unal, and Ivy KY Woo. Evasive LWE assumptions: Definitions, classes, and counterexamples. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 418–449. Springer, 2024. https://link.springer.com/chapter/10.1007/978-981-96-0894-2_14.

CC09. Melissa Chase and Sherman SM Chow. Improving privacy and security in multi-authority attribute-based encryption. In *Proceedings of the 16th ACM conference on Computer and communications security*, pages 121–130, 2009. https://doi.org/10.1145/1653662.1653678.

Cha07. Melissa Chase. Multi-authority attribute based encryption. In *Theory of cryptography conference*, pages 515–534. Springer, 2007. https://doi.org/10.1007/978-3-540-70936-7_28.

CHSS02. Liqun Chen, Keith Harrison, David Soldera, and Nigel P Smart. Applications of multiple trust authorities in pairing based cryptosystems. In *International Conference on Infrastructure Security*, pages 260–275. Springer, 2002. https://link.springer.com/chapter/10.1007/3-540-45831-x_18.

CHW25. Jeffrey Champion, Yao-Ching Hsieh, and David J Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. *Cryptology ePrint Archive*, 2025. https://link.springer.com/chapter/10.1007/978-3-032-01881-6_1.

CLW25. Valerio Cini, Russell WF Lai, and Ivy KY Woo. Lattice-based obfuscation from NTRU and Equivocal LWE. *Cryptology ePrint Archive*, 2025. https://link.springer.com/chapter/10.1007/978-3-032-01907-3_2.

Coo00. Stephen Cook. The P versus NP problem. *Clay Mathematics Institute*, 2, 2000. https://www.claymath.org/wp-content/uploads/2022/06/pvsnp.pdf.

DKW21a. Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority ABE for DNFs from LWE. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 177–209. Springer, 2021. https://doi.org/10.1007/978-3-030-77870-5_7.

DKW21b. Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority ABE for NCˆ1 from computational-BDH. *Cryptology ePrint Archive*, 2021. https://dl.acm.org/doi/abs/10.1007/s00145-023-09445-7.

GGSW13a. Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, pages 467–476, 2013. https://dl.acm.org/doi/abs/10.1145/2488608.2488667.

GGSW13b. Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, pages 467–476, 2013. https://doi.org/10.1145/2488608.2488667.

GHMR18. Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, and Ahmadreza Rahimi. Registration-based encryption: removing private-key generator from IBE. In *Theory of Cryptography Conference*, pages 689–718. Springer, 2018. https://dl.acm.org/doi/abs/10.1007/978-3-030-03807-6_25.

GKM+18. Jens Groth, Markulf Kohlweiss, Mary Maller, Sarah Meiklejohn, and Ian Miers. Updatable and universal common reference strings with applications to zk-snarks. In *Annual International Cryptology Conference*, pages 698–728. Springer, 2018. https://doi.org/10.1007/978-3-319-96878-0_24.

GKP+13. Shafi Goldwasser, Yael Tauman Kalai, Raluca Ada Popa, Vinod Vaikuntanathan, and Nickolai Zeldovich. How to run turing machines on encrypted data. In *Annual Cryptology Conference*, pages 536–553. Springer, 2013. https://doi.org/10.1007/978-3-642-40084-1_30.

GP21. Romain Gay and Rafael Pass. Indistinguishability obfuscation from circular security. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, pages 736–749, 2021. https://dl.acm.org/doi/10.1145/3406325.3451070.

GPSW06. Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 89–98, 2006. https://doi.org/10.1145/1180405.1180418.

GPV08. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 197–206, 2008. https://doi.org/10.1145/1374376.1374407.

GVW15. Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Attribute-based encryption for circuits. *Journal of the ACM (JACM)*, 62(6):1–33, 2015. https://dl.acm.org/doi/10.1145/2824233.

HBB99. Mark Hillery, Vladimír Bužek, and André Berthiaume. Quantum secret sharing. *Physical Review A*, 59(3):1829, 1999. https://doi.org/10.1103/PhysRevA.59.1829.

HKM+24. Taiga Hiroka, Fuyuki Kitagawa, Tomoyuki Morimae, Ryo Nishimaki, Tapas Pal, and Takashi Yamakawa. Certified everlasting secure collusion-resistant functional encryption, and more. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 434–456. Springer, 2024. https://doi.org/10.1007/978-3-031-58734-4_15.

HLWW23. Susan Hohenberger, George Lu, Brent Waters, and David J Wu. Registered attribute-based encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 511–542. Springer, 2023. https://link.springer.com/chapter/10.1007/978-3-031-30620-4_17.

HMNY21. Taiga Hiroka, Tomoyuki Morimae, Ryo Nishimaki, and Takashi Yamakawa. Quantum encryption with certified deletion, revisited: Public key, attribute-based, and classical communication. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 606–636. Springer, 2021. https://doi.org/10.1007/978-3-030-92062-3_21.

HMNY22. Taiga Hiroka, Tomoyuki Morimae, Ryo Nishimaki, and Takashi Yamakawa. Certified everlasting zero-knowledge proof for QMA. In *Annual International Cryptology Conference*, pages 239–268. Springer, 2022. https://doi.org/10.1007/978-3-031-15802-5_9.

KG10. Aniket Kate and Ian Goldberg. Distributed private-key generators for identity-based cryptography. In *International Conference on Security and Cryptography for Networks*, pages 436–453. Springer, 2010. https://doi.org/10.1007/978-3-642-15317-4_27.

KNY23. Fuyuki Kitagawa, Ryo Nishimaki, and Takashi Yamakawa. Publicly verifiable deletion from minimal assumptions. In *Theory of Cryptography Conference*, pages 228–245. Springer, 2023. https://doi.org/10.1007/978-3-031-48624-1_9.

LCL+25. Jiguo Li, Shaobo Chen, Yang Lu, Jianting Ning, Jian Shen, and Yichen Zhang. Revocable registered attribute-based encryption with user deregistration. *IEEE Internet of Things Journal*, 2025. https://doi.org/10.1109/JIOT.2025.3573050.

LCLS08. Huang Lin, Zhenfu Cao, Xiaohui Liang, and Jun Shao. Secure threshold multi authority attribute based encryption without a central authority. In *International Conference on Cryptology in India*, pages 426–436. Springer, 2008. https://doi.org/10.1016/j.ins.2010.03.004.

LW11. Allison Lewko and Brent Waters. Decentralizing attribute-based encryption. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 568–588. Springer, 2011. https://doi.org/10.1007/978-3-642-20465-4_31.

LWW25. George Lu, Brent Waters, and David J Wu. Multi-authority registered attribute-based encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2025. https://link.springer.com/chapter/10.1007/978-3-031-91131-6_1.

Mic07. Daniele Micciancio. Generalized compact knapsacks, cyclic lattices, and efficient one-way functions. *computational complexity*, 16(4):365–411, 2007. https://link.springer.com/article/10.1007/s00037-007-0234-9.

MKE08. Sascha Müller, Stefan Katzenbeisser, and Claudia Eckert. Distributed attribute-based encryption. In *International Conference on Information Security and Cryptology*, pages 20–36. Springer, 2008. https://doi.org/10.1007/978-3-642-00730-9_2.

Por22. Alexander Poremba. Quantum proofs of deletion for learning with errors. *arXiv preprint arXiv:2203.01610*, 2022. https://arxiv.org/abs/2203.01610.

PS08. Kenneth G Paterson and Sriramkrishnan Srinivasan. Security and anonymity of identity-based encryption with multiple trusted authorities. In *International Conference on Pairing-Based Cryptography*, pages 354–375. Springer, 2008. https://link.springer.com/chapter/10.1007/978-3-540-85538-5_23.

PST25. Tapas Pal, Robert Schädlich, and Erkan Tairi. Registered functional encryption for pseudorandom functionalities from lattices: Registered abe for unbounded depth circuits and turing machines, and more. *Cryptology ePrint Archive*, 2025. https://eprint.iacr.org/2025/967.

Reg09. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009. https://doi.org/10.1145/1568318.1568324.

RW15. Yannis Rouselakis and Brent Waters. Efficient statically-secure large-universe multi-authority attribute-based encryption. In *International Conference on Financial Cryptography and Data Security*, pages 315–332. Springer, 2015. https://link.springer.com/chapter/10.1007/978-3-662-47854-7_19.

SW05. Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 457–473. Springer, 2005. https://doi.org/10.1007/11426639_27.

SZ25. Omri Shmueli and Mark Zhandry. On One-Shot Signatures, Quantum vs. Classical Binding, and Obfuscating Permutations. In *Annual International Cryptology Conference*, pages 350–383. Springer, 2025. https://dl.acm.org/doi/10.1007/978-3-032-01878-6_12.

Tsa22. Rotem Tsabary. Candidate witness encryption from lattice techniques. In *Annual International Cryptology Conference*, pages 535–559. Springer, 2022. https://link.springer.com/chapter/10.1007/978-3-031-15802-5_19.

Tur36. Alan Mathison Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1936. https://doi.org/10.1112/plms/s2-42.1.230.

VWW22. Vinod Vaikuntanathan, Hoeteck Wee, and Daniel Wichs. Witness encryption and null-IO from evasive LWE. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 195–221. Springer, 2022. https://link.springer.com/chapter/10.1007/978-3-031-22963-3_7.

Wie83. Stephen Wiesner. Conjugate coding. *ACM Sigact News*, 15(1):78–88, 1983. https://doi.org/10.1145/1008908.1008920.

WW21. Hoeteck Wee and Daniel Wichs. Candidate obfuscation via oblivious LWE sampling. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 127–156. Springer, 2021. https://doi.org/10.1007/978-3-030-77883-5_5.

WW25. Hoeteck Wee and David J Wu. Unbounded distributed broadcast encryption and registered abe from succinct lwe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 204–235. Springer, 2025. https://link.springer.com/chapter/10.1007/978-3-032-01881-6_7.

WWW22. Brent Waters, Hoeteck Wee, and David J Wu. Multi-authority ABE from lattices without random oracles. In *Theory of Cryptography Conference*, pages 651–679. Springer, 2022. https://link.springer.com/chapter/10.1007/978-3-031-22318-1_23.

WZ09. William K Wootters and Wojciech H Zurek. The no-cloning theorem. *Physics Today*, 62(2):76–77, 2009. https://doi.org/10.1063/1.3086114.

ZZC+25. Ziqi Zhu, Kai Zhang, Zhili Chen, Junqing Gong, and Haifeng Qian. Black-box registered ABE from lattices. *Cryptology ePrint Archive*, 2025. https://eprint.iacr.org/2025/051.

ZZGQ23. Ziqi Zhu, Kai Zhang, Junqing Gong, and Haifeng Qian. Registered ABE via predicate encodings. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 66–97. Springer, 2023. https://link.springer.com/chapter/10.1007/978-981-99-8733-7_3.